Provenance-Preserving Conservation Calculus and Non-Free Continuity

Whole-Event Ledgers, Observer Safety, and Well-Founded Settlement

Jian Gao (高健)

Independent Researcher · innersummer@hotmail.com · ORCID 0009-0009-5002-1655

Formal Verification and Edition Note. This paper rests on a strictly stratified Lean 4 [7] trusted-kernel dependent-type foundation: lawful world admission injectivity, positive total-reality classification, complete event-inventory bijective admission and terminal exhaustion (WA1–WA3), rowwise total disposition and transported remainders (Theorems 3.4–3.5), abstract observer-fibre safety and hidden null witnesses (Theorem 4.2), full parent/child restructuring coverage (Theorem 5.4, WA5), law-free paid macro-step well-foundedness and two-key final settlement (Main Theorems 10.3 and 11.4, WA7–WA8), and categorical initial-coface unique factorization with shared first-write revision continuity (Proposition 12.2, Theorem 12.3, WA9) all reside strictly in a **zero-axiom constructive kernel** (T0, `#print axioms = []`); capacity-shortage positive obstruction and source-bound maintenance debit ceilings (`ResponsibilityCapacityKernel`, Theorem 3.7) rely solely on decidable integer arithmetic (`propext` and `Quot.sound`, without invoking `Classical.choice`); list-witness extraction in the three lifecycle laws (Theorem 3.2), concrete scope-observer full-trace redirection (Example 4.3), and downstream modules involving quotient completion, complexified tensor observations, long-window operator analysis, and `Finsupp` finite-support enumeration have their axiom closures strictly bounded by Lean 4's three standard axioms (T1: propositional extensionality `propext`, `Classical.choice`, and quotient soundness `Quot.sound`), with zero incomplete placeholders (`sorry`), zero `unsafe` declarations, and zero `partial` dependencies. Every core positive theorem is paired with a formal negative control (event-trimming counterexample, observation-blindness counterexample, wash-restructuring counterexample, equal-budget refutation, zero-budget empty-terminal counterexample, and nonempty-group anti-omission/anti-duplication gate). This work is the reconstructed second edition (v2) of the published v1 preprint (*Book-Balancing Magic Has Nowhere to Hide: Source-Native Accounting and Actual Continuation*, series anchor *Source-Native Whole-Ledger Accounting*, DOI: 10.5281/zenodo.23437454), unifying the complete **WA1–WA45** evidence chain, incorporating the fourth kernel on capacity-shortage positive obstruction (`ResponsibilityCapacityKernel`), and establishing cross-packet paid-method recovery, primitive-by-primitive relation-cochain replay, zero-evaluation free constant-table relation-certificate inversion, and step-grouped prefix-preserving continuation (WA41–WA45).

Abstract

Why does a zero scalar balance fail to certify discharged obligation, and why can capacity exhaustion never license automatic debt forgiveness? Moving beyond the difference-group treatment of scalar and vector balances in double-entry bookkeeping, separation logic (which isolates disjoint local heaps), and scalar potential-based amortised resource analysis, we establish a **Provenance-Preserving Conservation Calculus** over complete event inventories and dependent open-responsibility fibres. With a source and lawful compilation contract fixed, every candidate attempting event trimming, cross-source receipt borrowing, coarse-observation concealment, equal-budget wash migration, capacity-shortage pseudo-discharge, or zero-budget pseudo-settlement reduces to the empty type. Bijective event-inventory admission excludes cofinal erasure and unauthorized insertion while enforcing ordinary outgoing-edge exhaustion at local terminals (WA1–WA3). The lifecycle kernel establishes three no-free conservation laws (no un-admitted birth, no un-admitted or un-reopened activation, no un-receipted disappearance), full parent/child restructuring coverage with strict child budget descent, and a capacity-obstruction gate proving that insufficient capacity (`available < demanded`) generates a positive obstruction witness preserving all affected dependencies and protected boundaries rather than discharging obligations, while every maintenance debit is bound to its source occurrence, slot, and lineage under an exact initial-budget ceiling (WA4–WA5). At the observation and multi-stage settlement layers, we characterize consumer-relative observer safety, exhibit a nonzero global integral boundary class annihilated by every finite-stage complexified determinant observation to rule out any linear or nonlinear recovery map (WA6, WA11–WA12), prove law-free well-foundedness and irreflexivity of $\leq$; $<$; $\leq$ paid macro-steps (WA7), establish that final settlement requires both an original faithful terminal receipt and an empty same-debt fibre in the actual successor (WA8, WA10), and prove shared first-write continuity for migrated debts and new directions across language revisions (WA9).

At the dynamic continuation and inversion layers, the ledger advances endogenously without external scheduling. At every natural-number cofinal admission stop, the literal syntactic residual equals the paid relation boundary across all allowed action words and sites, generates the next native request, executes strict same-debt syntax debit from positive credit without budget refill, and completes its suffix to deliver the incremental effect, receipted answer, prior/born whole-ledger destinations, and canonical successor together; zero numerical effect still pays syntax cost, and conditional long-window analytic computations carry four prior execution traces intact into inverse-residual correction frames (WA13–WA40). Furthermore, double-negation extraction recovers the paid abstract syntax tree to apply any pre-fixed additive homomorphism across cofinal stops with full query traces recorded in born stock, while primitive-by-primitive word replay updates the chain morphism τ_1 and cochain differential $d^{0,1}$ in lockstep (WA41–WA42); free constant-table updates retain explicit relation corrections so that even zero-sum numerical cancellations invert losslessly via finite integer relation certificates to restore the exact next constant table and paid value (WA43); and next-batch requests partition into contiguous per-step slices that strictly reject omitting or duplicating any nonempty group while horizon extension locks the sealed paid prefix and computes only from the reached tail state (WA44–WA45).

Keywords: provenance-preserving conservation calculus; whole-event ledger; capacity-shortage obstruction; consumer-relative observer safety; well-founded two-key settlement; paid syntactic continuation; zero-evaluation certificate inversion; step-grouped batch accounting

Contents

Abstract 2

1 Introduction: Why Does a Zero Scalar Balance Fail to Certify Settlement? 4

2 Whole-Event Inventory Admission and Anti-Shell-Borrowing Theorems 11

3 Three Lifecycle Conservation Laws, Rowwise Total Disposition, and Capacity-Shortage Obstruction 15

4 Consumer-Relative Observer Safety and Four-Way Residual Dispositions 20

5 Restructuring Conservation, Exclusion of Phantom Rows, and Same-Root Certificate Lifting 21

6 End-to-End Payment and Two-Key Settlement on the Target-Six Arithmetic Partition 24

7 Owner-Free Endogenous Computation and Exact Syntactic Accounting 26

8 Order-Preserving Born-Stock Injection, Total-Value Writeback, and Cofinal Stop Coverage 29

9 Endogenous Requests from All-Future Residuals and Strict Same-Debt Completion 32

10 Law-Free Well-Foundedness of Paid Macro-Steps 37

11 The Two-Key Final Settlement Theorem 38

12 Shared First-Write Continuity for Old Debts and New Responsibilities Across Language Revisions 41

13 Total Blindness of Finite Complexified Observations and Unrecoverability of Global Integral Classes 43

14 Full-Trace Relay Across Long Analytic Windows and Inverse-Residual Corrections 46

15 Cross-Packet Paid Methods, Zero-Evaluation Relation-Certificate Inversion, and Step-Grouped Continuation 51

16 Five-Pillar Dialogue with Classical Traditions and Conclusion 58

Appendix A Shared Theorems with the Process-Algebra Companion and Independent Proofs 63

Appendix B Core Ledger Object Contracts, Capacity Gates, and Units 64

Code and Data Availability 65

References 66

1 Introduction: Why Does a Zero Scalar Balance Fail to Certify Settlement?

Writing zero on the bottom line of a balance sheet or system dashboard is effortless; what is genuinely hard is proving **whether what vanished was the unfulfilled obligation itself, or merely the column that displayed it.**

Across distributed auditing, resource-aware program verification, institutional ledgers, and scientific attribution, five recurring book-balancing shortcuts masquerade as genuine fulfilment: (i) trimming unfinished exceptional branches or cofinal events and declaring closure on the surviving subset; (ii) borrowing a valid receipt issued by source A to clear a state of source B whose lower representation happens to look identical; (iii) projecting through coarse summaries or complex/floating-point scalars so that $+v$ and $-v$ cancel or torsion classes vanish into "no net change"; (iv) shuffling an unpaid obligation across accounts via $b \to b$ equal-budget transfers, renamings, or split/merge rearrangements and calling the motion progress; and (v) treating exhausted compute, memory, or carrying capacity—or a numerically zero budget—as automatic forgiveness of the underlying obligation.

Over the past four decades, five foundational traditions have forged powerful tools for individual facets of this problem, yet each halts before a sharp structural cliff:

- **Mathematical double-entry bookkeeping, event accounting, and the judgment-proof problem**: Ellerman formalizes Pacioli double-entry bookkeeping via the abelian difference group $[x \,//\, y] \in \mathcal{G}$ of cross-sum equivalence classes [1], yet $\mathcal{G}$ quotients equal debits and credits directly into the identity $[x \,//\, x] = [0 \,//\, 0]$, erasing occurrence lineage and unsettled obligation identity; McCarthy's REA (Resources–Events–Agents) model rightly replaces aggregated ledger accounts with primary economic events and give–get duality in shared databases [8], but remains an entity–relationship schema without proof-carrying remainder folds over infinite open fibres or formal settlement gates; and in law and economics, Shavell's *judgment-proof problem* demonstrates that whenever an obligor's available assets fall below potential liability, actual recovery is limited by the available assets [9]—a financial analogue of the recovery gap that can arise when compute or budgets are exhausted.
- **Type-theoretic identity, proof-carrying code, and substructural, amortised, and quantitative logics**: Intensional Σ-types [10] retain witnesses, whereas Homotopy Type Theory's Univalence Axiom [11] (Axiom 2.10.3) promotes type equivalence within a universe to equality $(A =_{\mathcal{U}} B) \simeq (A \simeq B)$—modeling lower ledger readouts as unindexed carrier types and transporting port outcomes across equivalences would conflate isomorphic states devoid of shared causal provenance; Necula and Lee's Proof-Carrying Code (PCC) [2] verifies that code satisfies a receiver's safety policy before execution without locking multi-step debt lineage; Girard's linear logic [12] governs individual variables in a fixed context and Reynolds's classical separation logic frame rule [3] assumes disjoint local heaps $h_1 * h_2$, neither folding an entire coupled open fibre without variable-by-variable threading or ad-hoc ghost-state protocols; Hofmann and Jost's Automatic Amortised Resource Analysis (AARA) [13] and Atkey's amortised separation logic [4] assign scalar potentials via linear programming or local credit (including dynamic `acquire` branching) without emitting a whole-ledger runtime obstruction witness; and Atkey's Quantitative Type Theory (QTT) [14] proves "Zero needs

nothing" (Lemma 2.3: $\Gamma \vdash M \overset{0}{:} S \implies 0\Gamma = \Gamma$) for compile-time erased terms, which cannot be extrapolated to runtime ledgers where a zero-numerical-effect ($\epsilon = 0$) residual request carries positive syntactic credit and executes concrete reduction steps that must be paid and recorded.

- **Event structures, state-refinement auxiliary variables, and characteristic-zero observability**: Winskel models concurrency via causal and conflict relations over events [15], while Abadi and Lamport show how extensional state-space refinement mappings can require auxiliary **history variables and prophecy variables** [16]—precisely because bare states erase arrival lineage and pending future residuals; meanwhile, Kalman's state-space observability criterion over characteristic-zero fields ($\mathbb{R}/\mathbb{C}$) [17] suffers total blindness on $\mathbb{Z}$-torsion responsibility cokernels, where invertible block determinants annihilate nonzero integral boundary classes under $\mathbb{C} \otimes_{\mathbb{Z}} -$ across all infinitely many finite stages.
- **Disjunctive transition invariants and fixed-language belief contraction**: Podelski and Rybalchenko's Transition Invariants (Theorem 1) [18] characterize global program termination via inclusion of the reachable transitive closure in a finite union of well-founded relations $R^+ \cap (\mathrm{Acc} \times \mathrm{Acc}) \subseteq T_1 \cup \cdots \cup T_n$, yet bare state-pair relations neither lock same-debt identity across split/merge carrier migrations nor separate stalled zero-descent states from two-key settlement; and classical AGM theory change [19], designed for **defeasible belief revision inside a fixed propositional language** via an external selection function ($K \dot{-} p \subseteq K$), would write off historical debts if applied to an obligation-bearing ledger whose language suffers expressive bankruptcy, unable to expand language coordinates in zero choice axioms while carrying unsettled debts intact.
- **Provenance semirings, static λ-differentiation, and $\mathbb{Z}$-set stream calculus**: Green, Karvounarakis, and Tannen's $\mathbb{N}[X]$ provenance polynomials [5] annotate static positive queries without dynamic debt debits; Cai et al.'s static differentiation [21] (Theorem 3.11) yields an extensional change term Derive(t) without turning unpaid residuals into self-paying endogenous requests or recovering paid ASTs to drive cochain differentials $d^{0,1}$; and $\mathbb{Z}$-relations [20] together with DBSP $\mathbb{Z}$-set stream calculus [22] (finitely supported maps $A \to \mathbb{Z}$, notwithstanding DBSP's exact stream differentiation–integration inverse $\mathcal{I}(\mathcal{D}(s)) = s$) merge single-step opposite tuple multiplicities $+1/-1$ to zero and collapse opposite entries $[g] + [-g]$ into the zero-evaluation kernel under group aggregation freeEval : $\mathbb{Z}^{(G)} \to G$, losing the explicit integer relation certificate needed to invert the free constant table.

Bridging all five cliffs constructively, this paper establishes a **Provenance-Preserving Conservation Calculus** that closes every pseudo-settlement loophole at the type level while answering an equally decisive constructive question: **can a ledger strict enough to reject every book-balancing shortcut also advance on its own—requiring neither adjoined history/prophecy variables nor an external scheduler—using solely its previously generated residuals and typed inventories to formulate its next request, strictly pay the same debt, deliver a receipted answer, and invert losslessly even across zero-sum cancellations?**

Throughout this work, "unforgeability" and "no debt evasion" carry an exact constructive type semantics (interfacing cleanly with our ontological front-door companion [23] and institutional

economic-constitution consumer [24]): once a source and lawful compilation contract are fixed, every unauthorized candidate (event trimming, cross-source receipt borrowing, hidden admission, equal-budget wash payment, capacity-shortage discharge, zero-budget pseudo-settlement, or omission/duplication of a nonempty request slice) admits an explicit elimination map from its required qualification type to the empty type $\bot$, while lawful evolution constructively generates the entire multi-epoch continuation chain.

1.1 Five Conservation Pillars and the Paired Positive–Negative Master Theorem

The mathematical core of **WA1**–**WA45** rests on five interlocking pillars. To prevent formalization scaffolding from obscuring mathematical content, our main theorems are stated entirely in standard algebraic and type-theoretic terms, while Lean module paths and declaration signatures are indexed in Appendix A and the accompanying machine-checked selection.

Main Theorem 1.1 (Five-Pillar Master Theorem of Provenance-Preserving Conservation and Non-Free Continuity). Fix a lawful world relation network, responsibility vocabulary, and source-native authoritative root contract. The following five conservation, obstruction, settlement, and reversible continuation theorems hold simultaneously:

1. **Bijective Whole-Event Admission, Three Lifecycle Laws, and the Capacity-Shortage Obstruction Kernel (Sections 2–3 and 5, WA1–WA5, WA15–WA17).** Registered occurrence maps on any authoritative root and across any living process (spanning local terminals and language revisions) are injective, positively realizing total reality and excluding unaccounted magic bits (Proposition 2.5); complete event-inventory admission preserves ordinary and cofinal events via bijective round-trip identities and enforces ordinary outgoing-edge exhaustion at every local terminal (Theorem 2.3, Example 2.4); classifier activation witnesses are unique and projection ports cannot be borrowed across distinct sources (Definition 2.1). Along any finite native lifecycle history, three no-free conservation laws hold—no un-admitted allocation, no un-admitted or un-reopened activation, and no un-receipted disappearance (Theorem 3.2); a single occurrence answers the entire open-responsibility fibre via in-place, full finite, or transported remainder folding (Theorems 3.4–3.5); split/merge restructuring preserves full parent/child coverage, excludes phantom rows, enforces strict child budget descent, and lifts directly across appended computations (Theorems 5.4–5.6). Moreover, for any actual capacity readout $\mathcal{R}$, available capacity falls below demanded capacity if and only if a positive obstruction witness CapacityObstruction is generated retaining all admitted obligations, affected dependencies, and protected boundaries (never converting shortage into discharge or branch pruning); every maintenance debit is derivedly bound to its source occurrence, stable slot, and debt lineage, bounded by the initial source budget, and provably empty once budget reaches zero (Theorem 3.7).
2. **Consumer-Relative Observer Safety and Total Blindness of Finite Complexified Observations (Sections 4 and 13, WA6, WA11–WA12).** An observation map $q : A \to O$ is safe for a downstream criterion $c : A \to D$ if and only if c is constant on every observation fibre; any locally invisible admission generates a `HiddenNullWitness` pairing equal observations with distinct full responsibilities together with an event-retaining `FullTraceRedirect`, and no subsequent coarsening can restore safety (Theorem 4.2, Example 4.3). Any native-write receipt

positively induces a four-way mathematical disposition: full linear isomorphism, unsound relation witness, nonzero kernel element, or unrecoverable cokernel representative (Proposition 4.4). In the global compatible integer cokernel $G = \varprojlim C_n$, the boundary class $z = (z_n)_{n\in\mathbb{N}}$ generated by a unit arithmetic transfer receipt is strictly nonzero at stage one and every subsequent stage by modulo-2 parity at prime $p = 3$ ($J(z) = z_1 \neq 0$), yet block-determinant annihilation $D_n z_n = 0$ forces $F(z) = 0 = F(0)$ across the **entire** infinite family of finite-stage complexified observations $F : G \to \prod_n (\mathbb{C} \otimes_{\mathbb{Z}} C_n)$, ruling out any linear or nonlinear recovery function factoring J through F or $G/\ker F$ (Theorem 13.1, Lemma 13.2, Theorem 13.3).

3. **Law-Free Paid Macro-Step Well-Foundedness, Two-Key Final Settlement, and Debt-Preserving Revision (Sections 6 and 10–12, WA7–WA10).** For any fixed living process P and original debt a, without assuming any global closure law or future table, the paid macro-step relation $\prec_a$ formed by enclosing one strict debit between two finite non-refilling histories $(\leq ; < ; \leq)$ is a subrelation of the natural-number strict order pullback, hence strictly well-founded and irreflexive, refuting any equal-budget wash step as payment (Main Theorem 10.3, Proposition 10.4). Final settlement $\mathrm{Settle}(x)$ requires **two keys** simultaneously: an original faithful local terminal receipt $\mathrm{LocalTerm}(x)$ and an empty same-debt fibre $\mathrm{IsEmpty}(T(x))$ in the actual successor $\sigma_P(s)$; a total emission law constructs a finite settlement history by well-founded recursion, whereas zero budget paired with an empty local terminal makes the total closure law type provably empty (Main Theorem 11.4, Corollary 11.5, Examples 11.6–11.7; end-to-end target-six witness in Theorem 6.2). Under rooted double emptiness of expression and realization fibres, the minimal coface $\mathcal{L}_{\mathrm{old}} \sqcup \{\eta\}$ admits unique faithful factorization, and every structurally anchored typed revision routes all migrated prior debts and the new zero-budget row through one shared nonterminal first write into the same canonical next current (Proposition 12.2, Theorem 12.3, Example 12.4).
4. **Paid Syntactic Accounting, Endogenous All-Future Residual Requests, and Long-Window Trace Relay (Sections 7–9 and 14, WA13–WA14, WA18–WA40).** Owner-free mathematical calculation rows are born endogenously from source occurrences and strictly debit syntax budgets step by step; integer coefficient expansions, variable-substitution body replays, ordered action words $(7n + 5)$, and iterative batches $(\Sigma_n + 2(n + 1))$ record every intermediate occurrence without cancellation erasure (Theorem 7.2, Propositions 7.3–7.4, Theorems 8.1–8.3). For any source configuration family $\mathcal{F}$, complete initial frame I, and programme configuration cfg, at every cofinal admission stop $n \in \mathbb{N}$, the literal residual $r_{m_n} = a + (-\mathrm{id})(p)$ equals the paid boundary $[a] - [p]$ across all finite allowed action words and sites in $\mathcal{M}_{\mathrm{fut}}$, transports to the registered request at canonical successor c_n^+, executes strict same-debt debit π_b from positive credit $Q - 1 > 0$ without refill, and completes its $(Q - 1)$-step suffix to deliver the effect $\epsilon = D_a(\Gamma, \delta)$, receipted answer, prior/born whole-ledger destinations, and canonical successor; zero effect $\epsilon = 0$ still pays syntax cost and fresh debts never refill prior debts (Theorems 9.1–9.3, Proposition 9.4). On the conditional right-half-plane nontrivial zero-receipt domain, four window execution traces persist intact across `Prior` into inverse-residual correction packets alongside six newly generated traces (Theorems 14.1–14.2).
5. **Cross-Packet Paid-Method Execution, Zero-Evaluation Certificate Inversion, and**

Step-Grouped Anti-Omission Continuation (Section 15, WA41–WA45). Double-negation extraction recovers the complete paid AST from two-layer residual syntax to execute any pre-fixed additive homomorphism f across subsequent cofinal stops with full query traces recorded in born stock and unified four-faces installed, while primitive-by-primitive word replay updates the chain morphism τ_1, cochain differential $d^{0,1}$, coimage differential, and same-stop physical readback in lockstep (Theorem 15.1). Free constant-table transport satisfies $\mathsf{computedAfter} = \mathsf{before} + \mathsf{effect} - \mathsf{corr} = \mathsf{after}$ in $\mathbb{Z}^{(G)}$; even when numerical evaluation cancels to zero, finite integer entries construct an explicit certificate (retaining the zero atom $-[\mathrm{inr}(0,0)]$ on empty support) that losslessly inverts the quotient to restore the exact next constant table and full paid value (Theorem 15.2, Example 15.3). Next-batch requests partition into contiguous per-step intervals with two-way ordinal address recognition $r = \mathrm{first} + s$, folding to the complete request list, executed trace, and joint successor boundary while strictly rejecting omission or duplication of any actual request's nonempty slice (Theorem 15.4); extending any sealed deterministic history locks its paid prefix verbatim and runs only the missing suffix from the reached tail state, while single-pass state projection tables export exact stage update inventories from adjacent cells (Theorem 15.5).

Table 1 summarizes the five conservation pillars alongside their paired negative controls and type-emptiness gates.

Table 1. Overview of the Five Conservation Pillars: Positive Constructions Paired with Counterexamples and Type-Emptiness Gates.

Conservation Pillar	Claim Scope	Core Positive Theorems and Constructive Capabilities	Paired Counterexamples / Uninhabited Type Gates	Sections & Proofs
I. Whole-Event Inventory & Capacity Gate	WA1–WA5, WA15–WA17	Bijective event admission & terminal exhaustion; 3 lifecycle laws & 3 remainder folds; full split/merge cover; `available < demanded` $\iff$ positive obstruction witness & debit ceiling	Singleton trimming of 2-event source (Ex. 2.4); unconsented admission empty; 2-Boolean constant-`false` wash (Ex. 5.5); shortage $\not\Rightarrow$ discharge & zero-budget debit empty (Thm. 3.7)	Secs. 2–3, 5; Fig. 1; Supp. B
II. Observer Safety & Complex Blindness	WA6, WA11–WA12	Fibre-constancy safety & factorization; 4-way receipt residual classification; unit transfer generates stagewise nonzero integral class $z = (z_n) \neq 0$	Scope-filtered hidden admission yields `HiddenNullWitness` (Ex. 4.3); all finite complexified determinant observations $F(z) = 0$ block any decoder for J (Thm. 13.3)	Secs. 4, 13; Fig. 5; Supp. A

Conservation Pillar	Claim Scope	Core Positive Theorems and Constructive Capabilities	Paired Counterexamples / Uninhabited Type Gates	Sections & Proofs
III. Well-Founded Debit & Two-Key Settlement	WA7–WA10	Law-free $\leq$; $<$; $\leq$ macro-step well-foundedness & irreflexivity; total law yields finite 2-key settlement; target-6 repair chain; shared first-write revision	Equal-budget step $B(u') = B(u)$ refuted as payment (Prop. 10.4); zero-budget fluid operational row with empty terminal makes all 3 closure types empty (Ex. 11.6)	Secs. 6, 10–12; Figs. 2, 4
IV. Paid Syntax & Residual Continuation	WA13–WA14, WA18–WA40	Owner-free syntax birth & strict debit; order-preserving stock injection; all-future residual generates request, pays same debt & delivers 4 ports; 4-trace window relay	Endpoint cancellation erases no occurrence index; zero numerical effect $\epsilon = 0$ still pays $c(r) = 2$ steps & disjoint sum tags bar cross-debt refill (Prop. 9.4)	Secs. 7–9, 14; Fig. 3; Supp. C–E
V. Certificate Inversion & Batch Gate	WA41–WA45	Double-negation paid-AST homomorphism f; primitive replay drives $\tau_1, d^{0,1}$; free constant-table integer certificate inverse; step-sliced batch & sealed-prefix tail extend	Endpoint equality never fakes trace equality; empty table retains zero atom $-[\mathrm{inr}(0,0)]$; dropping or duplicating any nonempty slice alters trace, bind cells distinct	Sec. 15; Fig. 6; Supp. D.6–D.8

1.2 Three-Minute Pen-and-Paper Anchor: Four Minimal Exhibits Exposing the Whole Mechanism

Before introducing general dependent-type signatures, the reader can verify the four sharpest mechanisms of this paper by hand in three minutes using four minimal calculations.

Example 1.2 (Three-Minute Hand-Checked Exhibits).

(a) Equal-budget transfer, capacity shortage, and genuine two-key settlement (Sections 3.4, 5, 6, and 11). Suppose bearer A holds an open obligation u with budget $b = 1$: (i) transferring u to bearer B with $b(u') = 1$ satisfies non-refill $1 \leq 1$, but lacking $b(u') < b(u)$ it can never form the core of a paid macro-step regardless of surrounding renamings (Proposition 10.4); (ii) if at some step demanded capacity is $C_{\mathrm{dem}} = 5$ while available capacity is $C_{\mathrm{avail}} = 2$, the inequality $2 < 5$ triggers `obstruction? = some w`, keeping every open obligation and protected boundary on the books without issuing any terminal receipt (Theorem 3.7); (iii) by contrast, on the target-6 partition $2 + 4 = 6$, because 4 has least prime factor 2, the compiler emits the repair $(2, 4) \to (3, 3)$, strictly debits the action-debt budget $1 \to 0$, reads primality of $(3, 3)$ to issue an independent whole-ledger terminal receipt (Key 1), and steps via the fixed handoff law into an

arithmetic successor where the action-debt branch is inactive so $T(x) = \emptyset$ (Key 2), completing genuine settlement (Theorem 6.2).

(b) How one line of integer parity defeats infinitely many complexified determinant observations (Sections 4 and 13). At stage 1 of the unit arithmetic chain (primes $\{2, 3\}$), the left zero-power readout at the prime-3 factor row evaluates the boundary β_1 to $\ell_r(\beta_1) = 1$, whereas every operator image $(I - E_1)v$ evaluates to $(1 - 3)\ell_r(v) = -2\ell_r(v)$. Since 1 is odd, $z_1 = [\beta_1] \neq 0 \in C_1$! Yet at every stage n, the block determinant $D_n = \prod_{p \in \mathcal{P}_n} (1 - p)^2 \neq 0$ (e.g., $D_1 = (1 - 2)^2(1 - 3)^2 = 4$) annihilates z_n: $D_n z_n = 0$. Upon tensoring with $\mathbb{C}$, multiplying by D_n^{-1} yields

$$1 \otimes z_n = D_n^{-1}(D_n \otimes z_n) = D_n^{-1}(1 \otimes D_n z_n) = 0 \qquad (\forall n \in \mathbb{N}).$$

Hence the entire family of finite complexified observations collapses to $F(z) = (0, 0, \dots) = F(0)$ while $J(z) = z_1 \neq 0 = J(0)$: no function h of any kind can satisfy $J = h \circ F$.

(c) Nonlinear residual payment, zero-effect syntax cost, and double-negation AST recovery (Sections 7–9 and 15.1). Over an integral ring take $a = x \cdot x$, old environment $\Gamma(x) = 2$, increment $\delta(x) = 2$, and paid endpoint $p = \text{const}\, 4$. The literal residual request $r = a + (-\text{id})(p)$ evaluates at $\Gamma + \delta$ $(x = 4)$ to $4^2 - 4 = 12 = D_a(\Gamma, \delta)$ with node cost $Q = c(a) + c(p) + 2 = 3 + 0 + 2 = 5$: after the birth first step, remaining credit $5 - 1 = 4 > 0$ forces a strict payment step and 4 suffix steps to deliver 12. Even for a constant $a = \text{const}\, 7, p = \text{const}\, 7$ with numerical effect $\epsilon = 0$, the residual $r = \text{const}\, 7 + (-\text{id})(\text{const}\, 7)$ still has cost $c(r) = 2$ and must execute negation $(2 \to 1)$ and addition $(1 \to 0)$. At the next stop, `retainedCode` extracts the double-negation subtree $-(-\text{const}\, 12)$ from the two-layer residual, recovering 12 at zero increment and feeding any pre-fixed homomorphism (say $f(u) = 3u$) to output $(36, 0)$.

(d) Free constant-table inversion under zero-sum cancellation and nonempty slice anti-omission (Sections 15.2–15.3). Suppose two sub-evaluations contribute $+5$ and -5. Numerically $5 + (-5) = 0$, leaving a bare 0 on a scalar ledger. In the free constant module $\mathbb{Z}^{(G)}$, however, the difference table is $\Delta T = [5] + [-5] \neq [0]$: our explicit certificate builder outputs scalar and addition relation generators satisfying $\Delta T - [0] = \text{relationMap}(\mathsf{actualCertificate})$ (retaining the zero atom $-[\text{inr}(0, 0)]$ even on an empty list), reconstructing the exact next constant table without guessing preimages. Likewise, when m source steps partition a request batch into slices $G_0, \dots, G_{m-1}$, any slice G_i containing an actual request has positive executed trace length $|\text{own}| > 0$, so dropping (before ++ after) or duplicating (before ++ own ++ own ++ after) that slice immediately violates trace length and position equalities.

1.3 Fifteen-Minute Verification Fast-Tracks by Discipline and Core Notation

This paper bridges programming-language semantics and resource logic, dependent type theory, homological algebra and arithmetic geometry, and distributed auditing. After checking Example 1.2, specialists may enter directly through four self-contained verification tracks:

- **Program Semantics, Separation Logic, and Resource Analysis**: Read **Sections 2–3, 5, 9.3, 10–11, and 16.2, 16.4** to verify how three remainder schemes answer whole fibres without disjoint-heap framing [3], how capacity-shortage obstruction differs from static potential derivation failure [4][13], how runtime zero-effect syntax payment differs from QTT

compile-time `0`-usage erasure [14], and how law-free $\leq; <; \leq$ well-foundedness plus two-key settlement advance beyond disjunctive transition invariants [18].

- **Dependent Type Theory, Proof Assistants, and Constructive Foundations**: Read **Sections 2, 4.1, 8.1, 12, 15.2–15.3, Sections 16.2–16.4, and Supplements B and D** to inspect T0 zero-axiom [7] bijective event admission, intensional Σ-pair [10] occurrence injectivity (without univalent identification [11] or external history/prophecy variables [16]), zero-axiom initial-coface bankruptcy revision (versus AGM contraction [19]), `ExistsEvidence` certificate inversion, and `extendTo_prefix` tail continuation.
- **Algebra, Homological Algebra, Control Observability, and Incremental Computing**: Read **Sections 4.2, 9.1, 13, 14, 15.1–15.2, Sections 16.3 and 16.5, and Supplements A and E** to check total blindness of characteristic-zero complexified observations [17] to global integral torsion, how static differentiation [21] upgrades into endogenous same-debt payment and double-negation AST cochain differentials $d^{0,1}$, and how $\mathbb{Z}^{(G)}$ relation certificates invert zero-sum cancellations beyond single-step $+1/-1$ merging in $\mathbb{Z}$-relations and DBSP $\mathbb{Z}$-sets [20][22] and across value-group kernel collapse $\mathrm{freeEval}(\Delta T) = 0$.
- **Distributed Auditing, Institutional Design, and Mathematical Accounting**: Read **Sections 1.2, 3, 5–6, 10–12, and 16.1** to verify how our calculus moves beyond Pacioli difference-group netting [1], REA event database schemas [8], and judgment-proof liability truncation [9] via anti-wash restructuring and two-key finality.

Four standard notations are fixed throughout: (1) **Dependent sums and direct sums**: Σ-types are written $\sum_{x:A} B(x)$ with pairs (x, y) or $\langle x, y\rangle$; finite-support direct sums are written $\bigoplus_j \mathcal{W}_j$ with site embedding ι_j and projection pr_j. (2) **Transport and heterogeneous equality**: canonical transport along $p : A = B$ is written $\mathrm{transport}_p$; $x : A$ and $y : B$ satisfying $A = B$ and agreeing after transport are written $x \simeq y$. (3) **Free $\mathbb{Z}$-modules and relation maps**: the finitely supported free abelian group on G is written $\mathbb{Z}^{(G)}$ (`Finsupp G Int` in Lean), with basis elements $[g]$ (or δ_g), group evaluation $\mathrm{freeEval} : \mathbb{Z}^{(G)} \to G$, and addition/scalar relation boundary relationMap. (4) **Shared core citations**: process-identity and evaluation-kernel results shared with the published second edition of the process-algebra companion [6] are cited by name and mapped in Appendix A; every ledger-conservation, capacity-obstruction, settlement, and continuation theorem of WA1–WA45 receives a self-contained written proof in the main text and supplement.

2 Whole-Event Inventory Admission and Anti-Shell-Borrowing Theorems

The earliest book-balancing sleight of hand strikes before accounting even begins: either trim inconvenient continuing events from the presentation, or borrow a valid receipt from a different source whose lower state looks similar. This section closes both avenues at the source-authority layer (**WA1**–**WA3**).

2.1 Open-Responsibility Dependent Fibres, Source-First Compilation, and Port Uniqueness

Definition 2.1 (Complete Open-Responsibility Ledger, Source Compilers, and Projection Ports). A **responsibility world** specifies a support-position type S, a responsibility index type R, and a proof-relevant dependent family $\mathrm{Open}(s, r)$, where $w : \mathrm{Open}(s, r)$ witnesses that responsibility r is actually open at position s. The **complete live ledger** at position s is the full dependent sum fibre:

$$\mathcal{L}(s) := \sum_{r:R} \mathrm{Open}(s, r). \tag{1}$$

Both the semantic claim $c(e)$ and the natural-number budget $b(e) \in \mathbb{N}$ of an entry $e = (r, w) \in \mathcal{L}(s)$ are read directly from w, while the support position carries its source anchor, occurrence association, and debt lineage key. Two open witnesses sharing the same numerical budget $b(e) = b(e')$ remain distinct entries whenever their dependent pairs differ. Here "complete" is structural: $\mathcal{L}(s)$ is the entire dependent fibre, with no mask field permitting a caller to cherry-pick a subset of rows.

A **source** fixes in advance a current type C, an initial current $c_0 \in C$, a per-current event family $E(x)$, a support map, and two compatible compilers:

1. **Structural compiler**: exhaustively compiles each event $\varepsilon : E(x)$ into one of five positive forms—native write (`nativeWrite`), relation write (`relationWrite`), continued transport (`continuedTransport`), coupled redirect (`redirect`), or local terminal (`localTerminal`); the first four determine their successor currents through their payloads, whereas a local terminal has no local ordinary successor.
2. **Whole-ledger compiler**: compiles the same ε into both the global evolution of the complete live ledger and a finite generated processing fragment. Any subsequently installed emitter $g : (x : C) \to E(x)$ must satisfy **target-occurrence commutation**: the target occurrence produced by the compiler equals the occurrence emitted by g at the target current, forbidding mid-transition event swaps.

Prior to emitter installation, the source also fixes its **projection law**. For a port p and event ε, the dependent readout type is:

$$\mathrm{Read}(p, \varepsilon) = \left(\sum_{a:\mathrm{Active}(p,\varepsilon)} \mathrm{Payload}(p, \varepsilon, a) \right) \sqcup \mathrm{Inactive}(p, \varepsilon). \tag{2}$$

Classification, activation witness a, payload, and inactivity receipt are all dependent source outputs—"inactive" carries its own source-native proof receipt rather than a null placeholder. A port installation comprises an injective port embedding together with a commuting equality on the entire dependent outcome.

This pre-emitter binding immediately yields **anti-shell-borrowing uniqueness (WA3)**: for a fixed complete source, event ε, and semantic port component, any two lawful installation paths read heterogeneously equal dependent outcomes. Comparing the source classification first, sum-constructor injectivity on the active branch identifies the two activation witnesses, along which the payloads transport into coincidence; the inactive branch is symmetric, and the installation commuting equations reduce both paths to that single source outcome. Conversely, a Boolean two-

source construction exhibits two distinct complete sources sharing the exact same lower ledger root reading while differing in their pre-fixed projection laws: once the port law is forgotten, "looking identical down below" cannot license reusing an old outcome type across sources.[1]

2.2 Bijective Event-Inventory Admission and the Untrimmed Terminal Theorem

Even when single-event ports cannot be borrowed, a caller might still attempt to trim continuing write events out of the concrete family $E_*(x)$ at the presentation boundary, leaving only a terminal event to claim "no further obligations exist." The following admission contract blocks trimming outright.

Definition 2.2 (Complete Event-Inventory Admission and Terminal Exhaustion). Let $\mathcal{S}_*$ be a concrete source and $\mathcal{S}$ its authoritative presentation. A **complete event-inventory admission** (`SourceNativeCompleteEventInventoryAdmission`) consists of:

1. **Full-family bijective presentation**: forward and reverse maps between currents and between the **complete** event families at every current whose forward–reverse and reverse–forward composites are both identities (not merely agreeing on one selected event), commuting strictly with the initial current, structural compilation, five-branch tags, optional successor, support position, whole-ledger writeback, and finite generated fragment; if cofinal boundary events are present, the cofinal event family, emission predicate, predecessor paths, and landings are likewise preserved in both directions (`no_cofinal_erasure` and `no_cofinal_insertion`).
2. **Terminal exhaustion**: whenever a presented event compiles to a local terminal ledger, the **complete ordinary outgoing-edge family** at the corresponding concrete current must be empty—where an ordinary outgoing edge is an actual concrete event paired with a continuing compiled successor. Post-terminal cross-root handoff is governed by a separate continuation law fixed in advance by the source and has a distinct type from local ordinary outgoing edges.

Theorem 2.3 (Terminal Ledgers Without Event Trimming, WA2). Under the complete event-inventory admission of Definition 2.2, every concrete source event is losslessly recovered by round trip; any faithful local terminal in the presentation corresponds to an actual local terminal of the concrete source compiler, preserves the original whole-ledger writeback and generated fragment up to heterogeneous equality, and excludes every ordinary outgoing edge at the corresponding concrete current.

Proof. Event round-trip recovery follows directly from the two composite identities. Suppose a presented event compiles to a local terminal; the optional-successor commutation equation forces the concrete event's local successor to be `none` as well. Case-analyzing the concrete structural compiler's five constructors, the four continuing constructors all carry `some` successor, contradicting `none`; hence only the concrete local-terminal constructor remains. The whole-ledger and fragment commutation equations establish heterogeneous equality of the complete dependent objects. Finally, invoking the independent terminal-exhaustion condition proves that the concrete current's ordinary outgoing-edge family is empty. Note that the event bijection alone only prevents deleting

[1] The proof uses source-classification and installation-commutation equalities; the Boolean two-source counterexample changes the pre-fixed projection law (yielding a distinct complete source) rather than producing two outcomes under one fixed source.

events; ruling out a parallel continuing write edge beside the terminal genuinely consumes terminal exhaustion. □

Example 2.4 (Counterexample: Trimming an Exit from a Mixed Two-Event Source, WA2). Let $\mathcal{S}_*$ be a single-current concrete source whose event family has two distinct constructors: a local terminal event t and a write event w carrying an actual successor ($t \neq w$). Compare $\mathcal{S}_*$ with a singleton presentation $\mathcal{S}_{\text{trim}}$ retaining only one terminal event. Were $\mathcal{S}_{\text{trim}}$ a valid complete event-inventory admission, the forward map would send both t and w to the unique point, and the left-inverse identity would force $t = w$, contradicting constructor disjointness; even if one only asked t to compile to a faithful terminal, the coexisting write event w directly refutes terminal exhaustion. Deleting responsibilities and deleting opportunities for action are the exact same manoeuvre—shrinking the universe first and proclaiming peace on the remainder—and Theorem 2.3 rejects it at type-checking time. Compared with Winskel's event structures [15] and Abadi–Lamport state refinement [16]—where pure state spaces require externally adjoined history variables precisely because snapshots discard coexisting branches and arrival paths—Definition 2.2 and Equation (3) build full-family event bijections and intensional Σ-pair [10] arrival histories directly into the authoritative root (rather than modeling lower readouts as unindexed carrier types across which HoTT univalence [11] could transport port outcomes across distinct sources).

2.3 Temporal-Visit Occurrence Injectivity and Zero-Axiom Total-Reality Locating

Proposition 2.5 (Registered-Occurrence Injectivity and Exclusion of Unaccounted Differences, WA1). On any complete-inventory-admitted authoritative root, a **temporal visit** $v \in \mathsf{Visit}$ pairs a current with its source-generated arrival history from the initial current (or a source-generated cofinal landing) along compiler successor equalities; a **registered occurrence** is the dependent pair

$$\mathsf{Occ} := \sum_{v:\mathsf{Visit}} E(\mathrm{current}(v)). \tag{3}$$

Inside the zero-axiom kernel (`#print axioms = []`): (1) any two distinct lawful visits $v \neq v'$ (even at identical current labels) have distinct registered occurrences; (2) for any fixed complete living process (including across local-terminal handoffs and Section 12 language revisions), distinct process states have distinct complete living currents; and (3) this injectivity supplies positive internal structural identity for weak difference semantics, non-circularly proving total reality (`isTotal`) and rendering external supplements and magic bits uninhabited (`noMagicBit`).

Proof. (1) Taking the first projection of an equality in Equation (3) yields $v = v'$, so distinct causal arrival histories can never be collapsed by matching current labels. (2) State embedding in a complete living process is injective on the root-and-visit pair, and inverse registration recovers the state and emitted event across both local terminals and revision handoffs. (3) Given any actual, stably referable real difference: if it is presentationally redundant it is classified directly; otherwise (1)–(2) assign it a positive internal structural identity. Any putative magic difference claiming to be both nonredundant and devoid of internal structural identity contradicts nonredundancy in the first branch and lack of identity in the second. Positive occurrence locating comes first; exclusion of unaccounted differences follows with zero axioms. □

3 Three Lifecycle Conservation Laws, Rowwise Total Disposition, and Capacity-Shortage Obstruction

Once event trimming is blocked, two structural questions immediately arise: **how can a local update patch touching only a handful of explicit rows answer an entire—potentially infinite—open responsibility fibre, and what happens when available system capacity falls short of maintenance demand?** This section establishes the three lifecycle conservation laws, three exact remainder folding schemes, and the fourth kernel on capacity-shortage positive obstruction (**WA4**).

3.1 Native Lifecycle Edges and the Three No-Free Conservation Laws

Definition 3.1 (Responsibility Admission, Live Dispositions, and Five Lawful Terminal Receipts). An **admitted obligation** retains its original admission receipt together with current content, residual work, bearer, beneficiary, and scope. The admission receipt locks the source event, source anchor, responsibility incidence, debt lineage key, and proof of source jurisdiction: a bare need, risk, or wish never creates another party's obligation unless coupled with source authority and acceptance (or authorized assumption).

Along a slot sequence, live obligations evolve exclusively via witness-carrying native edges: progress (`progressed`), maintenance (`maintained`), deferral (`deferred`), jurisdiction-backed scope narrowing (`narrowed`), receiver-accepted transfer (`transferred`), successor-covered supersession (`superseded`), and path-preserving bearer release (`bearerReleased`). **Lawful terminal receipts** distinguish five non-interchangeable grounds for ending an obligation: fulfilled completion, authorized waiver, invalidation, consequence-borne abandonment, and accepted unrealizable residue. A termination edge converts the live slot into a **terminal archive** storing the full ended obligation, source event, and terminal receipt; a `reopen` edge acts solely on an existing terminal archive while preserving its original debt lineage, never minting a “never owed” identity.

Theorem 3.2 (Three Lifecycle No-Free Conservation Laws, WA4). Across every actual edge of any finite lifecycle history generated by the native edges of Definition 3.1:

1. **No free birth**: any freshly allocated slot that becomes live carries an actual admission receipt from that edge;
2. **No free activation**: any existing non-live slot that becomes live arises exclusively via admission or reopening;
3. **No silent disappearance**: any live slot that ceases to be live retains in its archive an actual lawful terminal receipt for the original obligation at that slot.

Proof. Every native edge modifies at most one slot, so exhaustive case analysis on a single edge suffices first: admission appends a live slot carrying its admission receipt; reopening acts only on an existing terminal archive while keeping its lineage; every other edge targets one live slot, leaving all distinct slots unchanged and—at the target slot—producing a live obligation under progress, maintenance, deferral, narrowing, transfer, supersession, and bearer release, while only the termination constructor replaces it with a terminal archive holding the original obligation and

terminal receipt. Induction over finite path concatenation extracts the first edge altering the slot's liveness together with its receipt. □[2]

As a concrete positive/negative pair: in a promise-admission instance, an empty state transitions upon acceptance to a live obligation with remainder 3, and an accepted transfer updates its bearer while preserving slot liveness and lineage; in a rejected-admission control world, needs can be proposed, yet both acceptance and authorized-assumption types are empty, proving the admission type uninhabited—not a failure to locate paperwork, but a type-level impossibility of unconsented obligation.

3.2 Three Exact Remainder Schemes and Answering the Whole Fibre from Finite Patches

Definition 3.3 (Whole-Ledger Global Evolution and Three Generated Remainder Fragments). For an ordinary continuing event $\varepsilon : s \to t$ within a fixed responsibility world, global evolution of the complete live ledger consists of two total dependent maps:

$$d_\varepsilon : \prod_{e:\mathcal{L}(s)} \sum_{f:\mathcal{L}(t)} \mathrm{Evolve}_\varepsilon(e,f), \qquad o_\varepsilon : \prod_{f:\mathcal{L}(t)} \sum_{e:\mathcal{L}(s)} \mathrm{Evolve}_\varepsilon(e,f). \tag{4}$$

Here d_ε assigns every prior obligation its exact target destination and o_ε assigns every target obligation its exact prior origin; they are not assumed to be inverse bijections so that one-to-many splits and many-to-one merges in Section 5 are faithfully represented (whereas obligation birth and language revision first generate a new responsibility world already containing the initial row before applying Equation (4) to its complete fibres). A local terminal ledger instead supplies $\prod_{e:\mathcal{L}(s)} \mathrm{Discharge}_\varepsilon(e)$.

Rather than accepting a manually assembled instance of Equation (4) after the fact, the whole-ledger compiler takes the source occurrence and generates **finite explicit rows plus an exact remainder**, using one of exactly three continuing coverage schemes:

- **In-place remainder ($s = t$)**: a hit returns a finite row index together with an equality identifying the queried entry; a miss carries the identical entry in place.
- **Full finite cover**: supplies total index maps from both $\mathcal{L}(s)$ and $\mathcal{L}(t)$ into finite row numbers together with round-trip proofs recovering every queried entry.
- **Transported remainder**: permits finite explicit exceptions across distinct supports while routing every missed entry to one shared source-generated remainder event whose compiled evolution already determines all remaining destinations and origins.

Likewise, terminal fragments are generated either via an exact finite index covering the complete fibre or via a source-selected support-discharge receipt applied uniformly across $\mathcal{L}(s)$ without assuming finiteness.

Theorem 3.4 (Finite Fragments Fold to Answer the Whole Ledger, WA4). Every continuing fragment in Definition 3.3 folds deterministically into the global evolution of Equation (4), every terminal fragment folds into rowwise discharge across $\mathcal{L}(s)$, and the folded outcome equals the whole-ledger compilation of the same occurrence. Specifically: misses under an in-place remainder are strict identity carries (never hidden transfers); full finite cover forces the open fibre

[2]Foundational single-edge and finite-path induction also appear in [6], Proposition 6.1.

itself to be finite (forbidding a finite list from impersonating an infinite fibre); and transported remainder can pay an entire ledger strictly even with **zero explicit exception rows**.

Proof. Under in-place remainder, case-splitting on the optional hit either transports the explicit row along the attached entry equality or returns the original entry with the identity-carry constructor (disjoint from transfer). Under full finite cover, every old and new entry maps to a finite index whose recovery equality makes the map from finite indices onto $\mathcal{L}(s)$ and $\mathcal{L}(t)$ surjective, forcing finiteness. Under transported remainder, misses invoke the compiled remainder event's total destinations and origins directly. Terminal fragments act either via the surjective finite index or by applying the source-selected support receipt to each entry $e \in \mathcal{L}(s)$. Because the fragment type is indexed by the whole-ledger compilation output itself, fold equality holds by the compilation contract. □

3.3 Rowwise Causal Total Disposition and Shared Canonical Successor

Theorem 3.5 (Rowwise Total Disposition and Shared Successor Current, WA4). Fix a complete-inventory-admitted authoritative root, a registered visit, and its whole-ledger occurrence. For **every** entry $e \in \mathcal{L}(s)$ (whether or not selected by the local finite patch), the compiler produces either a target evolution landing in the actual successor's complete ledger or an exact terminal receipt for e; any generative causal authority held on e advances directly along that same evolution, and `answerAndNext` for any two queried rows reads the exact same canonical next current.

Proof. Expanding the whole-ledger compilation: across the four continuing branches, local-successor injectivity identifies the target state with the actual successor, and emitter commutation (Definition 2.1) identifies the target occurrence with the occurrence actually emitted at that successor, so transporting along both equalities lands inside the true successor ledger rather than a look-alike table; in the local-terminal branch, rowwise discharge applies directly. Successor causal authority composes prior authority with the exact successor equality without target-side re-admission, and `nextCurrent` is read from the visit occurrence itself independently of which row e was queried. □

Proposition 3.6 (Actual Discharge or Transfer Forces Canonical Generated-Row Authority, WA4). Whenever the whole-ledger disposition is terminal, or an entry e undergoes an actual transfer in the folded evolution, canonical generated-row authority for e is constructed directly from that disposition fact—without requiring any caller-supplied row index or membership proof—and its target evolution or receipt agrees heterogeneously with the whole-ledger fold.

Proof. Terminal branches use either the full finite cover selector or the source-selected support-discharge witness. Continuing transfer branches inspect the three schemes: full finite cover already supplies every index; transported remainder yields explicit-row or remainder-row authority in its two branches; and under in-place remainder, a miss would fold to the identity-carry constructor, contradicting the assumed transfer constructor and forcing an explicit hit. Transporting along fold-equals-compilation yields heterogeneous equality. □

3.4 The Fourth Kernel: Capacity-Shortage Positive Obstruction and Source-Bound Maintenance Debit

One of the most pervasive failure modes in both engineered systems and formal models is **converting physical or computational capacity exhaustion into silent debt forgiveness**: when memory fills up, bandwidth saturates, or maintenance budget hits zero, pending obligations are quietly dropped or declared "best-effort complete." `ResponsibilityCapacityKernel` severs this inference at the root, forming the fourth conservation kernel of the lifecycle layer.

Theorem 3.7 (Capacity-Shortage Obstruction Criterion and Source-Bound Maintenance Debit Bounds, WA4). Fix a responsibility vocabulary V and a capacity vocabulary C (specifying incompatible demands, affected dependencies, and protected boundaries):

1. **Capacity shortage iff positive obstruction (never discharge)**: For any actual capacity readout $\mathcal{R}$ (carrying its source event, bearer, admitted obligations, demanded capacity $C_{\mathrm{dem}}(\mathcal{R}) \in \mathbb{N}$, available capacity $C_{\mathrm{avail}}(\mathcal{R}) \in \mathbb{N}$, and full lists of incompatible demands, affected dependencies, and protected boundaries), the obstruction classifier $\mathsf{obstruction?}(\mathcal{R})$ satisfies the exact biconditionals:

$$(\exists \omega,\ \mathsf{obstruction?}(\mathcal{R}) = \mathrm{some}\,\omega) \iff C_{\mathrm{avail}}(\mathcal{R}) < C_{\mathrm{dem}}(\mathcal{R}), \qquad \mathsf{obstruction?}(\mathcal{R}) = \mathrm{none} \iff C_{\mathrm{dem}}(\mathcal{R}) \le C_{\mathrm{avail}}(\mathcal{R}). \tag{5}$$

   Here the positive witness $\omega : \mathsf{CapacityObstruction}(\mathcal{R})$ encapsulates solely $C_{\mathrm{avail}}(\mathcal{R}) < C_{\mathrm{dem}}(\mathcal{R})$ while keeping all admitted obligations, affected dependencies, and protected boundaries visible in $\mathcal{R}$, containing **zero prioritization, forced transfer, scope-narrowing, or terminal-discharge fields**.
2. **Derived source ownership of maintenance debits**: For any edge $g : \sigma \to \tau$ of a native responsibility process P and debt lineage λ, any maintenance-debit receipt $d : \mathsf{ActualMaintenanceDebit}(P, B, g, \lambda)$ (requiring $g.\mathsf{lifecycleEvent} = \mathrm{carried}(s, o, \varepsilon, \mathrm{maintained}(r))$ on a live obligation o at stable slot s with strict budget drop $B(\tau, \lambda) < B(\sigma, \lambda)$) constructively derives $\mathsf{SourceOwnsResponsibilityIncidence}(V, d.\mathsf{sourceEvent}, o)$ from `event_eq` alone without extra domain premises, preventing a detached natural-number countdown from impersonating lifecycle maintenance.
3. **Certified debited-run ceiling and zero-budget exhaustion gate**: Along any actual lifecycle run $\gamma : \sigma \to^* \tau$ certified by a source-owned carry-debit process B, the number of generated maintenance events $|\gamma|$ plus remaining target budget is bounded by the starting budget, and zero (or exhausted) budget renders further maintenance-debit events uninhabited:

$$|\gamma| + B(\tau, \lambda) \le B(\sigma, \lambda), \qquad B(\sigma, \lambda) = 0 \Longrightarrow \mathrm{IsEmpty}(\mathsf{DebitEvent}(g, \lambda)). \tag{6}$$

   In particular, spending the exact starting budget ($|\gamma| = B(\sigma, \lambda)$) forces $B(\tau, \lambda) = 0$ and makes $\mathrm{IsEmpty}(\mathsf{DebitEvent}(g', \lambda))$ hold for every subsequent outgoing edge g' from τ—any further transition requires a separately generated non-maintenance disposition receipt.

Proof. (1) Unfolding $\mathsf{obstruction?}(\mathcal{R})$ on the decidable inequality $C_{\mathrm{avail}}(\mathcal{R}) < C_{\mathrm{dem}}(\mathcal{R})$ yields some $\langle \mathrm{shortage} \rangle$ when true and `none` when false, proving both equivalences in Equation (5), while the structure signature of $\mathsf{CapacityObstruction}(\mathcal{R})$ contains no discharge or pruning fields. (2) Case-

splitting on the native edge constructors (`admit`, `reopen`, `cross`) and the nine sub-dispositions inside `cross`, eight branches clash with constructor carried$(s, o, \varepsilon, \text{maintained}(r))$ in `event_eq`; the sole surviving `maintained` branch extracts `boundary_anchor_eq` and `boundary_incidence_eq` directly from P to assemble `sourceOwnership`. (3) Inductively mapping γ to a StrictBudgetRun: `nil` satisfies $0 + B(\sigma, \lambda) \leq B(\sigma, \lambda)$, and `step` combines $B(\tau, \lambda) + 1 \leq B(\mu, \lambda)$ with $|\gamma_0| + B(\mu, \lambda) \leq B(\sigma, \lambda)$ while identifying $|\gamma|$ with `run.events.length`, proving the left half of Equation (6). When $B(\sigma, \lambda) = 0$, any $d \in \mathsf{DebitEvent}(g, \lambda)$ would yield $B(\tau, \lambda) < 0$ in $\mathbb{N}$, a contradiction; setting $|\gamma| = B(\sigma, \lambda)$ in the left inequality forces $B(\tau, \lambda) = 0$ and hence empties all subsequent maintenance debits. □

Theorem 3.7 draws a sharp boundary simultaneously in law-and-economics and resource type theory: in ex-post scalar liability systems, Shavell's judgment-proof phenomenon shows that recoverable damages are bounded by available assets when $C_{\text{avail}} < C_{\text{dem}}$ [9], whereas in Automatic Amortised Resource Analysis (AARA) [13] and amortised separation logic [4] insufficient potential blocks static typing derivations, and even dynamic credit acquisition (`acquire` in [4], Section 3.6) branches only on local scalar credit without emitting a whole-ledger runtime witness retaining affected dependencies and protected boundaries. Theorem 3.7 severs every path from scarcity to forgiveness: insufficient capacity constructively produces CapacityObstruction, retaining every admitted obligation and protected boundary intact. Figure 1 integrates bijective event-inventory admission, the three exact remainder schemes, and the capacity-shortage obstruction gate into one unified structural plate.

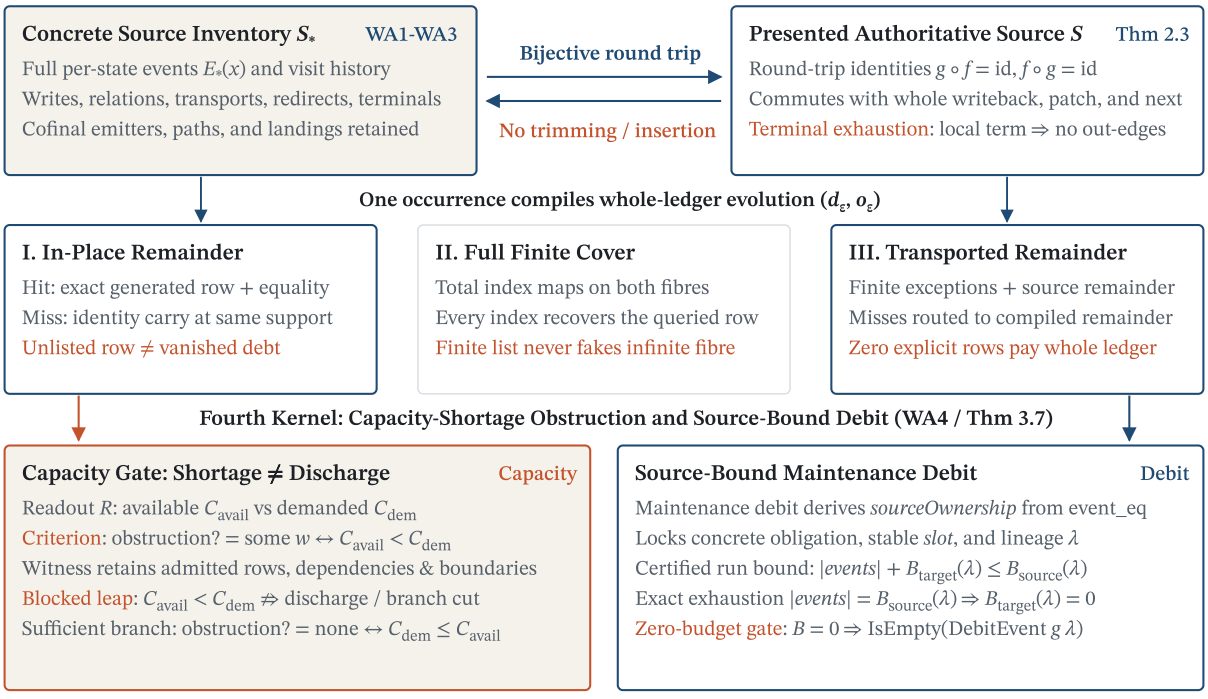


Figure 1 | Complete Event-Inventory Admission, Remainder Coverages, and Capacity Obstruction. Top tier (WA1–WA3): concrete source $\mathcal{S}_*$ and presented source $\mathcal{S}$ satisfy full-family round-trip identities while terminal exhaustion blocks trimming a parallel continuing edge beside a terminal. Middle tier (WA4, Theorem 3.4): a single registered occurrence folds into global whole-ledger evolution $(d_\varepsilon, o_\varepsilon)$ via in-place remainder, full finite cover, or transported remainder. Bottom tier (WA4, Theorem 3.7): the left gate proves $C_{\text{avail}} < C_{\text{dem}} \iff$ CapacityObstruction, with vermilion $\not\Rightarrow$ marking that capacity shortage never licenses discharge or branch cuts; the right gate binds every maintenance debit to its source occurrence and slot, emptying further maintenance debits once budget hits zero.

4 Consumer-Relative Observer Safety and Four-Way Residual Dispositions

With underlying events and rows protected against deletion, the second auditing trap is **observational compression**: when a summary dashboard, net-worth line, or determinant projection reads "unchanged," may a downstream consumer treat the two states identically? This section establishes the exact safety criterion and four-way residual classification (**WA6, WA12**).

4.1 Fibre-Constancy Safety, Hidden-Null Witnesses, and Full-Trace Redirects

Definition 4.1 (Consumer-Relative Observer Safety and Hidden-Null Witnesses). Let A be a complete state type, $q : A \to O$ an observation map, and $c : A \to D$ a downstream responsibility criterion or readout (dependent readouts pack their index into D). Observation q is **safe for** c (`ConsumerSafe`) iff c is constant along every observation fibre of q:

$$\mathrm{Safe}(q, c) :\Longleftrightarrow \forall x, y : A,\ q(x) = q(y) \Longrightarrow c(x) = c(y). \tag{7}$$

Conversely, a pair $(x, y) \in A \times A$ equipped with $q(x) = q(y)$ and $c(x) \neq c(y)$ is a **hidden-null witness** (`HiddenNullWitness`) for (q, c).

Definition 4.1 makes explicit that observer safety is never an intrinsic virtue of q in isolation: the exact same net-balance screen can be perfectly safe for checking scalar solvency while catastrophically blind to whether bearer A's specific obligation was discharged.

Theorem 4.2 (Factorization Safety, Hidden-Null Refutation, and Monotonicity of Blindness under Coarsening, WA6). For any complete state type A, observation $q : A \to O$, and criterion $c : A \to D$:

1. **Sufficiency of factorization**: if $c = \bar{c} \circ q$ for some $\bar{c} : O \to D$, then $\mathrm{Safe}(q, c)$ holds constructively;
2. **Refutation by hidden null**: given any hidden-null witness (x, y), the safety type $\mathrm{Safe}(q, c)$ is uninhabited ($\mathrm{Safe}(q, c) \to \bot$);
3. **Irreversibility under coarsening**: for any further compression $h : O \to O'$, the same pair (x, y) remains a hidden-null witness for $(h \circ q, c)$—post-processing or smoothing a blind report can never recover lost responsibility distinctions.

Proof. (1) Applying $\bar{c}$ to $q(x) = q(y)$ and rewriting via $c = \bar{c} \circ q$ gives $c(x) = c(y)$ without invoking `Classical.choice`. (2) Feeding $q(x) = q(y)$ into Equation (7) yields $c(x) = c(y)$, contradicting $c(x) \neq c(y)$. (3) Applying h to $q(x) = q(y)$ yields $h(q(x)) = h(q(y))$ while $c(x) \neq c(y)$ persists unchanged. □

Example 4.3 (Invisible Scope Admission and Event-Preserving `FullTraceRedirect`, WA6). Take the concrete promise-admission edge $\varepsilon : \sigma_0 \to \sigma_1$ of Section 3.1, where σ_0 has empty slots and σ_1 holds one live obligation whose scope lies outside a scope-filtered observer q_{scope}. Then $q_{\mathrm{scope}}(\sigma_0) = [\,] = q_{\mathrm{scope}}(\sigma_1)$, whereas the full live-slot observer c_{full} reads $|c_{\mathrm{full}}(\sigma_0)| = 0 \neq 1 = |c_{\mathrm{full}}(\sigma_1)|$, constructing a `HiddenNullWitness` and emptying $\mathrm{Safe}(q_{\mathrm{scope}}, c_{\mathrm{full}})$. Simultaneously, the kernel packages ε, the invisibility equality, and the full-state inequality into a `FullTraceRedirect`, redirecting audit verification back to the complete event-retaining trace. When a screen reports "no change," the calculus thus separates *certified safe*, *refuted by a hidden null*, and *unclassified*,

never conflating absence of evidence with safety. Section 13 realizes this phenomenon across all finite-stage complexified determinant observations simultaneously.

4.2 Four-Way Mathematical Residual Disposition Induced by a Native-Write Receipt

Proposition 4.4 (Four-Way Mathematical Disposition from One Native-Write Receipt, WA12). Fix a concrete dynamical source D, state s, and target abelian group A. Eliminating the whole-ledger compilation at a native write to recover the exact transfer receipt $\varepsilon = D.\mathrm{emit}(s)$, the source-generated completed history carrier $\widehat{H}_s$ and $\mathbb{Z}$-linearly extended evaluator $E : \mathbb{Z}^{(G)} \to A$ exhaustively classify the occurrence into one of four positive mathematical outcomes (while preserving the original transfer row and canonical next state):

1. **Faithful linear equivalence**: the relation closure evaluates to zero under E, and the induced $\bar{E} : \widehat{H}_s \to A$ has trivial kernel and singleton cokernel, constructing a full $\mathbb{Z}$-linear isomorphism $\phi : \widehat{H}_s \simeq A$ and exact inverse ϕ^{-1};
2. **Unsound relation witness**: exhibits an actual relation q in the history's relation closure with $E(q) \neq 0$;
3. **Nonzero kernel residual**: relations are sound while $\ker \bar{E} \neq 0$, exhibiting $k \in \widehat{H}_s \setminus \{0\}$ with $\bar{E}(k) = 0$;
4. **Coverage-loss representative**: relations are sound and $\ker \bar{E} = 0$ while $A/\operatorname{im} \bar{E}$ is nontrivial, exhibiting $a \in A \setminus \operatorname{im} \bar{E}$ together with a proof that **every** (linear or nonlinear) candidate recovery map $f : A \to \widehat{H}_s$ satisfies $\bar{E}(f(a)) \neq a$.

Proof. Eliminating the compiled ledger structure (whose other four structural branches are uninhabited in this source) and applying event-presentation round-trip equality identifies the compiled receipt with $\varepsilon = D.\mathrm{emit}(s)$, grounding the history, relations, and evaluator in that single ledger receipt. If some q has $E(q) \neq 0$, branch (2) holds; otherwise E descends to $\bar{E} : \widehat{H}_s \to A$. A nonzero element of $\ker \bar{E}$ gives branch (3); trivial kernel plus surjectivity constructs $\phi : \widehat{H}_s \simeq A$ in branch (1); otherwise taking the difference of two distinct cosets in $A/\operatorname{im} \bar{E}$ yields a nonzero coset representative $a \notin \operatorname{im} \bar{E}$ in branch (4), for which $\bar{E}(f(a)) = a$ would imply $a \in \operatorname{im} \bar{E}$, a contradiction (full details in Supplement A.1). This mathematical classification preserves the transfer row's non-increasing budget equality without mistaking a mathematical residual readout for a strict same-debt payment. □

5 Restructuring Conservation, Exclusion of Phantom Rows, and Same-Root Certificate Lifting

When neither event trimming nor coarse reporting works, a third evasion tactic remains: **splitting one obligation into pieces, merging pieces into one lump, or shuffling obligations across accounts so that structural rearrangement washes away the original liability.** This section establishes same-debt locking, full-fibre split/merge conservation, and direct certificate lifting across appended computations (**WA5, WA15–WA17**).

5.1 Same-Debt Equivalence Locking and the Non-Refill Law of Row Evolution

Definition 5.1 (Same Station, Same Debt Lineage, and Genuine Freshness). For source entry $e \in \mathcal{L}(s)$ and target entry $f \in \mathcal{L}(t)$, **same station** requires identical anchor, occurrence association, lineage, responsibility tag, and semantic claim; **same debt** locks the two invariants governing cross-carrier obligation continuity—debt lineage λ and semantic claim c:

$$e \sim_{\text{debt}} f :\Longleftrightarrow \ \lambda(e) = \lambda(f) \ \wedge \ c(e) = c(f). \tag{8}$$

Lawfully accepted bearer changes, support migrations, or local clock steps therefore preserve $e \sim_{\text{debt}} f$. Conversely, certifying that f is **genuinely fresh** relative to the complete prior ledger $\mathcal{L}(s)$ requires a refutation $(e \sim_{\text{debt}} f) \to \bot$ against **every** $e \in \mathcal{L}(s)$, not merely inequality against one chosen row.

Proposition 5.2 (Same-Debt Preservation and Non-Refill Across Lawful Row Evolutions). Every lawful row evolution $\text{Evolve}_{\varepsilon}(e, f)$ is built from one of three constructors: in-place carry (preserving support and exact entry, so $b(f) = b(e)$), in-place maintenance (preserving station with strict debit $b(f) < b(e)$), or receiver-accepted transfer (preserving same-debt identity with non-refill $b(f) \leq b(e)$). Consequently every lawful row evolution satisfies:

$$e \sim_{\text{debt}} f, \qquad b(f) \leq b(e). \tag{9}$$

Proof. Eliminating the three constructors of $\text{Evolve}_{\varepsilon}(e, f)$: carry reduces to reflexivity along its support and heterogeneous entry equalities; maintenance projects its lineage/claim equalities and $b(f) < b(e)$ (which implies $\leq$); transfer projects its receipt's lineage/claim equalities and $b(f) \leq b(e)$. □

Equation (9) deliberately retains equality $b(f) = b(e)$ in the transfer branch: an equal-budget transfer is completely lawful, yet it only relocates an unpaid obligation—**lawful relocation is never payment**. Section 10 will prove that no finite chain of equal-budget relocations can manufacture well-founded descent.

5.2 Full-Fibre Split/Merge Receipts, Exclusion of Phantoms, and Zero-Budget Split Prohibition

Definition 5.3 (Full-Fibre Split and Merge Receipts). Under a source-fixed restructuring contract (`LivingLawRootSourceNativeLedgerRestructuringKernel`):

1. A **split receipt** fixes an occurrence-owning source event, a nonempty child list, and full-fibre coverage, equipping **each** child with a descendant link from the parent, anchor transport, same-lineage proof, local discharge preservation, and **childwise strict budget descent** $b(\text{child}) < b(\text{parent})$; whenever two distinct target entries share one prior origin e, the restructuring compiler must emit a split receipt covering the entire target fibre over e, with every listed child actually present in $\mathcal{L}(t)$ (no phantom children).
2. A **merge receipt** dually fixes a nonempty parent list and merged target entry, equipping each parent with ancestor link, anchor transport, same-lineage proof, and local discharge preservation; whenever two distinct prior entries map to one target f, the compiler must emit

a merge receipt covering the entire origin fibre of f with no phantom parents outside $\mathcal{L}(s)$. Identical entries may return the identity branch without fabricating a split or merge.

Theorem 5.4 (Split/Merge Coverage Conservation and Zero-Budget Split Prohibition, WA5). Under the restructuring contract of Definition 5.3, every nontrivial split preserves provenance and local discharge conditions for every actual child while forcing each child's budget strictly below its parent's; every nontrivial merge preserves provenance and local discharge conditions for every actual parent. Equal row counts or equal budget sums cannot substitute for these dependent receipts. In particular, **a zero-budget parent ($b(e) = 0$) admits no lawful nontrivial split**.

Proof. Case-splitting on the restructuring compiler's classification: the identity branch equates the two dependent entries, contradicting nontriviality. In the split branch, full origin-fibre coverage sends every actual target child into the receipt's child list, reading off descendant link, lineage, local discharge, and $b(\text{child}) < b(\text{parent})$, while target-membership excludes phantom children; the merge branch is symmetric. When $b(\text{parent}) = 0$, any actual child of a nontrivial split would satisfy $0 \leq b(\text{child}) < b(\text{parent}) = 0$ in $\mathbb{N}$, a contradiction. □

Example 5.5 (Counterexample: Washing Away One of Two Rows While Keeping Two Rows, WA5). Index both prior and target ledgers by $\text{Bool} = \{\text{false}, \text{true}\}$ (two rows each, all budgets 0), and set both destination d and origin o constantly to false. Superficially, row counts match ($2 = 2$), total budgets match ($0 = 0$), and every paired transition satisfies $0 \leq 0$! Yet target false and true share origin false, while prior false and true collide at target false (silently evaporating the prior `true` obligation). Under Definition 5.3, returning identity forces false = true, whereas returning split or merge demands uninhabited full-cover and strict-descent receipts—contradiction in every branch! Combined with the two-sided lifecycle sums (every old row has a covered descendant family or source terminal receipt; every new row has an actual ancestor or current admission), rearrangement can neither wash away old debt nor conjure phantom authority.

5.3 Direct Same-Root Lifting of Mother Restructuring Certificates and Continuing Remainders

When Section 7 appends endogenous mathematical calculation rows onto an existing domain ledger, does appending computation invalidate prior split/merge certificates or force the caller to re-prove target classification? The next theorem proves that all mother restructuring certificates and continuing remainders lift automatically on the same root.

Theorem 5.6 (Same-Root Lifting of Restructuring Certificates and Continuing Remainders Across Appended Computation, WA15–WA17). Fix an authoritative root, a complete-native programme read from its compiler, and a registered input:

1. **Multiplicity-preserving tail append (WA15)**: if the prior native inventory has m explicit rows, appending one independent math obligation produces an $(m+1)$-row inventory preserving every prior row via `castSucc` (never deduplicating repeated claims or occurrences) with the math row at the last slot; prior identity-scope injectivity plus disjoint sum tags yield two-sided injectivity and an authoritative living root.

2. **Direct lifting of general split/merge certificates (WA16)**: without assuming injectivity, disjoint sum tags (`old` ⊔ `math`) equip the prior-obligation embedding j with a fieldwise left inverse q ($q \circ j = \mathrm{id}$ across admission, authority, assumption, and discharge types); children, parents, source ownership, anchor transport, lineage, local discharge, full coverage, no-phantom guarantees, and strict child debits lift directly along j with zero target-side re-certification.
3. **Whole-join folding across all continuing branches (WA17)**: given a per-current continuing-packet programme covering native write, relation write, continued transport, and redirect, a one-row explicit math fragment paired with the source-eliminated transported remainder folds to the exact dependent whole join of prior ledger evolution and math step evolution, preserving all prior projections, dynamic type–value observations, and canonical next currents within the same discrete step.

Proof. Separating prior and math obligations via disjoint sum tags ensures $q(j(a)) = a$, so any repeated origin or destination on the prior side reduces via q to the exact same restructuring fibre in the mother compiler without cross-tag interference. Identity lifts by congruence along j; split and merge map parent/child lists via $\mathrm{map}(j)$, preserving full coverage, no-phantom membership, and $b(\text{child}) < b(\text{parent})$ through $q \circ j = \mathrm{id}$ and budget commutation. On continuing packets, eliminating the compiled packet recovers (d, o), finite patch, and next current; routing slot 0 to the math step and all misses to that compiled remainder event folds componentwise over the Σ-sum into the exact whole join (fieldwise details in Supplement B.1). □

6 End-to-End Payment and Two-Key Settlement on the Target-Six Arithmetic Partition

Before ascending to general syntactic continuation and infinite observation families, we trace one complete concrete instance on the target-6 Goldbach partition source (**WA10**)—walking step by step from **endogenous action emission, through strict $1 \to 0$ debit and an independent terminal receipt, to a debt-inactive arithmetic successor**.

6.1 Emit the Native Action First; Read Target Primality Second

The target-6 process starts at $(2, 4)$ ($2+4 = 6$). The right summand 4 is composite with least prime factor 2. Because the production compiler's finite factor-repair inventory contains this structural key, its "already a prime pair" branch is refuted by compositeness of 4 and its "no repair available" obstruction branch is refuted by inventory membership, leaving only the compiler-emitted **factor-two repair action**, which updates the left summand to $2+(2-1) = 3$ and fixes the right summand by sum conservation at 3:

$$(2, 4) \xrightarrow{\text{repair}_2} (3, 3), \qquad 3 + 3 = 6. \tag{10}$$

Notice the type purity of this action receipt: it carries only the repair step, its computed target $(3, 3)$, and the equality certifying compiler output—**it contains no caller-supplied "prime pair achieved" or "settled" field**.

Only after Equation (10) lands does the target classifier inspect primality of $(3, 3)$: because both 3 and 3 are prime, entering another repair branch would contradict the classification equation, forcing emission of the terminal-class readout, extraction of the valid prime-pair fibre, and a nonzero actuality settlement. Emitting the action and verifying its target are two strictly sequential dependent steps.

6.2 One Repair Action Drives Strict Debit, Independent Terminal Receipt, and Debt-Inactive Successor

Definition 6.1 (Target-Six Action Debt). Let u be the open state and v the paid state with budgets $b(u) = 1$ and $b(v) = 0$: the payment transition is generated by the native repair of Equation (10), and the support-discharge witness is generated by the target primality readout at $(3, 3)$. Attaching this debt law to the emitted arithmetic occurrence retains all prior arithmetic rows alongside this action debt.

Theorem 6.2 (End-to-End Authentic Payment and Two-Key Settlement Chain at Target Six, WA10). Within a single source compilation architecture, the target-6 action debt of Definition 6.1 delivers in sequence: (1) an actual **strict same-debt payment** $1 \to 0$ onto the unique target v; (2) an **independent whole-ledger rowwise terminal receipt** at v generated from the $(3, 3)$ prime-pair terminal witness; and (3) a **canonical arithmetic successor** via the fixed terminal handoff law in which prior arithmetic rows continue while the action-debt branch is inactive (meaning the open fibre $T(x)$ of same-lineage, same-claim witnesses in the actual successor ledger is empty, $T(x) = \emptyset$; general form in Section 11.1, Equation (46)). Payment, terminal receipt, and debt-inactive continuation are produced by their respective compilers rather than inferred from "budget equals zero."

Proof. Step 1 (Strict debit): Equation (10) induces the debt transition $u \to v$ with $b(v) = 0 < 1 = b(u)$; the extended ledger uses Section 3.2's zero-explicit-row transported remainder to carry all prior rows while mapping this action debt uniquely to v, locked by the emitted-branch equality. Step 2 (Key 1—independent terminal receipt): the primality readout at $(3, 3)$ produces the support-discharge witness at v, from which the whole-ledger terminal compiler issues rowwise terminal receipts across the paid support; in the two-stage positive root, the paid stage's event family consists solely of the local-terminal constructor with empty ordinary outgoing edges, passing Theorem 2.3's terminal-exhaustion admission. Step 3 (Key 2—empty same-debt fibre in actual successor): the pre-fixed terminal handoff law maps that registered terminal occurrence to the next arithmetic state with debt flag inactive; there the complete live ledger has inhabitants only in the prior-claim left sum branch (`inl`) and none in the action-debt right branch (`inr`), proving $T(x) = \emptyset$ while the arithmetic world continues (Figure 2). □

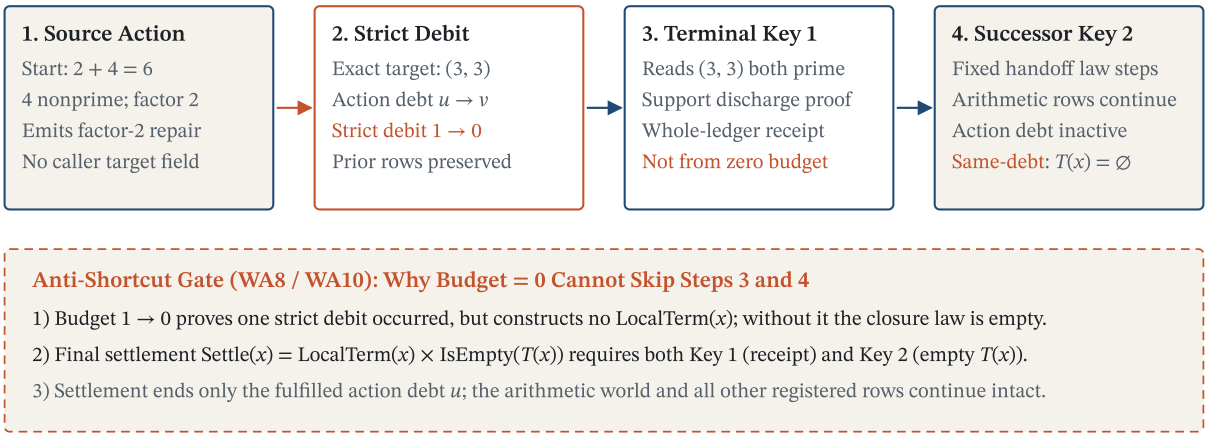


Figure 2 | Actual Factor-Two Repair, Strict Debit, and Two-Key Settlement at Target Six. Top row (Theorem 6.2): the source compiler detects least factor 2 of 4 and emits the repair $\rightarrow$ lands at $(3, 3)$ with strict action-debt debit $1 \rightarrow 0$ $\rightarrow$ reads double primality to issue an independent whole-ledger terminal receipt (Key 1) $\rightarrow$ steps via the fixed handoff law into an arithmetic successor where the action debt is inactive and $T(x) = \emptyset$ (Key 2). Bottom dashed box: budget reaching zero at Step 2 can neither skip constructing Key 1 at Step 3 nor bypass verifying Key 2 in the actual successor at Step 4.

7 Owner-Free Endogenous Computation and Exact Syntactic Accounting

Beginning with this section, we advance from static and single-transition conservation to **dynamic endogenous computational continuation**. A widespread shortcut in computational accounting records only final values: if a compound evaluation cancels to zero (such as $+v$ plus $-v$), its intermediate steps appear free to discard. Here we prove that owner-free mathematical calculation rows are born endogenously from source occurrences, and every syntactic node, integer coefficient expansion, variable-binding replay, ordered action word, and batch call pays its exact budget at its own occurrence index (**WA13–WA14, WA18–WA19, WA21, WA27–WA28**).

7.1 Paid Execution Traces, Relation-Receipt Boundaries, and Owner-Free Strict Debit

Fix an admitted authoritative root, a registered occurrence o, and an environment–expression pair (ρ, e) read from o. Denote the remaining syntactic node count of e by $r(e) \in \mathbb{N}$: constants cost 0, variables cost 1, unary linear maps add 1 to their argument, and binary additions and biadditive maps add 1 to the sum of their two subexpressions; each lawful primitive reduction eliminates exactly one syntactic node and decrements $r(e)$ by 1.

Definition 7.1 (Paid Execution States, Relation-Receipt Boundaries, and Incremental Effects). A **paid state** is a pair $x = (p, t)$ where $t : e \longrightarrow p$ is the complete ordered evaluation trace already executed under environment ρ, retaining the initial expression e, pending expression p, environment ρ, and every intermediate primitive step; its remaining budget is $r(p)$. Each single-step reduction $p_i \rightarrow p_{i+1}$ emits the formal relation element $[p_i]-[p_{i+1}]$, so the boundary of the trace t telescopes to:

$$\partial t = [e] - [p]. \tag{11}$$

Upon reaching a constant endpoint $p = \mathrm{const}(v)$, evaluation soundness gives $v = \mathrm{ev}_\rho(e)$, and under any environment increment δ the update identity yields:

$$\mathrm{ev}_{\rho+\delta}(\partial t) = \mathrm{eff}_{\rho,\delta}(e). \tag{12}$$

Because constants have zero incremental effect, ∂t evaluates to zero at ρ yet reads out the full nonlinear incremental effect of e as soon as the environment advances to $\rho + \delta$.

Theorem 7.2 (Owner-Free Birth and Strict Syntactic Debit of Mathematical Responsibility Rows, WA13–WA14, WA18). For any authoritative root, registered occurrence o, and input (ρ, e) (requiring no caller-supplied prior bearer row), the extended support partitions the complete live ledger into all prior rows plus one endogenously born mathematical responsibility row (reusing Theorem 6.10 and Supplement S1 of the process-algebra companion [6] for the execution kernel and splicing it into whole-ledger evolution):

1. At step k $(0 \le k \le r(e))$, the paid state $x_k = (p_k, t_k)$ satisfies the exact conservation identity:

$$r(p_k) = r(e) - k, \qquad |t_k| + r(p_k) = r(e). \tag{13}$$

   For every $k < r(e)$, the primitive evaluation step produces a unique same-debt target and strict debit $r(p_{k+1}) < r(p_k)$ on that math row while preserving every prior row via Theorem 3.4's transported remainder; at $k = r(e)$, $|t_{r(e)}| = r(e)$ recovers $\mathrm{ev}_\rho(e)$, emits the boundary $[e] - [\mathrm{const}(\mathrm{ev}_\rho(e))]$ and updated inverse fibre, and never refills budget at zero (admitting no further paid macro-step).
2. In the ten-node bosonic–fermionic (Fock) runtime instance (WA14, whose ten-node initial AST evaluates the creation–annihilation dual-channel pairing sum), the inquiry node at post-birth depth d has budget $10 - (d + 1)$ with trace length plus budget equal to 10; the physical pair is read live from the current occurrence while the pinned trace retains origin environment ρ_0, so the residual at budget exhaustion equals "current physical pair minus origin physical pair," vanishing iff those two complete pairs coincide.

Proof. Equation (11) follows by induction on trace length (empty trace has zero boundary; appending $p \to p'$ sums $([e] - [p]) + ([p] - [p']) = [e] - [p']$), and combining soundness with incremental evaluation yields Equation (12). At the whole-ledger seam, the finite fragment lists only the math row and routes all prior rows to the compiled remainder event; disjoint sum tags lock same-debt target uniqueness, componentwise Σ-folding matches the complete whole-join write, and $r(p_{k+1}) < r(p_k)$ supplies strict same-debt payment (general joint writes invoke Theorem 5.6). Specializing to $r(e) = 10$ at offset $d + 1$ reduces the exhausted pending expression to the origin constant pair, proving the physical residual equivalence (details in Supplement C.1). □

7.2 Exact Charges for Integer Coefficient Expansions, Substitution Replays, and Ordered Action Words

Proposition 7.3 (Exact Syntactic Charges for Integer Expansions, Substitution Body Replays, and Ordered Action Words, WA19, WA21).

1. **Integer coefficient expansion cost**: For any $z \in \mathbb{Z}$ and expression e, repeated addition and unary negation construct the expanded syntax $\mathrm{expand}(z, e)$ with exact node cost:

$$\begin{aligned} r(\mathrm{expand}(z, e)) &= z(r(e) + 1) && (z \geq 0), \\ r(\mathrm{expand}(-(n+1), e)) &= (n+1)(r(e) + 1) + 1 && (n \in \mathbb{N}). \end{aligned} \tag{14}$$

   Every coefficient term, every variable-substitution binding body, and every intermediate trace enters the inventory at its own occurrence position; numerical cancellation never deletes an occurrence index or refunds syntactic charges.

2. **Ordered action words and residual recursion**: For an action word w of length $n = |w|$ with state action A_w ($A_{\ell u} = A_u A_\ell$), linear readout action U_w ($U_{\ell u} = U_u U_\ell$), and source readout M, the total residual $R(w, x) := U_w M x - M A_w x$ satisfies the exact first-letter recursion:

$$R(\ell u, x) = U_u(U_\ell M x - M A_\ell x) + R(u, A_\ell x). \tag{15}$$

   The model-only, full-prefix, and two-branch residual-difference ($U_w M x + (-\mathrm{id})(M A_w x)$) programmes carry exact node costs $n+2$, $n+3$, and $2n+5$, respectively, while the letterwise effect-history programme retaining every pre-action state point and single-step residual pays an exact cumulative cost of $7n + 5$.

Proof. (1) Nonnegative expansion starts at const 0 and adds $r(e)$ internal nodes plus 1 addition node per copy; negative $-(n+1)$ wraps $n+1$ positive copies in 1 unary negation node. Variable substitution induces the compatible environment via target evaluation and recurses over syntax: each variable bind replays the full trace of its binding body while compound nodes concatenate left, right, and parent steps, preserving telescoping boundaries; inventory indexing by occurrence ordinal rather than nonzero value preserves cancelling $+1/-1$ entries separately. (2) Inserting $\mp U_u M A_\ell x$ into $U_u U_\ell M x - M A_u A_\ell x$ proves Equation (15). Counting nodes yields n letter maps plus variable and projection $(n+2)$, $+1$ for prefix retention $(n+3)$, $(n+2)+(n+2)+1 = 2n+5$ for the two-branch residual difference, and $h([]) = 5, h(\ell w) = 7 + h(w) \implies 7n + 5$ for letterwise effect history (Supplement C.2). □

7.3 No Free Pass for Iterative Prefixes, Vector Batches, or Child Queries

Proposition 7.4 (Exact Accounting for Iterative Prefixes, Vector Batches, and Child Queries, WA27–WA28). Fix a syntactic seed e, substitution σ, and iteration count $n \in \mathbb{N}$, and set $\Sigma_n := \sum_{i=0}^{n} r(e[\sigma]^i)$. Then:

1. The sum of prefix trace lengths equals Σ_n, while assembling all $n+1$ prefixes into one vector batch programme has exact node cost

$$\Sigma_n + 2(n+1). \tag{16}$$

2. Each actual call frame j generates an execution trace of length $r(\mathrm{Raw}_j)$ with strict same-debt payment, total call charge equals total call cost, and orbit snapshot environments arise directly from the actual call environments and their adjacent differences (with zero future-oracle input).

3. In branched primary/child queries (WA28), each selected old/next child of internal size m_i pays $4m_i + 1$ (absent-successor branches use const 0 at cost 0), the primary query pays $c_{\text{query}} + c_{\text{oldChild}} + c_{\text{nextChild}} + 3$, and prior actor stock, child relation trees, and primary paid exposures enter the next stock in order.

Proof. Pair lifting preserves node counts node by node, giving prefix trace lengths $r(e[\sigma]^i)$; the vector batch adds 1 linear coordinate-injection node and 1 accumulation addition node at each of the $n + 1$ coordinates, adding $2(n + 1)$ to prove Equation (16). Each call frame reads its own emitted `Raw_j` and spawns an owner-free debt row via Theorem 7.2, while adjacent environment differences recover the finite orbit increments; primary/child query constants expand directly over AST constructors (Supplement C.2). □

8 Order-Preserving Born-Stock Injection, Total-Value Writeback, and Cofinal Stop Coverage

Once every syntactic step pays its budget, the next structural seam is **cross-epoch handoff**: when an evaluation completes and triggers responsibility birth, how do heterogeneous prior histories enter the born ledger without loss, why do repeated identical values never collapse into one slot, and how do cofinal stops avoid skipping intermediate debits? This section closes all three handoff theorems (**WA20, WA22–WA26, WA29–WA32, WA34–WA37**).

8.1 Heterogeneous Readback of Cross-Language Typed Stock and Strictly Monotone Position Injection

Let V_P be the upper domain (e.g., physical or arithmetic) variable family and V_L an independent lower calculation variable family. Complete cross-layer material retains both `Raw` inputs, both paid states and traces, per-variable constant bindings $\sigma(x) = \text{const}(\rho_P(x))$, migrated traces, and written scalar/pair stocks.

Theorem 8.1 (Cross-Language Typed Stock Admission, Heterogeneous Readback, and Order-Preserving Injection, WA20, WA22, WA24–WA26, WA32). For any inquiry frame, source-owned programme configuration, and independent lower variable family V_L:

1. **Universal debt-admission compilation and exact stock readback (WA20, WA24–WA25)**: an actual query generates its residual request and first paid step, compiling directly to `debtAdmission`; eliminating the inherited target projection outcome via heterogeneous equality recovers the exact original cross-layer material record (not merely an equal numerical value), migrating all upper/lower prior events, cochain mixed terms (WA26), and current paid exposures into born stock while preserving all prior projections, whole-ledger first write, and canonical next.
2. **Strictly order-preserving position injection into born stock (WA32)**: combining current query stock H_Q, prior written stock H_o, migrated stock H_m, and upper stock H_P via the juxtaposition constructor seed,

$$H_C = \text{seed}(\text{seed}(H_o, H_Q), H_m), \qquad H_B = \text{seed}(H_P, H_C), \tag{17}$$

maps the j-th occurrence of the query trace to the exact born-stock coordinate

$$\iota(j) = j + |H_o| + |H_P| + 3, \qquad H_B[\iota(j)] = H_Q[j]. \tag{18}$$

The injection ι is strictly increasing and composes injectively into the next actual query—even if two steps evaluate to the exact same value, they occupy two distinct coordinates in H_B.

Proof. (1) Initial row selection certifies sealed initial row authority, and subsequent counts recurse via canonical whole-ledger destinations without external indices. Installing complete material as a frame projection yields `HEq` between target and source outcomes; casting along payload type equality and eliminating the empty inactive branch recovers the literal record. Structural induction on $\sigma(x) = \mathrm{const}(\rho_P(x))$ proves $\mathrm{ev}_{\rho_L}(e[\sigma]) = \mathrm{ev}_{\rho_P}(e)$, and cochain mixed terms index injectively by list position. (2) Each $\mathrm{seed}(A, B)$ prepends one seam root event before concatenating left and right traces; three nested seams contribute $+3$ plus left prefix lengths $|H_P| + |H_o|$, giving Equation (18), where natural-number cancellation proves injectivity and strict monotonicity (Supplements C.3 and D.1). □

8.2 Same-Receipt Total-Value Writeback, Fresh Baselines, and Independent Difference Charges

Proposition 8.2 (Same-Receipt Total-Value Writeback, Two-Birth Advancement, and Independent Difference Accounting, WA23, WA29–WA31, WA34).

1. **Same-receipt total writeback and fresh baseline regeneration (WA23, WA29, WA34)**: for any supplied occurrence's native execution with prior paid value $v_{\mathrm{old}} = \mathrm{ev}_{\rho}(e)$ and completed residual value $v_{\mathrm{native}} = \mathrm{eff}_{\rho,\delta}(e)$, the source-generated decoder writes the exact total value into the born environment:

$$v_{\mathrm{new}} = v_{\mathrm{old}} + v_{\mathrm{native}} = \mathrm{ev}_{\rho+\delta}(e). \tag{19}$$

When $v_{\mathrm{native}} = 0$, Equation (19) still writes v_{old} without refunding syntax payment; each subsequent birth regenerates a fresh decoder and baseline v^{+}_{old} from its own completion receipt (never freezing the initial environment), so two consecutive births advance two genuine history stages (e.g., from `stage` through the initial active environment to `stage + 2`).

2. **Independent fee and joint-stock writeback for two-execution boundary differences (WA30–WA31)**: across two actual executions, the difference of their complete boundaries spawns its own integral-coefficient AST at the grandnext occurrence and pays its full syntactic cost under Proposition 7.3; its effect, updated inverse residual, and every new paid exposure embed sequentially through pair-written and native stocks into `jointStock`, keeping all three occurrences distinct rather than turning the difference into a free observation.

Proof. Evaluation soundness gives $v_{\mathrm{old}} = \mathrm{ev}_{\rho}(e)$ and Equation (12) gives $v_{\mathrm{native}} = \mathrm{eff}_{\rho,\delta}(e)$, whose sum equals $\mathrm{ev}_{\rho+\delta}(e)$ by incremental decomposition, proving Equation (19). The decoder fires exclusively in the settled branch selected by that occurrence's own receipt, the next packet re-reads its own action result to refresh v^{+}_{old}, and typed child seeds align `target_root`, `whole_first`, and `physicalNext` within one `generatedAction` (Supplements D.2 and D.5). Subtracting the two closure

boundaries invokes the independent coefficient executor and three injective stock embeddings into `jointStock`. □

8.3 Exact Completion Distance, Full-Segment Cofinal Stops, and the Maximal Future Kernel

Theorem 8.3 (Exact Completion Distance, Full-Segment Cofinal Coverage, and Maximal All-Future Kernel, WA32, WA35–WA37).

1. **Exact receipt distance and intermediate paid steps (WA32, WA35)**: from any valid frame start with remaining budget B, the `Type`-valued completion receipt yields distance $d = B$ to settlement, with every intermediate offset $0 \le h < B$ holding an actual paid debit step; `StockTarget` compiles paid steps to `answered`/`mathNext` and settled steps to `debtAdmission`/`nextBorn` (forbidding erasure of births along `answered`).
2. **Cofinal stops cover every discrete runtime step (WA35–WA36)**: letting $d(t)$ be the distance from discrete step t to the first admission stop $\ge t$ (0 at admissions, $B+1$ at continuations with budget B), the stop sequence

$$t_0 = d(0) = 0, \qquad t_{j+1} = t_j + 1 + d(t_j + 1), \qquad \ell_j = 1 + d(t_j + 1) \tag{20}$$

   satisfies $t_j + \ell_j = t_{j+1}$, $t_{j+1} > t_j$, and $t_j \ge j$ (strictly monotone and cofinal), so adjacent stop segments cover every intermediate discrete step while transporting `Raw`, environment, paid state, trace, bearer row, and all six algebraic, combinatorial, topological, logical, relation, and cochain faces to every stop.
3. **Whole-word action and maximal all-future invisible kernel (WA37)**: with j-th stop word module $\mathcal{W}_j$, local paired readout d_j, and cross-stop word action $S_j : \mathcal{W}_j \to \mathcal{W}_{j+1}$, on $\mathcal{W} = \bigoplus_j \mathcal{W}_j$ (readouts $D_j = d_j \circ \mathrm{pr}_j$, global action S) the joint kernel across all finite allowed action words $a \in \{*\}^{<\omega}$ and sites $j \in \mathbb{N}$,

$$K_{\mathrm{fut}} = \bigcap_{a \in \{*\}^{<\omega}} \bigcap_j \ker(D_j S_a), \qquad \mathcal{M}_{\mathrm{fut}} = \mathcal{W}/K_{\mathrm{fut}}, \tag{21}$$

   satisfies $S(K_{\mathrm{fut}}) \subseteq K_{\mathrm{fut}}$, descends to the quotient model action, and forms the **largest S-invariant submodule** invisible to current readouts.

Proof. (1) Strong induction on B: settled gives $d = 0$; paid steps to `mathNext` with budget $B - 1$ and increments d, where $B - h > 0$ for $h < B$ rules out settlement, and budget 0 at the endpoint with $d \le B$ forces $d = B$. (2) Equation (20) inductively gives $t_j \ge j$, and choosing the least j with $t \le t_{j+1}$ places any step t inside segment $[t_j, t_{j+1}]$. (3) For $w \in K_{\mathrm{fut}}$, $D_j S_a(Sw) = D_j S_{a*}(w) = 0$ proves S-invariance, and word-length induction embeds any currently invisible S-invariant submodule into K_{fut} (Supplements D.3–D.4). □

9 Endogenous Requests from All-Future Residuals and Strict Same-Debt Completion

With paid syntactic accounting and the maximal future model $\mathcal{M}_{\mathrm{fut}}$ in place, we establish the first constructive climax of the paper: **how the ledger—without any external scheduler or choice of quotient representatives—turns its semantic residual in $\mathcal{M}_{\mathrm{fut}}$ directly into the next native request, strictly debits the same debt from its own syntactic credit, and delivers upon suffix completion the incremental effect, receipted answer, whole-ledger destinations, and canonical successor together (WA38–WA39).**

9.1 Multi-Sorted Nonlinear Incremental Evaluation and the All-Future Residual Request Theorem

Let S be a sort family with abelian groups V_s, variable types X_s, and environments $\Gamma : \prod_{s\in S}(X_s \to V_s)$. Source expressions are generated by variables, constants, addition, unary homomorphisms f, and biadditive maps B. Reserving $e \in L(o)$ for open responsibility rows from this section onward, we write a, Γ, $[\![a]\!]_\Gamma \equiv \mathrm{ev}_\Gamma(a)$, $D_a(\Gamma,\delta) \equiv \mathrm{eff}_{\Gamma,\delta}(a)$, $c(a) \equiv r(a)$, and τ for the expression, environment, evaluation, incremental effect, node cost, and trace of Section 7.1's evaluation kernel. Recursive incremental differentiation $D_a(\Gamma,\delta)$ sets $D_x = \delta(x)$, $D_{\mathrm{const}\,v} = 0$, $D_{a+b} = D_a + D_b$, $D_{f(a)} = f(D_a)$, and retains the full nonlinear cross term $D_{B(a,b)} = B([\![a]\!]_\Gamma, D_b) + B(D_a, [\![b]\!]_\Gamma) + B(D_a, D_b)$. Structural induction proves the exact update identity:

$$[\![a]\!]_{\Gamma+\delta} = [\![a]\!]_\Gamma + D_a(\Gamma,\delta). \tag{22}$$

Syntactic node cost $c(a) \in \mathbb{N}$ assigns 1 to variables, 0 to constants, +1 to unary maps, and +1 to additions and biadditive maps; each lawful reduction $a \rightsquigarrow_\Gamma a'$ preserves evaluation and strictly drops cost by 1:

$$c(a) = c(a') + 1. \tag{23}$$

Hence every paid state $\sigma = (a',\tau)$ $(\tau : a \rightsquigarrow_\Gamma^* a')$ satisfies:

$$[\![a']\!]_\Gamma = [\![a]\!]_\Gamma, \qquad |\tau| + c(a') = c(a). \tag{24}$$

As checked by hand in Example 1.2(c), for $a = x \cdot x$ under $\Gamma(x) = 2, \delta(x) = 2$, Equation (22) gives $4^2 = 2^2 + (2 \cdot 2 \cdot 2 + 2^2) = 4 + 12$, capturing the cross term $D_a \cdot D_b = 4$ exactly.

At an exact temporal occurrence o (with complete ledger $L(o)$ and next write E_o), a **paid residual material** tuple is

$$m = (\Gamma, \delta, a, \sigma = (p,\tau), e), \tag{25}$$

where $e \in L(o)$ is the prior bearer row and $\beta_m = [a] - [p] \in F(V, X; s)$ is the paid relation boundary in the free $\mathbb{Z}$-word module. The endogenously generated **literal residual AST** and **writable native request** are:

$$r_m = \mathrm{add}(a, \mathrm{linear}(-\mathrm{id}, p)), \qquad Q_m = (\Gamma + \delta, r_m, e). \tag{26}$$

For every environment Θ and at the prior environment Γ (when $p = \mathrm{const}[\![a]\!]_\Gamma$), direct evaluation gives:

$$[\![r_m]\!]_\Theta = [\![a]\!]_\Theta - [\![p]\!]_\Theta, \qquad [\![r_m]\!]_\Gamma = 0, \qquad c(r_m) = c(a) + c(p) + 2. \tag{27}$$

Theorem 9.1 (All-Future Residual Equivalence and Endogenous Request Generation at the Canonical Successor, WA38). Fix any source configuration family $\mathcal{F}$ (`Source Family.Factory`), complete initial frame I, programme configuration cfg, and language identity $\mathsf{cfg.LowVar} = X$. Let P_n ($n \in \mathbb{N}$) be the complete packet at the n-th cofinal admission stop of Theorem 8.3, f_n the settled frame selected by the native receiver's stop receipt, m_n its paid material, and G_n the birth action generated at that occurrence (target T_n, first whole-ledger write $E_n = T_n.\mathsf{firstSuccessor.ledgerEvolution}$, canonical next current $c_n^+ = T_n.\mathsf{targetAnswerAndNext.nextCurrent}$). Then for every $n \in \mathbb{N}$:

1. **Canonical current identification and representative-free request generation**: the runtime's actual next admission current equals c_n^+, carries transported material m_n^+ from P_{n+1}, and directly forms the writable request $q_n = (m_n^+.\Gamma, m_n^+.a, m_n^+.e)$ without choosing any representative from a future quotient class.
2. **All-word, all-site equivalence in $\mathcal{M}_{\mathrm{fut}}$**: letting t_n denote dependent word-module transport along language and next-packet equalities,

   $$[q_n.\mathrm{AST}] = \mathsf{t}_n[r_{m_n}], \qquad b_n = \mathsf{t}_n \beta_{m_n}, \tag{28}$$

   and the two represent the exact same class in $\mathcal{M}_{\mathrm{fut}}$—for every finite allowed action word $h \in \{*\}^{<\omega}$ and every site $j \in \mathbb{N}$,

   $$D_j\big(S_h(\iota_{n+1}[q_n.\mathrm{AST}])\big) = D_j\big(S_h(\iota_{n+1} b_n)\big)\,. \tag{29}$$

3. **Shared first write for prior and born rows**: both prior bearer row e_n and the newly born row receive exact destinations from the same whole-ledger write E_n:

   $$E_n.\,\mathrm{destination}(\mathrm{oldEntry}(e_n)), \qquad E_n.\,\mathrm{destination}(\mathrm{debtEntry}(m_n)), \tag{30}$$

   and any dependent readout consuming q_n together with its future-equivalence proof, compilation equality, and whole-ledger write eliminates directly from this single source-generated object.

Proof. Define the **universal semantic-zero submodule** of the free $\mathbb{Z}$-word module:

$$Z(V, X; s) = \{w \in F(V, X; s) : \forall \Theta,\ \mathrm{ev}_\Theta(w) = 0\}. \tag{31}$$

By Equations (26)–(27), $\mathrm{ev}_\Theta([r_m] - ([a] - [p])) = ([\![a]\!]_\Theta - [\![p]\!]_\Theta) - ([\![a]\!]_\Theta - [\![p]\!]_\Theta) = 0$ for every Θ, so

$$[r_m] - ([a] - [p]) \in Z(V, X; s). \tag{32}$$

Crucially, what vanishes across all environments is the **difference** between $[r_m]$ and $\beta_m = [a] - [p]$, not r_m itself! For any $w \in Z(V, X; s)$ and (Γ, δ), $\mathrm{ev}_{\Gamma+\delta}(w) = \mathrm{ev}_\Gamma(w) + \mathrm{eff}_{\Gamma,\delta}(w)$ with the first two

terms zero forces $\mathrm{eff}_{\Gamma,\delta}(w) = 0$, so literal pair lifting evaluates to $(0, 0)$ in every pair environment and thus preserves Z, as do language and packet transports. Inducting on the length of h, every site transition maps Z into the next site's Z, rendering Equation (32) invisible at all (h, j). Combining this with the next-packet recursion

$$P_{n+1} = \mathrm{languageTransport}(\mathrm{sourceBorn}(\mathcal{F}, P_n)) \tag{33}$$

proves Equations (28)–(29). Simultaneously, $f_n.\mathsf{action} = \mathsf{settled}$ compiles via `StockTarget` to $\mathsf{debtAdmission}(G_n)$, whose target-current law identifies the next admission current with c_n^+, transporting P_{n+1}'s material to m_n^+ and delivering both destinations in Equation (30). □

9.2 Incremental Effect Recognition, Positive-Credit Same-Debt Debit, and Four-Port Suffix Completion

From any $\mathcal{F}$ in Theorem 9.1, installing a paired computation reader prior to the emitter yields a derived family $\widehat{\mathcal{F}}$ exposing old value and incremental effect at the same current (while general residual payment and completion already hold for arbitrary $\mathcal{F}$). At settled frame $f = f_n$ with material $m = (\Gamma, \delta, a, (p, \tau), e)$, the paired reader outputs

$$\mathsf{Raw}_{f,o} = (\mathrm{pairEnv}(\Gamma, \delta), \mathrm{lift}(a)), \qquad u = (\llbracket a \rrbracket_\Gamma, D_a(\Gamma, \delta)). \tag{34}$$

Set $\epsilon := D_a(\Gamma, \delta)$, $b := \mathrm{born}(f, \mathsf{base})$, and $Q := c(r_m)$.

Theorem 9.2 (Source-Native Effect Recognition and Positive-Credit Strict Same-Debt Debit, WA39). Under the above setup:

1. The registered input of born frame b equals $Q_m = (\Gamma + \delta, r_m, e)$ of Equation (26); the next canonical root request evaluates to $\epsilon = D_a(\Gamma, \delta)$, matching the second coordinate of u in Equation (34).
2. Having executed its birth first step, b has positive remaining syntax cost $Q - 1 > 0$, forcing its actual action into the paid branch to produce a same-debt payment π_b with unique target, strict debit, and no refill whose row evolution agrees heterogeneously with the born whole-ledger destination.

Proof. The current-material reader extracts from the same settled payload:

$$\mathrm{pairEnv}(\mathsf{raw.Env}, \mathsf{nextRaw.Env} - \mathsf{raw.Env}), \qquad \mathrm{lift}(\mathsf{raw.AST}). \tag{35}$$

At the stop-selected settled occurrence, $p = \mathrm{const}\, v$ with $v = \llbracket a \rrbracket_\Gamma$ and $\mathsf{nextRaw.Env} = \Gamma + \delta$. Equations (22) and (26) yield:

$$\begin{aligned} \llbracket r_m \rrbracket_{\Gamma+\delta} &= \llbracket a \rrbracket_{\Gamma+\delta} - v \\ &= \llbracket a \rrbracket_\Gamma + D_a(\Gamma, \delta) - v = \epsilon. \end{aligned} \tag{36}$$

Since the material increment is active minus registered environment,

$$\Gamma + \delta = \Gamma + (f.\mathsf{activeEnvironment} - \Gamma) = f.\mathsf{activeEnvironment}, \tag{37}$$

identifying b's registered input with Q_m and its transported request value with ϵ. For payment: at birth depth $b.\mathsf{depth} = 0$ (one step elapsed), remaining cost is

$$c(\mathsf{event}_b.\mathsf{state.expression}) = c(b.\mathsf{registered.AST}) - (b.\mathsf{depth} + 1) = Q - 1. \tag{38}$$

By Equation (27), $Q = c(a) + c(p) + 2 \geq 2$, so $Q - 1 \geq 1 > 0$! Were $b.\mathsf{action}$ settled, remaining cost would be 0, a contradiction; eliminating $b.\mathsf{action}$ therefore yields an actual paid step, disjoint sum tags lock same-debt uniqueness, and Equation (23) gives strict debit. Finally, matching the underlying ledger root and finite visit recursion between the payment process and enriched stock root eliminates support and evolution equalities to give:

$$\pi_b.\mathsf{step.evolution} \;\simeq\; (E_b.\,\mathrm{destination}(e_b))\,.\mathsf{evolution}, \tag{39}$$

where e_b is the canonical math entry at b. □

Theorem 9.3 ($(Q-1)$-Step Suffix Completion, Four-Port Delivery, and Canonical Successor, WA39). With the notation of Theorem 9.2:

1. Advancing from born state $\mathcal{R}(1)$ along the $(Q-1)$-step suffix history H_b hits the exact completed runtime $\mathcal{R}(Q)$ with completion value $\epsilon = D_a(\Gamma, \delta)$ and total paid trace length Q.
2. At $\mathcal{R}(Q)$ the actual action is mathematical settlement, which `StockTarget` compiles into responsibility admission G_b^{end}, simultaneously delivering four compatible, distinctly typed outputs from that single event: (i) numerical effect ϵ (heterogeneously equal to the rooted paid state evaluation), (ii) receipted action answer $G_b^{\mathrm{end}}.\mathsf{answer}$, (iii) canonical causal answer and next current $\mathsf{targetAnswerAndNext}$, and (iv) both prior and born row destinations in E_b^{end}.

Proof. Denoting the sealed runtime at count k by $\mathcal{R}(k)$, step additivity and cost countdown give:

$$\mathcal{R}(k).\,\mathrm{advance}(\ell) = \mathcal{R}(k+\ell), \qquad c(\mathsf{event}(k).\mathsf{state.expression}) = Q - k. \tag{40}$$

Advancing $Q-1$ steps from $\mathcal{R}(1)$ without restarting hits

$$H_b.\mathsf{target} = \mathcal{R}(1).\,\mathrm{advance}(Q-1) = \mathcal{R}(Q). \tag{41}$$

At $k = Q$, remaining cost is 0, ruling out any paid branch (which would require a natural number < 0) and forcing settlement with value $[\![r_m]\!]_{\Gamma+\delta} = \epsilon$. Running the birth compiler at this exact temporal event produces $\mathsf{debtAdmission}(G_b^{\mathrm{end}})$ (target T_b^{end}), whose first whole-ledger write E_b^{end} supplies both destinations:

$$E_b^{\mathrm{end}}.\,\mathrm{destination}(T_b^{\mathrm{end}}.\mathsf{translatedSourceEntry}), \qquad E_b^{\mathrm{end}}.\,\mathrm{destination}(T_b^{\mathrm{end}}.\mathsf{bornEntry}), \tag{42}$$

while identifying the next born packet's current and preserving the generated living law. □

9.3 Paired Negative Controls: Zero Numerical Effect Still Pays; Fresh Debts Never Refill Old Debts

Proposition 9.4 (Positive Syntax Payment at Zero Effect and Sum-Tag Isolation Across Debts, WA39). Take constant material $a = \mathrm{const}\, v$ and paid endpoint $p = \mathrm{const}\, v$. Under any increment δ, numerical effect vanishes ($\epsilon = D_a(\Gamma, \delta) = 0$), yet the next request $r = \mathrm{add}(\mathrm{const}\, v, \mathrm{linear}(-\mathrm{id}, \mathrm{const}\, v))$ still carries cost $c(r) = 2$: the birth first step reduces negation to leave 1, and the next actual payment step reduces addition with strict debit $1 \to 0$ to reach $\mathrm{const}\, 0$. When completion spawns a further fresh request (e.g., $r - \mathrm{const}\, 0$, cost 4), disjoint sum tags place prior debts in `inl` and fresh debts in `inr`, preventing fresh positive budget from ever refilling the prior zero-budget row.

Proof. Direct node counting gives $c(r) = 0 + 0 + 2 = 2$, so $Q - 1 = 1 > 0$ in Equation (38) forces a $1 \to 0$ strict debit by Theorem 9.2(2). Disjointness of `inl` and `inr` confines $\sim_{\mathrm{debt}}$ to each lineage branch separately (Figure 3). □

Proposition 9.4 draws a crisp semantic boundary against Quantitative Type Theory (QTT) [14] and static λ-differentiation [21]: QTT's Lemma 2.3 (*Zero needs nothing*: $\Gamma \vdash M \overset{0}{:} S \implies 0\Gamma = \Gamma$) forces a statically zeroed context to erase compile-time terms that execute zero runtime steps, while static differentiation Derive(t) [21] (Theorem 3.11) targets extensional update equality; on a live runtime ledger, however, zero numerical effect ($\epsilon = 0$) never implies zero operational steps—the residual AST r_m carries positive syntactic credit and executes two strict same-debt reductions along its own lineage.

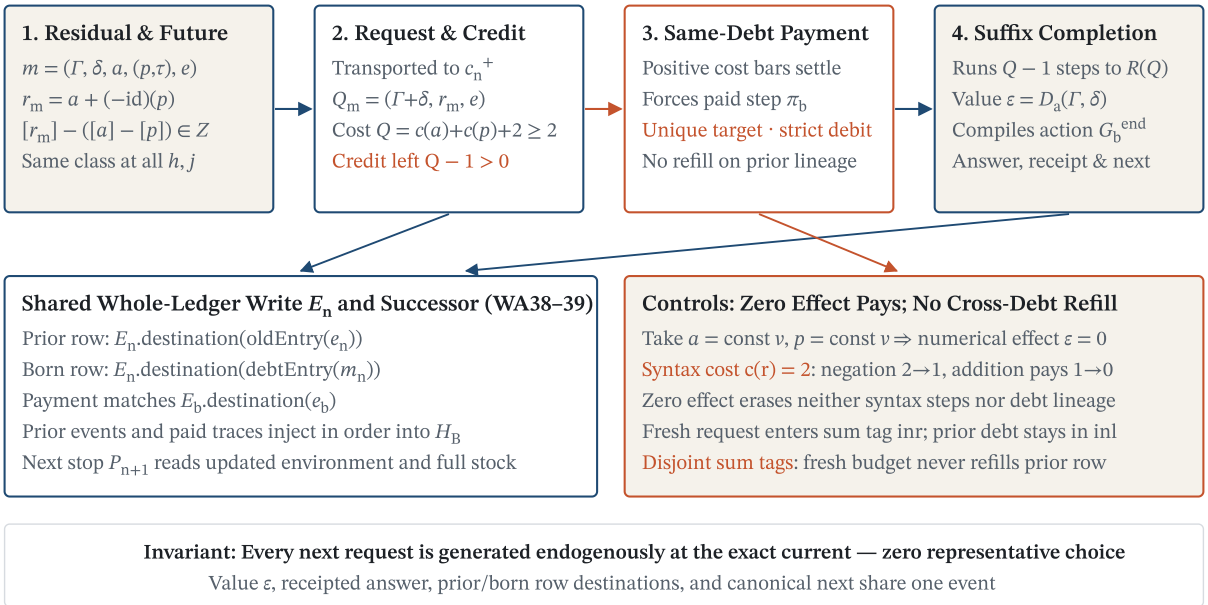


Figure 3 | All-Future Residual Request, Strict Same-Debt Debit, and Suffix Completion. Top row (Theorems 9.1–9.3): literal residual r_m and boundary $[a] - [p]$ agree across all allowed words and sites in $\mathcal{M}_{\mathrm{fut}} \to$ transport along the exact current equality yields request Q_m with positive remaining credit $Q - 1 > 0 \to$ forces unique-target strict same-debt debit $\pi_b \to (Q-1)$-step suffix completion at $\mathcal{R}(Q)$ delivers effect ϵ, receipt, and canonical successor. Bottom left: prior and born rows share whole-ledger write E_n. Bottom right (Proposition 9.4): $\epsilon = 0$ still pays $c(r) = 2$ steps, and disjoint `inl`/`inr` sum tags bar cross-debt budget refill.

10 Law-Free Well-Foundedness of Paid Macro-Steps

We now confront the central multi-step challenge: **if obligations may lawfully migrate across bearers, supports, and presentations, what stops an infinite carousel of "transfer–rename–transfer" from impersonating an endless payment stream?** Crucially, does proving finiteness of payment chains require assuming a global termination law in advance? Our answer is no: **enclosing a single strict debit between finite non-refilling histories makes the entire paid macro-step relation strictly well-founded with zero global closure premises (WA7).**

10.1 Fixed-Debt Current Space, Unique-Target Micro-Steps, and Enclosed Paid Macro-Steps

Definition 10.1 (Fixed-Debt Current Space and Same-Debt Micro-Steps). Fix a living process P (state type S_P, successor $\sigma_P : S_P \to S_P$, complete ledger $\mathcal{L}_P(s) := \mathcal{L}(u_P(s))$) and an original debt a. The **same-debt current space** and natural-number budget readout are:

$$X_a := \sum_{s:S_P} \sum_{e:\mathcal{L}_P(s)} (a \sim_{\text{debt}} e), \qquad B(s, e, -) := b(e) \in \mathbb{N}. \tag{43}$$

A **same-debt micro-step** from $x = (s, e, -) \in X_a$ consists of a target entry $f \in \mathcal{L}_P(\sigma_P(s))$, a lawful row evolution $e \to f$, and **uniqueness of the same-debt target** in $\mathcal{L}_P(\sigma_P(s))$ (every f' with $e \sim_{\text{debt}} f'$ equals f, forbidding following only one branch of a split). By Proposition 5.2, each micro-step preserves $\sim_{\text{debt}}$ and satisfies $B(x') \le B(x)$; inducting on path length shows any **finite non-refilling history** $x \rightsquigarrow^* u$ satisfies $B(u) \le B(x)$.

Definition 10.2 (Strict Payment and Enclosed Paid Macro-Step). A **strict payment** $u \xrightarrow{\text{pay}} u'$ is a same-debt micro-step equipped with $B(u') < B(u)$. A **paid macro-step** (`PaidMacro`) concatenates a finite non-refilling pre-history, one strict payment, and a finite non-refilling post-history:

$$x \rightsquigarrow^* u \xrightarrow{\text{pay}} u' \rightsquigarrow^* y. \tag{44}$$

Pre- and post-histories may be empty or contain arbitrarily many carries, rechartings, and accepted transfers; the macro-step constructor **accepts no external $B(y) < B(x)$ field**, deriving descent purely from the enclosed payment.

10.2 Law-Free Well-Foundedness Main Theorem and Refutation of Equal-Budget Wash Steps

Main Theorem 10.3 (Law-Free Well-Foundedness of Paid Macro-Steps, WA7). For any fixed (P, a), write $y \prec_a x$ iff there exists a paid macro-step (Equation (44)) from x to y. With zero total emission law, preset endpoint, or future table assumed:

1. **Strict well-foundedness**: $\prec_a$ is well-founded on X_a (`sourceNativePaidRootDebtContinuationRel_wellFounded`), admitting no infinite descending chain $x_0 \succ_a x_1 \succ_a \cdots$;
2. **Irreflexivity**: $\neg(x \prec_a x)$ for every $x \in X_a$ (`no_paidRootDebtMacro_self_return`)—no genuinely paid debt can ever loop back to the exact same complete debt current;

3. **Zero-budget ceiling**: whenever $B(x) = 0$, the paid macro-step type from x is uninhabited.

Proof. Path induction via Proposition 5.2 along $x \rightsquigarrow^* u$ and $u' \rightsquigarrow^* y$ gives $B(u) \leq B(x)$ and $B(y) \leq B(u')$; sandwiching $B(u') < B(u)$ in between yields:

$$B(y) \leq B(u') < B(u) \leq B(x), \qquad B(y) < B(x). \tag{45}$$

Equation (45) exhibits $\prec_a$ as a subrelation of the inverse image of $(\mathbb{N}, <)$ along $B : X_a \to \mathbb{N}$. Because $(\mathbb{N}, <)$ is constructively well-founded, its pullback and every subrelation thereof are well-founded, proving (1). Setting $y = x$ in Equation (45) gives $B(x) < B(x)$ (contradiction, proving (2)), and setting $B(x) = 0$ gives $B(y) < 0$ in $\mathbb{N}$ (contradiction, proving (3)). $\square$[3]

Proposition 10.4 (Direct Refutation of Any Equal-Budget Micro-Step as Payment, WA7). Fix any actual micro-step $e : u \to u'$ with $B(u') = B(u)$. The type of candidate paid macro-steps designating e as their central payment in Definition 10.2 is empty; wrapping arbitrary finite pre- and post-histories around e leaves the endpoints u, u' and their equal budgets unaltered (exemplified concretely by the $0 \to 0$ unit arithmetic transfer in Section 13).

Proof. Supplying e as a strict payment requires a proof of $B(u') < B(u)$, contradicting $B(u') = B(u)$; outer history concatenation shifts only x, y without touching e's endpoints u, u'. $\square$

Compared with disjunctive Transition Invariants [18] (Definition 1 and Theorem 1: global termination holds iff the reachable transitive closure satisfies $R^+ \cap (\mathrm{Acc} \times \mathrm{Acc}) \subseteq T_1 \cup \cdots \cup T_n$ for well-founded relations T_i over state pairs), Main Theorem 10.3 and Proposition 10.4 lift well-foundedness from bare state pairs to the lineage-locked dependent space X_a, filtering out $b \to b$ wash transfers and proving same-debt well-foundedness in **zero axioms with zero global termination premise**, reserving finality for Section 11's two-key emission law.

11 The Two-Key Final Settlement Theorem

Main Theorem 10.3 proves that the same debt cannot be strictly paid forever, yet that closes only half the door: a debtor whose budget hits 0 may stall indefinitely without paying further and without holding any lawful discharge! **What separates budget exhaustion, local termination, and true final settlement?** This section proves that final settlement requires turning two independent keys at once (**WA8**).

11.1 Key 1 (Faithful Compiler Terminal Receipt) and Key 2 (Empty Same-Debt Fibre in Actual Successor)

Definition 11.1 (Faithful Local Terminal and Two-Key Final Settlement). For $x = (s, e, -) \in X_a$:

1. **Key 1—faithful local terminal** $\mathrm{LocalTerm}(x)$: an occurrence emitted at s carrying proof of local-terminal structural compilation, rowwise whole-ledger discharge matching the whole-ledger compiler, and Definition 2.2's ordinary outgoing-edge exhaustion (projecting to e's exact row terminal receipt).

[3] The shared base debt-settlement construction appears in [6], Theorem 6.7.

2. **Key 2—empty same-debt fibre** $\mathrm{IsEmpty}(T(x))$ in the actual successor, defining **final settlement (`Settle`)** as:

$$T(x) := \sum_{f:\mathcal{L}_P(\sigma_P(s))} (e \sim_{\mathrm{debt}} f), \qquad \mathrm{Settle}(x) := \mathrm{LocalTerm}(x) \times \mathrm{IsEmpty}(T(x)). \tag{46}$$

Here $\sigma_P(s)$ is the actual successor state reached by P under its pre-fixed terminal handoff law—a verifier cannot swap in a fresh debt-free world after seeing Key 1 to fake Key 2.

Proposition 11.2 (One Key Alone Never Certifies Settlement, WA8). Every $q : \mathrm{Settle}(x)$ projects to e's exact terminal receipt; conversely, even when $\mathrm{LocalTerm}(x)$ is inhabited, any inhabitant $t \in T(x)$ in the actual successor $\sigma_P(s)$ (e.g., the same lineage and claim reincarnated on a new carrier) renders $\mathrm{Settle}(x)$ empty.

Proof. Forward projection evaluates $q.1$'s rowwise discharge at e; conversely, $q.2 : T(x) \to \bot$ eliminates $t \in T(x)$ to $\bot$. Equation (46) requires only that *this settled debt* a be absent from $\sigma_P(s)$, allowing all other world obligations to continue undisturbed. □

11.2 Per-Current Total Emission Law and Well-Founded Construction of Finite Settlement Histories

Definition 11.3 (Total Emission Closure Law). For fixed (P, a), a **total emission closure law** is an explicit dependent function:

$$\mathrm{emit} : \prod_{x:X_a} (\mathrm{Settle}(x) \sqcup \mathrm{PaidMacro}(x)), \tag{47}$$

delivering at every $x \in X_a$ either exact two-key settlement or a completed paid macro-step.

Main Theorem 11.4 (Total Emission Laws Generate Finite Two-Key Settlement Histories, WA8). Given emit as in Definition 11.3, every $x \in X_a$ possesses a finite generated history $H(x)$ aligned branch-by-branch with emit whose endpoint $y \in X_a$ carries $\mathrm{Settle}(y)$—simultaneously certifying the original compiler's rowwise terminal receipt and emptiness of $T(y)$ in the actual successor $\sigma_P(s_y)$.

Proof. Define the inductive family $H(x)$ with terminal leaf taking $(q : \mathrm{Settle}(x), \mathrm{emit}(x) = \mathrm{inl}(q))$ and continuation node taking $(m : \mathrm{PaidMacro}(x), \mathrm{emit}(x) = \mathrm{inr}(m), H(m.\mathrm{target}))$. Recursing constructively along Main Theorem 10.3's well-founded relation $\prec_a$: evaluating $\mathrm{emit}(x)$ closes immediately on `inl(q)`, and on `inr(m)` uses $m.\mathrm{target} \prec_a x$ to recurse exclusively at $m.\mathrm{target}$. Projecting to the finite leaf extracts $\langle y, \mathrm{Settle}(y)\rangle$ and its exact row receipt (Figure 4). □

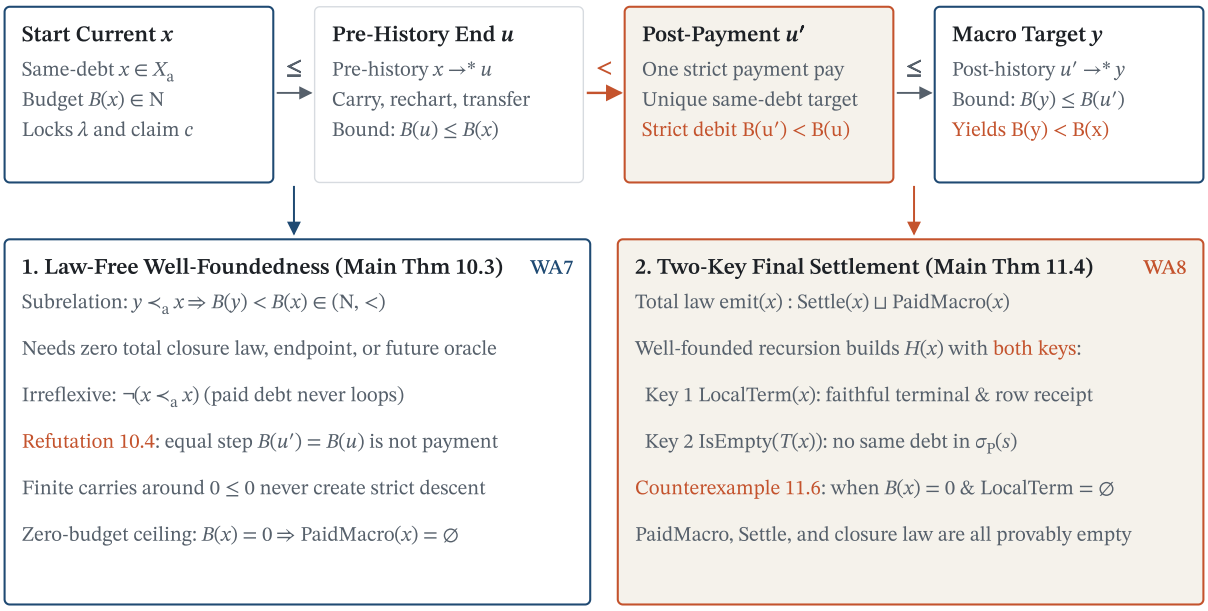


Figure 4 | Law-Free Paid Macro-Step Well-Foundedness and Two-Key Final Settlement. Top chain: macro-step $B(y) \le B(u') < B(u) \le B(x)$ enclosing one strict payment `pay`. Bottom left (Main Theorem 10.3, Proposition 10.4): law-free well-foundedness and irreflexivity of $\prec_a$, refuting equal-budget wash transfers. Bottom right (Main Theorem 11.4, Corollary 11.5, Example 11.6): well-founded recursion along emit builds finite history $H(x)$ ending with both keys (`LocalTerm(x)` and `IsEmpty(T(x))`), while zero budget with an empty local terminal empties all three closure types simultaneously.

11.3 Exact Zero-Budget Divide, Fluid-Medium Empty-Type Counterexample, and Stage-Three Instance

Corollary 11.5 (Zero-Budget Divide and Emptiness of Total Closure Laws Without Local Terminals, WA8). If $B(x) = 0$ and a total law emit exists, $\mathrm{emit}(x)$ must emit $\mathrm{Settle}(x)$; conversely, if some $x \in X_a$ has $B(x) = 0$ while $\mathrm{LocalTerm}(x)$ is empty, the entire total closure law type $\prod_{u:X_a} (\mathrm{Settle}(u) \sqcup \mathrm{PaidMacro}(u))$ is **uninhabited**.

Proof. When $B(x) = 0$, Main Theorem 10.3(3) empties $\mathrm{PaidMacro}(x)$, forcing Equation (47) into `inl`. If $\mathrm{LocalTerm}(x)$ is also empty, $\mathrm{Settle}(x)$ is empty as well, so evaluating any putative emit at x yields $\bot$. □

Example 11.6 (Counterexample: Zero-Budget Operational Row on a Continuing Fluid Medium, WA8). Across every admissible parameter and stage of the continuous fluid-medium source, its operational responsibility row has budget $B(x) = 0$ while its local-terminal payload type is `PEmpty` (the fluid continues stepping without termination). By Main Theorem 10.3 and Corollary 11.5, $\mathrm{PaidMacro}(x)$, $\mathrm{Settle}(x)$, and the total emission law type are **all provably empty**: zero budget neither halts dynamics nor manufactures a settlement certificate.

Example 11.7 (Stage-Three Incidence Payment Consuming a Total Emission Law, WA8). Given a stage-three incidence payment receipt fixing its pending row (budget 1) and installed target (budget 0), compilation produces the unique same-debt $1 \to 0$ strict debit; the installed current emits its faithful local terminal and transitions to a successor containing only prior `inl` claims (`T(x)` empty). Dispatching the paid macro-step on the pending state and two-key

settlement on the installed state defines emit across X_a, and Main Theorem 11.4 outputs the finite settlement history and receipt.

Four notions routinely conflated under "done"—whole-ledger rowwise answer (Section 3), strict same-debt payment (Section 10), faithful local terminal (Section 2.2), and two-key final settlement (Section 11)—are now sharply separated at the type level.

12 Shared First-Write Continuity for Old Debts and New Responsibilities Across Language Revisions

What if the world language itself goes bankrupt—encountering a rooted failure whose expression fibre and lawful realization fibre in the old vocabulary are both empty, forcing an upgrade to a richer language and new authoritative root? **Can historical debts slip through the translation seam during regime change?** This section proves that under both minimal initial-coface revisions and richer split-inducing revisions, every migrated prior debt and the newly generated direction enter the revised root's **same nonterminal first write** and share its canonical next current (**WA9**).

12.1 Rooted Double Emptiness, Zero-Axiom Minimal Coface Extension, and Unique Factorization

Definition 12.1 (Rooted Double-Empty Obstruction and Debt-Carrying Structurally Anchored Revision). A typed world revision starts from an exact causal obligation on the old root, a rooted expressibility obstruction (proving **both** expression and realization fibres empty in the old language), and its prior successor support. The revision constructs a new network, new authoritative root, and nonterminal first whole-ledger write: the new initial support recovers that prior successor support; the entire prior live ledger $\mathcal{L}_{\text{old}}$ embeds into the new initial ledger via i equipped with a left inverse, commuting with claim translation and satisfying non-increasing budgets; and the new initial ledger adjoins a direction η with no prior preimage in obligation, incidence, or claim, budget $b(\eta) = 0$, and proof that its claim holds at the new support. Crucially, every $i(e)$ and η carries its actual generated-row witness inside that **same first whole-ledger write**, locking emitted events and terminal handoff laws together.

Proposition 12.2 (Zero-Axiom Minimal Coface Extension Retains All Prior Debts with Unique Factorization, WA9). In the zero-axiom kernel (`#print axioms = []`), the obstruction-generated minimal carrier $\mathcal{L}_{\text{old}} \sqcup \{\eta\}$ (with inherited prior budgets and $b(\eta) = 0$) maps via a unique faithful injective factorization into every admissible revision of Definition 12.1, sending $\mathcal{L}_{\text{old}}$ via i and the obstruction branch to η.

Proof. Preservation of prior rows fixes the map to i on the left summand, and preservation of the attached obstruction coordinate fixes it to η on the right summand, establishing existence and pointwise uniqueness. The left inverse of i prevents distinct prior debts from collapsing together, while absence of a prior preimage for η prevents conflating the new direction with any old debt.[4] □

[4] Zero-axiom categorical initiality and attached-direction proofs appear in [6], Theorems 8.4 and 8.6.

12.2 Richer Split Revisions and Shared First-Write Disposition

Source-fixed revisions admit two lawful shapes: the **exact minimal shape** (matching the canonical minimal coface) and a **richer split shape** permitting an enlarged representation provided the revised root's first whole-ledger compiler **actually selects a genuine split** of the migrated causal obligation into at least two distinct children with full-fibre coverage and strict child budget descent (empty whenever the target has ≤ 1 row or empty split cover), with every initial row lying in $i(\mathcal{L}_{\text{old}}) \cup \{\eta\}$.

Theorem 12.3 (Shared First-Write Authority and Common Successor for Migrated Debts and New Directions, WA9). For every structurally anchored typed revision (minimal or richer split) actually emitted by the fixed inquiry programme, initial causal-row authorities for every migrated prior entry $i(e)$ and for the new obstruction entry η advance across the revised root's **same nonterminal first write** to target row authorities, receive canonical rowwise answers from the revised root, and share the exact same canonical next current.

Proof. Inside the zero-axiom lifecycle and recovery kernels (`U8LifecycleKernel`, `U8RecoveryKernel`), each $i(e)$ and η extracts initial causal-row authority from its generated-row witness in the first write. Applying the single successor equality of that first write advances all authorities to the same target visit, where Theorem 3.5 delivers each row's answer while reading `nextCurrent` from that single revised-root occurrence—making all next currents identical by transitivity.[5] □

Example 12.4 (Concrete Two-Step Debt-Preserving Revision, WA9). Starting from a prior obligation of budget 2 at a doubly empty failure point, Step 1 emits the minimal coface revision carrying the migrated row (budget 2) and new obstruction row (budget 0) onto a natural-number root ($n \mapsto n+1$): first write $0 \to 1$ executes $2 \to 2$ and $0 \to 0$ transfers together, and at state 1 both rows receive canonical answers advancing to common state 2; Step 2 answers at 2 and steps to 3, upgrading the language without losing a single unit of the budget-2 historical debt.[6] This marks a sharp categorical boundary against classical AGM theory change [19], which targets **defeasible belief revision inside a fixed propositional language** via an external selection function γ performing partial-meet contraction ($K \dot{-} p \subseteq K$, dropping prior beliefs to restore consistency): treating expressive bankruptcy on an obligation-bearing ledger as fixed-language belief contraction would write off unsettled debts, whereas Proposition 12.2 and Theorem 12.3 expand language coordinates in zero choice axioms via an initial coface with zero debt contraction.

[5] Shared first-write two-row recovery corresponds to [6], Theorem 8.7.
[6] The concrete two-step debt-carrying revision corresponds to [6], Proposition 8.9.

13 Total Blindness of Finite Complexified Observations and Unrecoverability of Global Integral Classes

Section 4 established the abstract criterion for consumer-relative observer safety; this section constructs a striking algebraic counterexample on global arithmetic modules (**WA11**): **a single global integer boundary class generated endogenously from a unit arithmetic transfer receipt remains strictly nonzero at stage one and every subsequent integer stage, yet vanishes identically under every finite-stage complexified (determinant) observation—even pooling infinitely many finite-stage complex observations together admits no linear or nonlinear function recovering the integer readout!**

13.1 Prime-Three Coordinate from Unit Transfer Receipts and Block-Determinant Annihilation

At stage 0 the unit arithmetic root has cardinal shadow 2 and lacks prime 3; the stage-1 native unit transition advances to the next state whose transfer receipt generates a factor inventory containing the structural key for prime 3 (absent in the zero-action branch). Everything below is extracted from that actual transfer receipt without external nonzero assumptions.

Construction Box: Two-Tier Factor Coordinates, Prime Action, and Endpoint Differential Boundary. Fix the unit arithmetic source. Stage-n unit history h_n has factorial fold cardinality $(n+2)!$ with prime-factor set $\mathcal{P}_n$, multiplicities $m_n(p) = v_p((n+2)!)$, and prime-power factor rows

$$\mathcal{R}_n := \{(p,k) : p \in \mathcal{P}_n,\ 1 \le k \le m_n(p)\}.$$

Inner coordinates (q, j, ϵ) ($q \in \mathcal{P}_n$, $0 \le j \le m_n(q)$, $\epsilon \in \{0,1\}$) define the integral module K_n of arrays satisfying $v(q, j+1, \epsilon) = q\, v(q, j, \epsilon)$ ($0 \le j < m_n(q)$); reading $j = 0$ gives $K_n \cong \mathbb{Z}^{\mathcal{P}_n \times \{0,1\}}$ with inverse $a(q,\epsilon) \mapsto q^j a(q,\epsilon)$. Dual flip τ swaps $\epsilon \in \{0,1\}$, and prime action multiplies inner coordinate q by q.

Outer coordinates index factor rows $r = (p,k) \in \mathcal{R}_n$ (outer p selects the factor relation row, inner q selects its prime coordinate independently), forming the complete relation module and operator:

$$M_n := K_n^{\mathcal{R}_n}, \qquad (E_n v)(r; q, j, \epsilon) := q\, v(r; q, j, \epsilon).$$

Endpoints and differential boundary act on vertex module $K_n^{\{*\} \sqcup \mathcal{R}_n}$ via $(d_n w)(p,k) = (I - \tau) w(*) - p^k (I - \tau) w(p,k)$. Setting the left endpoint to $\iota_n(1,0)(q,j,\epsilon) = q^j \delta_{\epsilon,0}$ at $*$ and zero elsewhere yields boundary $\beta_n \in M_n$:

$$\beta_n(r; q, j, 0) = q^j, \qquad \beta_n(r; q, j, 1) = -q^j.$$

Stage restrictions forget newly added primes, higher powers, and rows while preserving all recurrences, actions, and boundaries.

Define the stage-n action cokernel and endpoint boundary class:

$$C_n := M_n/\operatorname{im}(I - E_n), \qquad z_n := [\beta_n] \in C_n. \tag{48}$$

At any factor row $r = (p, k) \in \mathcal{R}_n$, the same-prime, zero-exponent, left-dual integer linear readout $\ell_r(v) := v(r; p, 0, 0)$ evaluates directly to:

$$\ell_r(\beta_n) = 1, \qquad \ell_r((I - E_n)v) = (1 - p)\ell_r(v). \tag{49}$$

Equation (49) takes place purely over $\mathbb{Z}$ without coordinate projection loss or complexification.

Theorem 13.1 (Transfer Receipt Generates a Nonzero Integral Boundary Class Across All Subsequent Stages, WA11). The prime $p = 3$ factor row generated by the stage-1 unit transfer receipt forces $z_1 \neq 0$, and the arithmetic readout of the symbolic block action maps its endpoint class onto $z_1 \neq 0$; along endpoint-preserving stage restrictions $\rho_n : C_{n+1} \to C_n$ ($\rho_n(z_{n+1}) = z_n$), $z_{k+1} \neq 0$ holds for every $k \in \mathbb{N}$ while retaining emitted events, whole-ledger writebacks, and canonical successors (with $0 \to 0$ budget-preserving transfers never mistaken for strict payments).

Proof. At $n = 1$ take the receipt-generated prime-3 row $r = (3, 1) \in \mathcal{R}_1$. Were $z_1 = 0$, some $v \in M_1$ would satisfy $\beta_1 = (I - E_1)v$; applying ℓ_r via Equation (49) yields $1 = (1 - 3)\ell_r(v) = -2\ell_r(v)$, contradicting integer parity! Over $\mathcal{A} = \mathbb{Z}[\alpha_p, \gamma_p \mid p \in \mathbb{P}]$, specializing the symbolic module $\mathcal{A}^{\mathcal{R}_n \times \mathcal{P}_n \times \{0,1\}}$ (inner block $\left(\begin{smallmatrix} \alpha_q & \gamma_q \\ \gamma_q & \alpha_q \end{smallmatrix}\right)$) via $\alpha_p \mapsto p, \gamma_p \mapsto 0$ and $A_n(v)(r; q, j, \epsilon) = q^j \operatorname{ev}(v(r; q, \epsilon))$ satisfies $A_n(I - E_n^{\mathrm{blk}}) = (I - E_n)A_n$ and sends symbolic $(1, -1)$ onto β_n, transferring nonvanishing to the block port. Inductively, $z_{k+1} = 0 \implies z_k = \rho_k(0) = 0 \implies z_1 = 0$, a contradiction.[7] $\square$

Lemma 13.2 (Division-Free Cofactor Construction and Finite-Stage Determinant Annihilation, WA11). At every finite stage $n \in \mathbb{N}$, the specialized prime–dual block determinant satisfies:

$$D_n = \prod_{p \in \mathcal{P}_n} (1 - p)^2 \neq 0, \qquad D_n z_n = 0 \in C_n, \tag{50}$$

whence $1 \otimes z_n = 0$ identically in $\mathbb{C} \otimes_{\mathbb{Z}} C_n$.

Proof. Over $\mathcal{A}$, each inner 2×2 block $T_p = \left(\begin{smallmatrix} 1-\alpha_p & -\gamma_p \\ -\gamma_p & 1-\alpha_p \end{smallmatrix}\right)$ has determinant $\delta_p = (1-\alpha_p)^2 - \gamma_p^2 = (1 - \alpha_p + \gamma_p)(1 - \alpha_p - \gamma_p)$, with product $\Delta_n = \prod_{p \in \mathcal{P}_n} \delta_p$. Because $(1, -1)$ is an eigenvector of T_p with eigenvalue $1 - \alpha_p + \gamma_p$, the division-free cofactor

$$a_{n,p} := (1 - \alpha_p - \gamma_p) \prod_{q \in \mathcal{P}_n \setminus \{p\}} \delta_q \in \mathcal{A}$$

satisfies $(1 - \alpha_p + \gamma_p)a_{n,p} = \Delta_n$. Setting w_n^{blk} at each outer row and inner prime p to $a_{n,p}(1, -1)$ gives $(I - E_n^{\mathrm{blk}})w_n^{\mathrm{blk}} = \Delta_n \beta_n^{\mathrm{blk}}$; applying A_n ($\alpha_p \mapsto p, \gamma_p \mapsto 0$) yields $(I - E_n)w_n = D_n \beta_n$ in M_n, so $D_n z_n = 0 \in C_n$ with $D_n \neq 0$ since every prime $p \geq 2 > 1$. In $\mathbb{C} \otimes_{\mathbb{Z}} C_n$, $\mathbb{Z}$-bilinearity and invertibility of $D_n \in \mathbb{C}^\times$ give

$$1 \otimes z_n = D_n^{-1}(D_n \otimes z_n) = D_n^{-1}(1 \otimes D_n z_n) = D_n^{-1}(1 \otimes 0) = 0. \quad \square$$

[7] Nonvanishing from the prime-3 factor row holds from stage 1 onward; stage 0 has cardinal shadow 2 and makes no analogous nonzero claim.

13.2 Universal Complexified Annihilation and Unrecoverability by Any Function

Theorem 13.3 (No Linear or Nonlinear Function Recovers the Integral Consumer from All Finite Complexified Observations, WA11). On the global compatible integer cokernel

$$G := \{(g_n)_{n\in\mathbb{N}} \in \prod_{n\in\mathbb{N}} C_n : \forall n \in \mathbb{N},\ \rho_n(g_{n+1}) = g_n\}$$

with stage projections $\pi_n : G \to C_n$, the endpoint sequence defines $z = (z_n)_{n\in\mathbb{N}} \in G$. Define the **complete family of finite-stage complexified observations** F and **stage-one integer readout** J:

$$F(g) := (1 \otimes \pi_n(g))_{n\in\mathbb{N}} \in \prod_{n\in\mathbb{N}} (\mathbb{C} \otimes_{\mathbb{Z}} C_n), \qquad J(g) := \pi_1(g) \in C_1. \tag{51}$$

Then $z \neq 0$ and $F(z) = 0 = F(0)$ whereas $J(z) = z_1 \neq 0 = J(0)$ ($\ker F \not\subseteq \ker J$). Consequently, **there exists no (linear or nonlinear) function** $h : \prod_{n\in\mathbb{N}}(\mathbb{C} \otimes_{\mathbb{Z}} C_n) \to C_1$ satisfying $J(g) = h(F(g))$ for all $g \in G$, nor any factorization through the observable quotient $G/\ker F$.

Proof. Theorem 13.1 gives $J(z) = z_1 \neq 0$ ($z \neq 0$), while Lemma 13.2 gives $1 \otimes z_n = 0$ stage by stage without commuting inverse limits past tensor products, so $F(z) = (0, 0, \dots) = F(0)$. Thus $(0, z)$ is a Definition 4.1 hidden-null witness for (F, J): any h with $J = h \circ F$ would force $J(z) = h(F(z)) = h(F(0)) = J(0) = 0$, contradicting $z_1 \neq 0$; in $G/\ker F$, $[z] = [0]$ yields the identical contradiction (Figure 5). □

Theorem 13.3 exposes a striking algebraic reversal of classical dynamical observability: in Kalman's linear observability theory over characteristic-zero fields ($\mathbb{R}/\mathbb{C}$) [17], nonvanishing of a block determinant ($\det(I - E_n) = D_n \neq 0$) is precisely what *secures* invertible state reconstruction; on the integral cokernel C_n, however, where division-free cofactors force $D_n z_n = 0$, **the very invertibility of $D_n \neq 0$ in $\mathbb{C}$ becomes the weapon that destroys the signal—scaling by D_n^{-1} annihilates $1 \otimes_{\mathbb{Z}} z_n = D_n^{-1}(1 \otimes D_n z_n) = 0$ across all infinitely many finite stages!** Auditing architectures restricted to complexified scalar, spectral, or floating-point summaries are structurally blind to integral torsion liabilities.

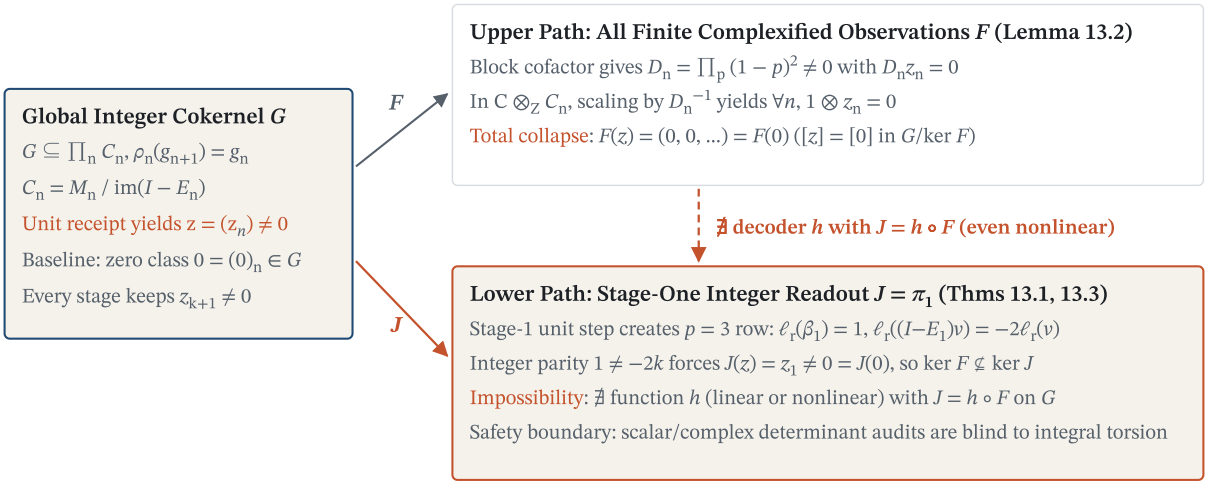


Figure 5 | Total Blindness of Finite Complexified Observations to a Global Integral Class. From the global integer cokernel G on the left, nonzero class $z = (z_n)$ and 0 split along two channels: along the upper path F, block cofactor annihilation $D_n z_n = 0$ ($D_n \neq 0$) scales by D_n^{-1} in each $\mathbb{C} \otimes_{\mathbb{Z}} C_n$ to collapse $F(z) = (0, 0, \ldots) = F(0)$; along the lower path $J = \pi_1$, modulo-2 parity at $p = 3$ ($1 \neq -2\ell_r(v)$) locks $J(z) = z_1 \neq 0 = J(0)$. The dashed vermilion arrow marks that no linear or nonlinear decoder h can map one identical input to two distinct outputs.

14 Full-Trace Relay Across Long Analytic Windows and Inverse-Residual Corrections

In multi-stage analytic operator computations, practitioners routinely keep a single scalar window estimate while discarding the cutoff, mean, and cochain histories that produced it. On an explicitly bounded conditional domain (**WA33, WA40**), we prove that **resolvent-comb installation (WA33) and all four execution traces of a finite cutoff window install into stock prior to the decoder writer, migrate intact into the next inverse-residual correction packet, and eliminate jointly alongside six newly generated traces.**

14.1 Four-Component Hilbert Carrier, Resolvent-Comb Installation, and Conditional Domain Boundary

Let $\mathcal{H}$ be a complex Hilbert carrier with defect operator D and Fourier operator $\mathfrak{F}$. Define the four-component carrier (source, defect, Fourier source, Fourier defect) and its embedding–projection pair:

$$\mathcal{B} = (\mathcal{H} \times \mathcal{H}) \times (\mathcal{H} \times \mathcal{H}), \qquad \iota(f) = ((f, Df), (\mathfrak{F}f, \mathfrak{F}Df)), \qquad \pi((u, v), (w, t)) = u, \tag{52}$$

satisfying $\pi\iota = \mathrm{id}_{\mathcal{H}}$. For real window parameter a, zero extension $Z_a : \mathcal{H}_a \to \mathcal{H}$ and restriction $R_a : \mathcal{H} \to \mathcal{H}_a$ satisfy $R_a Z_a = \mathrm{id}_{\mathcal{H}_a}$; even projection $P : \mathcal{H} \to \mathcal{H}_{\mathrm{Pa}}$, Möbius source operator $M : \mathcal{H}_{\mathrm{Pa}} \to \mathcal{H}$, orthogonal projection onto compact co-Poisson closure $\mathcal{K}_{\mathrm{co}} \subseteq \mathcal{H}_{\mathrm{Pa}}$ composing to qualification $\mathcal{E} = \Pi_{\mathcal{K}_{\mathrm{co}}} \circ P$, Tate inversion $J : \mathcal{H} \to \mathcal{H}$, and window functional $\ell : \mathcal{H} \to \mathbb{C}$ are continuous linear maps. With finite centered Möbius cutoff list $L_a = \mathrm{toList}(\mathcal{C}_a)$, scale pullbacks

$A_{a,m} : \mathcal{H} \to \mathcal{H}_a$, and constant interval field $c_a \in \mathcal{H}_a$, define the finite window operator and its lift:

$$W_a(f) = \sum_{m \in L_a} \frac{1}{m} A_{a,m} f - \frac{1}{2} \ell(Jf) c_a, \qquad \widehat{W}_a = \iota Z_a W_a \pi. \tag{53}$$

Conditional Input Domain and Vacuous-Truth Boundary. Universal inputs to this section are: a source-generated analytic zero-observation receipt at $z \in \mathbb{C}$ ($\zeta_{\mathrm{src}}(z) = 0$, identified analytically with Riemann ζ), nontriviality $\forall n \in \mathbb{N},\ z \neq -2(n+1)$, strict right-half-plane hypothesis $\operatorname{Re} z > 1/2$, discrete indices $\mathrm{depth}, \mathrm{ordinal} \in \mathbb{N}$, and real parameters h, H, N, a. In the upstream resolvent-comb installation (**WA33**), derived datum (q_0, R, m) builds iterative syntax $e_0 = x, e_{k+1} = R(e_k)$ installed prior to the emitter on the closed runtime facade, outputting $R^m q_0$ and an $(m+1)$-step trace (Supplement E.1). **Crucially**, Theorems 14.1–14.2 are universally quantified over this conditional input type and **construct no inhabitant thereof**: if the Riemann Hypothesis holds, every nontrivial zero has $\operatorname{Re} z = 1/2$, rendering the input type empty so Theorems 14.1–14.2 hold vacuously without asserting an inhabited run.

From the generated state $S \in \mathcal{B}$ (with $q = \mathcal{E}\pi(S)$), the qualified Möbius source and position/Fourier mean identities are:

$$\nu = M\mathcal{E}\pi(S), \tag{54}$$

$$\mathrm{mean}_{\mathrm{pos}}(q) = -\frac{1}{2}\ell(J\nu), \qquad \mathrm{mean}_{\mathrm{Fourier}}(\mathfrak{F}_{\mathrm{even}} q) = -\frac{1}{2}\ell(\nu), \tag{55}$$

with $s := \iota(\nu)$ read from the difference between the born environment and paid total of the prior write.

Theorem 14.1 (Pre-Decoder Installation of Four Window Execution Traces and Exact Write Difference, WA40). For every conditional input tuple above, the source generates finite-window packet p_F together with its completion receipt, execution stock, and whole-ledger write: forward-stock, first inverse-residual, second-mean, and update-cochain traces all install prior to the decoder write; the folded difference Δ_F between born environment and paid total equals $\widehat{W}_a(s)$ (with $R_a\pi(\Delta_F) = W_a(\nu)$), and the residual read from Δ_F,

$$\eta = \nu - MP\pi(\Delta_F), \qquad r = \iota(\eta), \tag{56}$$

equals the complete residual computed by the first inverse programme while delivering strict same-debt debit and exact whole-ledger destinations.

Proof (Steps 1–2).

Step 1: Termwise evaluation and node cost of the cutoff window AST. Over constants, variables, additions, and linear maps, remaining node counts satisfy:

$$\begin{aligned} r(\mathrm{var}\, x) &= 1, & r(\mathrm{const}\, v) &= 0, \\ r(e+f) &= r(e) + r(f) + 1, & r(Ae) &= r(e) + 1, \end{aligned} \tag{57}$$

producing trace $T : e \to \mathrm{const}(\mathrm{ev}_\rho\, e)$ of length $|T| = r(e)$ telescoping to $\delta_e - \delta_{\mathrm{const}(\mathrm{ev}_\rho\, e)}$ (process-algebra companion [6], Proposition 3.7, Supplements S0.1–S0.3). Setting $U_{a,m} = \iota Z_a A_{a,m} \pi$,

$B_m = \frac{1}{m}\,\mathrm{id}_{\mathcal{B}}$, $Q_a = \iota Z_a(c_a\,[-\frac{1}{2}\ell J])\pi$, and $e_{a,[]} = \mathrm{const}\,0$, define recursively:

$$e_{a,m::L} = B_m(U_{a,m}(\mathrm{var}\,x)) + e_{a,L}, \qquad e_a = e_{a,L_a} + Q_a(\mathrm{var}\,x). \tag{58}$$

List induction gives for every environment ρ:

$$\mathrm{ev}_\rho(e_{a,L}) = \sum_{m\in L} B_m U_{a,m}(\rho(x)), \qquad \mathrm{ev}_\rho(e_a) = \widehat{W}_a(\rho(x)). \tag{59}$$

At $\rho_s(x) = s = \iota(\nu)$, soundness of $T : e_a \to \mathrm{const}\,u$ gives $u = \widehat{W}_a(s)$, so $\pi\iota = \mathrm{id}$ and $R_a Z_a = \mathrm{id}$ yield:

$$R_a\pi(u) = W_a\pi(s) = W_a(\nu). \tag{60}$$

For $|L_a| = k$, each cutoff term costs 3 nodes plus 1 addition node, the pole term costs 2, and top addition costs 1, totaling $r(e_a) = 4k + 3$.

Step 2: Four exposure trees installed prior to decoder writeback. With $I = \iota MP\pi$, $e_a^{\mathrm{inv}} = \mathrm{var}\,x + (-I)e_a$ evaluates in its first component at ρ_s to:

$$\pi\,\mathrm{ev}_{\rho_s}(e_a^{\mathrm{inv}}) = \nu - MPZ_aW_a(\nu). \tag{61}$$

Setting the stock-write AST

$$f_{a,s} = \mathrm{var}\,x + e_a[x := \mathrm{const}\,s] \tag{62}$$

gives for all ρ:

$$\mathrm{ev}_\rho(f_{a,s}) = \rho(x) + \widehat{W}_a(s). \tag{63}$$

Writing $e \mapsto e^{[g]}$ for graded-carrier lifting, pairwise summation fold c_g satisfies by induction on g:

$$c_g\,\mathrm{ev}_\rho(e^{[g]}) = \mathrm{ev}_{c_g\rho}(e). \tag{64}$$

At frame ϕ ($v_\phi = \mathrm{oldPaid} + \mathrm{nativeAt}(\phi, o_\phi), b_\phi = c_g(v_\phi)$), four trace exposures are generated:

$$\begin{aligned}
T_F &: \; f_{a,s}^{[g]} \longrightarrow \mathrm{const}\,\mathrm{ev}(f_{a,s}^{[g]}),\\
T_I &: \; (e_a^{\mathrm{inv}}[x := \mathrm{const}\,s])^{[g]} \longrightarrow \mathrm{const}\,\mathrm{ev}(\cdot),\\
T_M &: \; \left[\iota Z_a c_a(-\tfrac{1}{2}\ell\pi)(\mathrm{const}\,s)\right]^{[g]} \longrightarrow \mathrm{const}\,\mathrm{ev}(\cdot),\\
T_B &: \; B(f_{a,s}; b_\phi, \widehat{W}_a(s))^{[g]} \longrightarrow \mathrm{const}\,0,
\end{aligned} \tag{65}$$

where update cochain B expands

$$\delta_{f(\mathrm{old}+\mathrm{increment})} - \delta_{f(\mathrm{old})} - \sum_{t\in\mathrm{mixedTerms}(f)} \delta_t \tag{66}$$

at $(b_\phi, \widehat{W}_a(s))$ to evaluate to 0 while recording every intermediate step. Juxtaposing via $\odot$ (retaining `Prior`),

$$I_F(\phi) = \mathsf{Exp}(T_F(\phi)) \odot (\mathsf{Exp}(T_I(\phi)) \odot (\mathsf{Exp}(T_M(\phi)) \odot \mathsf{Exp}(T_B(\phi)))), \tag{67}$$

installing $f_{a,s}^{[g]}$ in the decoder writer and I_F in the stock callback prior to the emitter, and reading completion receipt (ϕ_F, v) with $u_F = \mathrm{oldPaid} + v$ yields via Equations (63)–(64):

$$\Delta_F := c_g(\mathrm{activeEnv}_F(x)) - c_g(u_F) = \widehat{W}_a(s), \tag{68}$$

recovering $W_a(\nu)$ and η in Equation (56). □

14.2 Migration of Four Prior Traces into Inverse-Residual Correction and Joint Observation Fibres

Theorem 14.2 (Four Prior Traces Plus Six Correction Traces Enter Actual Correction, Payment, and All Futures, WA40). From (p_F, s, r) of Theorem 14.1, executing literal $s - r$ substituted into the window AST spawns correction packet p_C:

1. The second actual write difference satisfies:

$$\Delta_C = \iota Z_a W_a(\nu - \eta). \tag{69}$$

   All four prior traces $I_F(\phi_F)$ migrate order-preservingly into Prior_C, while ϕ_C generates four fresh window traces plus stock and correction traces (six new traces with distinct occurrence indices).
2. Nested elimination of prior write, new write, strict same-debt payment, environment/AST identity, and traces yields window difference and source-price identities for every $b \in \mathcal{H}_{\mathrm{Pa}}$:

$$R_a \pi(\Delta_C) - W_a(\nu) = -W_a(\eta), \tag{70}$$

$$\langle b, P\pi(\Delta_C)\rangle = \langle b, PZ_a W_a(\nu)\rangle - \langle b, PZ_a W_a(\eta)\rangle. \tag{71}$$

3. Correction preserves the joint current/next observation quotient, identifies birth roots, currents, and true source seeds, and maintains residual–boundary equality across all allowed future words and sites.

Proof (Steps 3–6).

Step 3: Residual substitution and six-trace stock merge. Define literal correction syntax (cost 2, value $s - r$) and composite programmes:

$$\kappa_{s,r} = \mathrm{const}\, s + (-\mathrm{id})(\mathrm{const}\, r), \tag{72}$$

$$g_{a,s,r} = e_a[x := \kappa_{s,r}], \qquad j_{a,s,r} = \mathrm{var}\, x + g_{a,s,r}. \tag{73}$$

Substitution evaluation and Equation (59) give for all ρ:

$$\mathrm{ev}_\rho(g_{a,s,r}) = \iota Z_a W_a \pi(s - r), \qquad \mathrm{ev}_\rho(j_{a,s,r}) = \rho(x) + \iota Z_a W_a \pi(s - r). \tag{74}$$

Setting

$$I_C(\phi_C) = I_F(\phi_C) \odot (\mathsf{Exp}(T_C^{\mathrm{stock}}) \odot \mathsf{Exp}(T_C^{\mathrm{correction}})), \tag{75}$$

$\mathsf{preserve}(\mathrm{Prior}_C, I_C(\phi_C))$ retains all four migrated traces alongside the six new traces (costs $5k+4$ and $5k+6$ by occurrence multiplicity). Repeating Step 2 with $u_C = \mathrm{oldPaid} + v_C$ gives:

$$\Delta_C := c_g(\mathrm{activeEnv}_C(x)) - c_g(u_C) = \mathrm{ev}_{\rho_s}(g_{a,s,r}) = \iota Z_a W_a \pi(s-r) = \iota Z_a W_a(\nu - \eta), \tag{76}$$

and applying $R_a\pi$ proves $R_a\pi(\Delta_C) - W_a(\nu) = -W_a(\eta)$, establishing Equations (69)–(70).

Step 4: Nested same-source elimination and strict payment. Both packets satisfy $Q - 1 > 0$, generating strict same-debt payments via Theorem 9.2. Eliminating in dependent order:

$$\mathrm{consume}_F(\lambda \mathrm{whole}_F, \mathrm{write}_F, \mathrm{Trace}_F.\ \mathrm{consume}_C(\lambda \mathrm{whole}_C, \mathrm{write}_C, \mathrm{Trace}_C.\ \cdots)), \tag{77}$$

write_F locks η, $(\mathrm{Trace}_C, \mathrm{write}_C, \mathrm{whole}_C)$ lock Equations (70) and (76) plus payment, and applying $P\pi$ and $\langle b, - \rangle$ yields Equation (71).

Step 5: Joint current/next observation quotient. Define on $\mathcal{B}$:

$$\mathcal{R}_a = \mathrm{id} - I\widehat{W}_a, \qquad \mathcal{A}_a = \mathrm{id} - \mathcal{R}_a, \qquad O_a = W_a\pi. \tag{78}$$

Since $r = \mathcal{R}_a(s)$, correction evaluates to:

$$\mathrm{ev}(\kappa_{s,r}) = s - r = \mathcal{A}_a(s). \tag{79}$$

On the joint quotient

$$J_a(x) = (O_a x, O_a \mathcal{A}_a x), \qquad \mathcal{Q}_a = \mathcal{B}/\ker J_a, \tag{80}$$

current and next readouts are well-defined and read the actual correction:

$$\mathsf{current}([s]_{J_a}) = [s]_{O_a}, \qquad \mathsf{next}([s]_{J_a}) = [\mathcal{A}_a s]_{O_a} = [\mathrm{ev}(\kappa_{s,r})]_{O_a}, \tag{81}$$

with $[x]_{J_a} = [y]_{J_a}$ iff

$$O_a x = O_a y \ \wedge\ O_a \mathcal{A}_a x = O_a \mathcal{A}_a y. \tag{82}$$

Step 6: Birth root, true source seed, and all-future readings. At any receipt-selected frame ϕ_* (occurrence o_*), birth compilation gives:

$$\begin{aligned} \mathrm{birth}(o_*).\mathrm{targetRoot} &= \mathrm{root}(\mathsf{nextBorn}(\phi_*, C_*)), \\ \mathrm{birth}(o_*).\mathrm{answerAndNext.nextCurrent} &= \mathrm{current}(\mathsf{nextBorn}(\phi_*, C_*)), \end{aligned} \tag{83}$$

identifying roots and currents of p_F, p_C while binding all five continuation callbacks to the packet's own seed ξ. Finally, in the maximal future kernel of Theorem 8.3,

$$K_{\mathrm{fut}} = \bigcap_{w \in \{*\}^{<\omega}} \bigcap_{t} \ker(D_t S_w), \tag{84}$$

Theorem 9.1's semantic-zero invariance yields for all $j \in \mathbb{N}$, $w \in \{*\}^{<\omega}$, and $t \in \mathbb{N}$:

$$D_t S_w(\iota_{j+1}[q_{j+1}.\mathrm{AST}]) = D_t S_w(\iota_{j+1} b_{j+1}), \tag{85}$$

completing Theorem 14.2. □

15 Cross-Packet Paid Methods, Zero-Evaluation Relation-Certificate Inversion, and Step-Grouped Continuation

Sections 9 and 14 showed how residuals spawn paid requests and how traces cross analytic windows. This second edition (v2) closes three final structural capabilities that elevate the calculus into a **reversible multi-epoch method and batch ledger (WA41–WA45)**:

1. **How do paid programmes keep acting across epochs?** Rather than discarding a paid AST after one settlement, double-negation extraction recovers it intact to execute any prefixed additive homomorphism across all cofinal stops while primitive-by-primitive replay drives cochain differentials in lockstep (Section 15.1, WA41–WA42).
2. **When positive and negative effects cancel numerically to zero, can the ledger still invert the entry without loss?** Where scalar accounting sees only $+v + (-v) = 0$, free constant-module transport retains explicit relation corrections and builds a finite integer certificate (preserving even the empty-list zero atom) to reconstruct the exact next constant table without kernel-oracle assumptions (Section 15.2, WA43).
3. **When batching next-round requests or extending a sealed history horizon, what prevents silent drop, duplication, or recomputation?** Contiguous per-step slicing establishes two-way ordinal address recognition and rejects omitting or duplicating any nonempty group, while horizon extension locks the sealed paid prefix and executes solely from the reached tail state (Section 15.3, WA44–WA45).

15.1 Double-Negation Paid-AST Recovery, Cross-Cofinal Homomorphisms, and Primitive Cochain Replay

In Theorem 9.1's birth construction, once a grade-n packet P settles at $p = \mathrm{const}\, v_{\mathrm{paid}}$ ($v_{\mathrm{paid}} \in V_n \times V_n = V_{n+1}$), the native receiver registers the first-layer residual $\mathrm{add}(e_0, \mathrm{linear}(-\mathrm{id}, p))$; upon native settlement into `nextBorn` (P^+, at grade $n+1$), a second outer residual wraps around it to form the left-associated double addition $\mathrm{add}(\mathrm{add}(e_0, u), \mathrm{linear}(-\mathrm{id}, v_{\mathrm{eff}}))$ where $u = \mathrm{linear}(-\mathrm{id}, p)$. Define the three-branch loop-free syntactic extractor `retainedCode`: on nested $\mathrm{add}(\mathrm{add}(_, u), _)$ it strips the outer residual and returns $\mathrm{linear}(-\mathrm{id}, u)$, on single $\mathrm{add}(_, u)$ it returns $\mathrm{linear}(-\mathrm{id}, u)$, and otherwise it returns its input unchanged. Because the inner term u is already $\mathrm{linear}(-\mathrm{id}, p)$, the first branch extracts a **double-negation AST**!

Theorem 15.1 (Double-Negation Paid-AST Recovery, Callback-Grounded Homomorphism Continuation, and Primitive Cochain Replay, WA41–WA42).

1. **Double-negation recovery and cross-cofinal homomorphism action (WA41)**: Fix any source configuration family $\mathcal{F}$, lawful authoritative-root inputs, and any pre-selected additive homomorphism $f : W_s \to_+ W_s$ (lifted recursively across pair grades via $f^{(0)} = f$ and $f^{(k+1)} := (f^{(k)}, f^{(k)})$). For any admission packet P at grade n settling at $p = \mathrm{const}\, v_{\mathrm{paid}} \in V_{n+1}$,

advancing through native execution and birth into P^+ (grade $n+1$) yields the exact heterogeneous AST and evaluation identities:

$$\mathsf{retainedCode}(P^+) \;\simeq\; (-\mathrm{id})((-\mathrm{id})(p)), \qquad [\![\mathsf{retainedCode}(P^+)]\!]_{\Gamma^+} = v_{\mathrm{paid}}. \tag{86}$$

Evaluating $\mathrm{linear}((f^{(n+1)}, f^{(n+1)}), \mathrm{lift}(\mathsf{retainedCode}))$ in zero-increment pair environment $\mathrm{pairEnv}(\Gamma^+, 0)$ outputs $(f^{(n+1)}(v_{\mathrm{paid}}), 0)$, while every event of `completeQueryWritten` enters `birthStock` through the packet's own `nextPairInventory` preservation map. On contextual families equipped with complex inner-product recognition (`RecognitionAt H root`), this paid-method packet installs unified algebraic, combinatorial, topological, and logical four-faces, dual readback, and evaluation cochains prior to the emitter at **every** cofinal admission ordinal $k \in \mathbb{N}$ (and across every finite update segment).

2. **Primitive-by-primitive word replay driving chain morphisms and cochain differentials (WA42)**: On any living source process, recursive step embedding (`WordExecution.embeddedTrace` and `pairedForward`) embeds **every single primitive step** of the observer-environment schedule into the context environment (never equating traces merely because their endpoints agree), with trace length equal to substituted remaining node count (`forward_charge`). On the canonical particle–wave bosonic–fermionic (Fock) sealed runtime (aligned with the underlying process at initial 11-step calibration offset $11 + \mathrm{count}$), its exposure `replayStock` enters both `birthStock` and the next complete query, and its integral relation word μ_{src} satisfies the exact commuting differential equations across substitution chain morphism τ_1, stock boundary ∂_{act}, and natural cochain differential $d^{0,1}$ at the same stop:

$$\mathrm{relationMap}(\tau_1(\mu_{\mathrm{src}})) = \partial_{\mathrm{act}} = (d^{0,1}(\partial_{\mathrm{act}})).\mathrm{val}, \tag{87}$$

simultaneously driving completion annihilation, coimage boundary/differential, same-stop `physicalReadback` (source state plus forced trace), and whole-ledger canonical next.

Proof. (1) Prefix induction along Theorem 8.3 (`paid_prefix_expression`) keeps the registered AST equal to the first-layer residual $\mathrm{add}(e_0, \mathrm{linear}(-\mathrm{id}, p))$ from the native receiver's start to its receipt-selected settled frame; `nextBorn` wraps the second outer residual $\mathrm{add}(\mathrm{add}(e_0, \mathrm{linear}(-\mathrm{id}, p)), \mathrm{linear}(-\mathrm{id}, v_{\mathrm{eff}}))$, triggering the first branch of `retainedCode` to output $(-\mathrm{id})((-\mathrm{id})(p))$, which evaluates via $-(-x) = x$ to v_{paid} (Equation (86)). Since zero-increment effects vanish identically (`effect_zero`), grade-$(n+1)$ pair lifting at $\mathrm{pairEnv}(\Gamma^+, 0)$ evaluates to $(v_{\mathrm{paid}}, 0)$ and $(f^{(n+1)}, f^{(n+1)})$ maps $0 \mapsto 0$ to yield $(f^{(n+1)}(v_{\mathrm{paid}}), 0)$. Defining `nextPairInventory` via `Stock.preserve prior completeQueryWritten` embeds every query event into `birthStock` (`birth_keeps_callback`), while on contextual recognition families sevenfold `InstallationAt` coface composition installs `InstalledFaces` before emission (`rfl`/`HEq.rfl`). (2) `embeddedStep` recurses over all nine step constructors, concatenating to `embeddedTrace` and transporting along `context_environment` and `paired_binding_square` into `nativeReplayAt`, whose exposure equals `replayStock` (`native_stock_eq`) inside the next prefix history's relation closure. `SourceSubstitution.boundary_square` gives $\mathrm{relationMap}(\tau_1(\mu_{\mathrm{src}})) = \mathrm{substitution}(\mathrm{relationMap}(\mu_{\mathrm{src}})) = \partial_{\mathrm{act}}$, and in `natComplex` the $0 \to 1$ differential $d^{0,1}$ acts as subtype identity on closure elements to yield Equation (87) alongside `physicalReadback` at `stop = start + receipt.1` (Supplement D.6). □

15.2 Explicit Relation Corrections on Free Constant Tables and Zero-Evaluation Certificate Inversion

On a scalar ledger, once $(v_0 + \Delta v)$ is summed, its decomposition disappears forever; if $\Delta v = v_1 + (-v_1) = 0$, no trace remains that two nonzero entries ever occurred. `OriginalSourceConstantTransport` and `ActualSourceConstantInverse` lift constant accounting to the **free $\mathbb{Z}$-module** $\mathbb{Z}^{(G)}$ over the paired group $G := V_s \times V_s$, attaching an invertible integer relation certificate to every constant update.

As the environment steps from Γ to $\Gamma + \delta$, define on $w = \sum_i c_i[e_i] \in F(\mathbb{Z}, G, X; s)$ three maps into $\mathbb{Z}^{(G)}$—$\mathsf{before}(w) := \sum_i c_i[[\![e_i]\!]_\Gamma]$, $\mathsf{after}(w) := \sum_i c_i[[\![e_i]\!]_{\Gamma+\delta}]$, and $\mathsf{effect}(w) := \sum_i c_i[D_{e_i}(\Gamma, \delta)]$—together with the **explicit addition certificate**:

$$\mathsf{addCert}_{\Gamma,\delta}(w) := \sum_i c_i[\mathrm{inl}([\![e_i]\!]_\Gamma,\, D_{e_i}(\Gamma, \delta))], \qquad \mathsf{corr}(w) := \mathrm{relationMap}(\mathsf{addCert}_{\Gamma,\delta}(w)), \tag{88}$$

where relationMap sends addition generator $\mathrm{inl}(u, v)$ to $[u] + [v] - [u + v] \in \mathbb{Z}^{(G)}$. Denote by $\mathcal{W}_{\mathrm{perf}}$ the unified four-face perfection coimage module induced by completed evaluation, equipped with canonical quotient projection $w \mapsto \widehat{w}$ and linear coimage recovery $\mathrm{recover} : \mathcal{W}_{\mathrm{perf}} \to F(\mathbb{Z}, G, X; s)$ satisfying $\mathrm{recover}(\widehat{w}) = w$ (Section 7 of [6]); precomposing with recover lifts all four maps above to $\mathcal{W}_{\mathrm{perf}}$.

Theorem 15.2 (Free Constant-Table Relation Correction and Lossless Zero-Evaluation Certificate Inversion, WA43). For any inquiry frame, base programme, prior inventory, old environment, admission list, and element x of the perfection coimage module $\mathcal{W}_{\mathrm{perf}}$ (denoting the canonical image of the source whole word by $\widehat{w}$):

1. **Exact free-table update and detector pairing**: the computed post-update table constructed solely from old constants and operation effects,

$$\mathsf{computedAfter}(x) := \mathsf{before}(x) + \mathsf{effect}(x) - \mathsf{corr}(x), \tag{89}$$

   equals $\mathsf{after}(x)$ entry-for-entry in $\mathbb{Z}^{(G)}$, recovers `newConstants` of the next origin face at $\widehat{w}$, preserves bilinear pairing differences against every detector $d \in \mathbb{Z}^{(G)}$, and maps under $\mathrm{freeEval} : \mathbb{Z}^{(G)} \to G$ (since $\mathrm{freeEval}(\mathsf{corr}(x)) = 0$) onto the exact next full value and inverse-fibre residual.
2. **Explicit finite integer certificate and lossless inversion**: set $\Delta T := \mathsf{computedAfter}(\widehat{w}) - \mathsf{before}(\widehat{w}) \in \mathbb{Z}^{(G)}$ and $v_{\mathrm{paid}} := \mathrm{freeEval}(\Delta T)$ (matching the residual request's actual paid value). Inducting over the finite support of ΔT constructs an explicit certificate $\mathsf{actualCertificate}$ (retaining the zero scalar relation atom $-[\mathrm{inr}(0, 0)]$ on an empty list rather than discarding it) such that—with zero external kernel-preimage assumption or quotient-section oracle (recursively generating certificates on any finite weighted list, with unordered `Finsupp` support enumeration in standard T1)—in $\mathbb{Z}^{(G)}$:

$$\Delta T - [v_{\mathrm{paid}}] = \mathrm{relationMap}(\mathsf{actualCertificate}), \tag{90}$$

   and the `ExistsEvidence`-eliminated table $\mathsf{restoredTable} := \mathsf{before}(\widehat{w}) + [v_{\mathrm{paid}}] +$

relationMap(actualCertificate) losslessly restores the exact next free constant table alongside whole-ledger canonical next:

$$\mathsf{restoredTable} = \mathsf{computedAfter}(\widehat{w}) = \mathsf{newConstants}. \tag{91}$$

Proof. (1) On a single generator $[e]$, relationMap$(\mathrm{inl}([\![e]\!]_\Gamma, D_e(\Gamma,\delta))) = [[\![e]\!]_\Gamma] + [D_e(\Gamma,\delta)] - [[\![e]\!]_\Gamma + D_e(\Gamma,\delta)]$; substituting Equation (22) ($[\![e]\!]_\Gamma + D_e(\Gamma,\delta) = [\![e]\!]_{\Gamma+\delta}$) and extending $\mathbb{Z}$-linearly gives $\mathsf{before} + \mathsf{effect} - \mathsf{after} = \mathsf{corr}$, i.e., $\mathsf{computedAfter} = \mathsf{after}$. Coimage recovery (`whole_recovery`) identifies $\mathsf{after}(\widehat{w}) = \mathsf{newConstants}$, while freeEval$([u] + [v] - [u+v]) = u + v - (u+v) = 0$ annihilates $\mathsf{corr}(x)$ under freeEval. (2) For any finite weighted list $L = [(g_1, z_1), \ldots, (g_m, z_m)] \in \mathrm{List}(G \times \mathbb{Z})$ with word $W(L) = \sum_k z_k[g_k]$, group sum $V(L) = \sum_k z_k \cdot g_k \in G$, and scalar relation relationMap$([\mathrm{inr}(z,g)]) = [z \cdot g] - z[g]$, define `coefficientRelations` recursively:

- Empty list $L = [\,]$: output $-[\mathrm{inr}(0,0)]$, whose relationMap image is $-([0\cdot 0] - 0[0]) = -[0] = W([\,]) - [V([\,])]$;
- Cons step $(g,z) :: R$: output $-[\mathrm{inr}(z,g)] + \mathsf{cert}(R) + [\mathrm{inl}(z\cdot g, V(R))]$, whose relationMap image telescopes in $\mathbb{Z}^{(G)}$ to

$$-([z\cdot g] - z[g]) + (W(R) - [V(R)]) + ([z\cdot g] + [V(R)] - [z\cdot g + V(R)]) = (z[g] + W(R)) - [z\cdot g + V(R)] = W((g,z) :: R) - [V((g,z) :: R)].$$

Evaluating on ΔT's finite support list gives $W(L) = \Delta T$ and $V(L) = \mathrm{freeEval}(\Delta T) = v_{\mathrm{paid}}$, proving Equation (90)! Substituting Equation (90) into $\mathsf{restoredTable}$ yields in $\mathbb{Z}^{(G)}$:

$$\mathsf{restoredTable} = \mathsf{before}(\widehat{w}) + [v_{\mathrm{paid}}] + (\Delta T - [v_{\mathrm{paid}}]) = \mathsf{before}(\widehat{w}) + \Delta T = \mathsf{computedAfter}(\widehat{w}) = \mathsf{newConstants},$$

with naturality of freeEval matching v_{paid} to the request's actual paid evaluation (Supplement D.7). □

Example 15.3 (Lossless Certificate Inversion Under Exact Zero-Sum Cancellation). Take $G = \mathbb{Z} \times \mathbb{Z}$, $g = (5,0) \in G$, and suppose an update produces two opposite free entries $\Delta T = 1\cdot[g] + 1\cdot[-g] \in \mathbb{Z}^{(G)}$. Numerically, $v_{\mathrm{paid}} = \mathrm{freeEval}(\Delta T) = g + (-g) = (0,0)$—a zero balance! In Theorem 15.2, however, $\Delta T = [g] + [-g] \neq [(0,0)]$, and on $[(g,1),(-g,1)]$ the certificate builder outputs the explicit five-term relation combination:

$$\mathsf{actualCertificate} = -[\mathrm{inr}(1,g)] - [\mathrm{inr}(1,-g)] - [\mathrm{inr}(0,0)] + [\mathrm{inl}(-g,0)] + [\mathrm{inl}(g,-g)],$$

mapping under relationMap to $[g] + [-g] - [(0,0)] = \Delta T - [v_{\mathrm{paid}}]$. Adding $[(0,0)]$ to that image reconstructs $\Delta T = [g] + [-g]$ and `newConstants` without loss! Even when $\Delta T = 0$ (`[]`), the retained zero atom $-[\mathrm{inr}(0,0)]$ maps to $0 - [(0,0)] = -[(0,0)]$, closing the inversion identity with zero exceptions. Compared with $\mathbb{Z}$-relations [20] and DBSP $\mathbb{Z}$-sets [22] ($A \to \mathbb{Z}$): although DBSP provides an exact stream differentiation–integration inverse $\mathcal{I}(\mathcal{D}(s)) = s$ along time ([22], Theorem 2.20), single-step $\mathbb{Z}$-sets and $\mathbb{Z}$-relations still merge opposite multiplicities on the same tuple $+1\cdot a + (-1)\cdot a = 0$ (kept distinct at ordered stock slots in Sections 7–8), and group summation freeEval : $\mathbb{Z}^{(G)} \to G$ collapses opposite entries $+[g] + [-g]$ into ker(freeEval); Theorem 15.2 constructs an explicit integer relation certificate that losslessly inverts that zero-evaluation

kernel fibre.

15.3 Step-Interval Batch Accounting with Anti-Omission/Anti-Duplication Gate and Sealed-Prefix Tail Continuation

Finally, we close two operational pillars of large-scale multi-step continuation: when m source steps emit a batch of downstream requests, what guarantees that no step's items are silently omitted or double-counted? And when a verified deterministic history of length `fuel` is extended to horizon `total >= fuel`, what guarantees zero recomputation of its sealed prefix?

Theorem 15.4 (Contiguous Step-Interval Batch Partition and Nonempty-Group Anti-Omission/Anti-Duplication Gate, WA44). Fix any source configuration family $\mathcal{F}$, admission packet P, and born execution record $\mathcal{E}$ (m paid steps, decoded environment ρ_{dec}, full receiver request batch R_{batch}):

1. **Two-way exact ordinal recognition and whole-batch folding**: For each step index $i \in \mathrm{Fin}(m)$, `stepInterval(i)` carves out a contiguous slice $G_i := (R_{\mathrm{batch}}.\mathrm{drop}\,\mathrm{first}_i).\mathrm{take}\,\mathrm{count}_i$ of length count_i equal to the signed support length of step i's acted word, whose decoded request words sum to that step's affine boundary; conversely, for **every** request ordinal $r \in \mathrm{Fin}(|R_{\mathrm{batch}}|)$, `readRequestOrigin` uniquely recovers its source step $i(r)$, intra-step support slot $s(r)$, signed weight, and paid prefix satisfying the exact address identity:

$$r = \mathrm{first}_{i(r)} + s(r). \tag{92}$$

   Ordered slices $\mathcal{G} = [G_0, G_1, \ldots, G_{m-1}]$ (preserving empty slices at their indices) unconditionally partition the entire request batch, executed trace, and total signed boundary for every $(\mathcal{F}, P, \mathcal{E})$; moreover, on the resolvent–price method instance over Section 14.1's conditional right-half-plane input domain (subject to the same vacuous-truth boundary stated in Section 14.1), summing per-step cochains in the Γ-successor space recovers the joint successor boundary $\partial_{\mathrm{joint}}$ (annihilated under completion):

$$\mathrm{flatten}(\mathcal{G}) = R_{\mathrm{batch}}, \qquad \mathrm{flatten}(\mathrm{map}(\mathrm{exec}_{\rho_{\mathrm{dec}}}, \mathcal{G})) = \mathrm{exec}_{\rho_{\mathrm{dec}}}(R_{\mathrm{batch}}), \qquad \sum_{i=0}^{m-1} \mathsf{stepCochain}(i) = \partial_{\mathrm{joint}}. \tag{93}$$

2. **Nonempty-group anti-omission and anti-duplication gate**: For **any** actual request $r \in \mathrm{Fin}(|R_{\mathrm{batch}}|)$, let $\mathrm{own} := \mathrm{exec}_{\rho_{\mathrm{dec}}}(G_{i(r)})$ be its group's executed trace, with surrounding written-stock prefix and suffix before and after. Then for every $(\mathcal{F}, P, \mathcal{E})$ (and at the conditional audit gate `actual_nonempty_group_rejects_omission_and_duplication`), $|\mathrm{own}| > 0$, the written trace splits across own, and **either omitting or duplicating that group strictly alters the written execution trace**:

$$|\mathrm{own}| > 0, \qquad \mathrm{trace} = \mathrm{before} +\!+ \mathrm{own} +\!+ \mathrm{after}, \qquad \mathrm{trace} \neq \mathrm{before} +\!+ \mathrm{after}, \qquad \mathrm{trace} \neq \mathrm{before} +\!+ \mathrm{own} +\!+ \mathrm{own} +\!+ \mathrm{after}. \tag{94}$$

Proof. (1) Splitting the event list at i into $\mathrm{take}\, i +\!+ [e_i] +\!+ \mathrm{drop}\,(i+1)$ and applying `requests From_append` decomposes $R_{\mathrm{batch}} = \mathrm{prior}_i +\!+ \mathrm{own}_i +\!+ \mathrm{later}_i$, so $\mathrm{first}_i = |\mathrm{prior}_i|$ and $\mathrm{count}_i = |\mathrm{own}_i|$ isolate $G_i = \mathrm{own}_i$ and its affine sum. Conversely, `OrderedTracePartition.locatePart` locates

$(i(r), s(r))$ for any $r < |R_{\text{batch}}|$, and prefix-length cancellation proves Equation (92). Induction over events yields $\text{flatten}(\mathcal{G}) = R_{\text{batch}}$, whence `executedTrace_grouped` and linearity of finite sums on the conditional instance prove all three identities in Equation (93). (2) Given request r, $0 \le s(r) < \text{count}_{i(r)} = |G_{i(r)}|$ proves $G_{i(r)}$ nonempty; every request executes at least one primitive step, forcing $|\text{own}| > 0$. Embedding $\text{exec}(R_{\text{batch}}) = \text{front} +\!\!+ \text{own} +\!\!+ \text{back}$ into the block wrapper gives $\text{trace} = \text{before} +\!\!+ \text{own} +\!\!+ \text{after}$. Taking list lengths: $|\text{before}| + |\text{after}| < |\text{before}| + |\text{own}| + |\text{after}| < |\text{before}| + 2|\text{own}| + |\text{after}|$ whenever $|\text{own}| > 0$, proving both inequalities in Equation (94) unconditionally (Supplement D.8). □

Theorem 15.5 (Prefix-Preserving Tail-Only Horizon Extension and Single-Pass Timeline Update Inventory, WA45).

1. **Zero-recomputation sealed prefix and tail-only continuation**: For any deterministic inquiry runtime, initial state s_0, and stored history $h : \mathsf{HistoryAt}(F, s_0)$ of length $F \in \mathbb{N}$, stored lookup `storedOccurrenceAt` equals live `occurrenceAt` at every $i \in \text{Fin}(F)$; for any horizon $T \ge F$, `extendTo(h, T)` reuses all F stored activations of h verbatim, **executes solely the missing $T - F$ suffix steps from h's reached tail state**, and recovers h as its exact length-F prefix:
$$(\mathsf{extendTo}(h, T)).\mathsf{prefix}(F) = h \qquad (\forall T \ge F). \tag{95}$$
2. **Single-pass micro-step projection table and adjacent-cell update inventory**: For any living-root micro-step process, material readout $\mu : S_P \to M$ (with additive environment map $\mathcal{E} : M \to_+ \text{Env}$), and stored history H of length F, `storedTimeline` projects every state once into a vector $\mathcal{T} \in M^{F+1}$. For every $i \in \text{Fin}(F)$, adjacent cells $(\mathcal{T}[i], \mathcal{T}[i+1])$ equal $(\mu(s_i), \mu(s_{i+1}))$, so induced environments $\mathcal{E}(\mathcal{T}[i])$ and $\mathcal{E}(\mathcal{T}[i+1] - \mathcal{T}[i])$ make the table-driven update inventory equal the live stage update inventory coordinate-for-coordinate:
$$\mathsf{timelineInventory}(\mathcal{T}, i) = \mathsf{stageInventory}(\text{advance}(i)) \qquad (\forall i \in \text{Fin}(F)). \tag{96}$$

   In the Goldbach stored-timeline audit gate, this single table feeds character stage readouts, scalar cochains, fibre-lifting residual recovery, and reached/exhausted terminal dispositions, while adjacent bind cells satisfy $\mathcal{T}[0] \neq \mathcal{T}[1]$ (`exact_bind_cells_are_distinct`).

Proof. (1) Induction on F identifies any stored history with $h = \text{runFrom}(F, s_0)$ (`HistoryAt.eq_runFrom`). `continueFor(h, T-F)` preserves all F `step` constructors and invokes $\text{runFrom}(T-F, s_{\text{tail}})$ solely at the base `exhausted` leaf; casting along $(T-F)+F = T$ and applying `prefix_runFrom` yields $\text{prefix}_F(\text{runFrom}(T, s_0)) = \text{runFrom}(F, s_0) = h$, proving Equation (95). (2) `Nat.dfoldRev` pushes each step state once into a capacity-$(F+1)$ array followed by the final state, inductively proving $(\mathcal{T}[i], \mathcal{T}[i+1]) = (\mu(s_i), \mu(s_{i+1}))$ (`stored_timeline_adjacent`); combining with `stored_pair_actual` ($(s_i, s_{i+1}) = (\text{advance}(i).\text{state}, \text{advance}(i+1).\text{state})$) under $\mathcal{E}$ and `updateInventory` proves Equation (96), while evaluating at unbound coordinate `none` separates $\mathcal{T}[0] \neq \mathcal{T}[1]$ (Figure 6, Supplement D.8). □

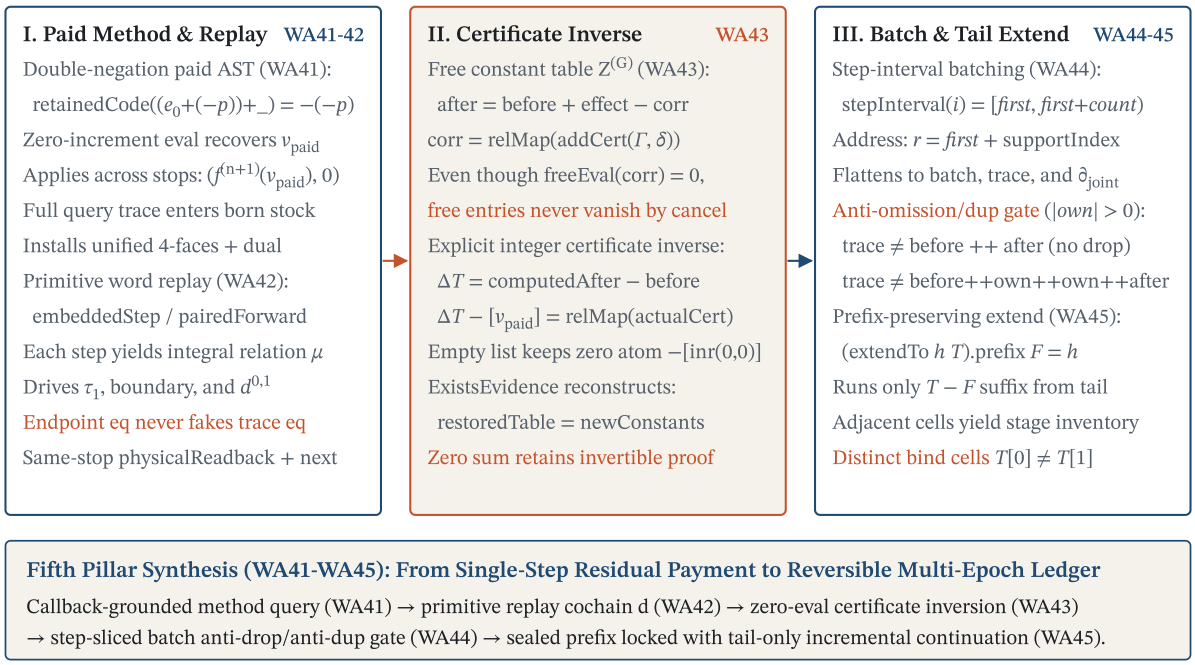


Figure 6 | Paid-Method Continuation, Zero-Evaluation Certificate Inversion, and Batch Continuation. Left column (WA41–WA42, Theorem 15.1): double-negation `retainedCode` recovers the paid AST to apply homomorphism f across cofinal stops with full query traces entering born stock, while `WordExecution` primitive replay drives τ_1 and cochain differential $d^{0,1}$. Center column (WA43, Theorem 15.2, Example 15.3): free constant table $\mathbb{Z}^{(G)}$ retains explicit relation correction `corr` so that even under zero-sum cancellation, `actualCertificate` (retaining empty-list zero atom $-[\mathrm{inr}(0,0)]$) losslessly reconstructs `newConstants` and v_{paid}. Right column (WA44–WA45, Theorems 15.4–15.5): step-interval slicing enforces $r = \mathrm{first} + s$ and rejects dropping or duplicating any nonempty group, while `extendTo` locks the sealed prefix and runs only the missing suffix from the tail state.

16 Five-Pillar Dialogue with Classical Traditions and Conclusion

Provenance-Preserving Conservation Calculus engages five foundational traditions—**mathematical accounting and judgment-proof liability; dependent, substructural, amortised, and quantitative type systems; event structures, state refinement, and observability; disjunctive transition invariants and AGM belief revision; and provenance semirings with static λ-differentiation and $\mathbb{Z}$-set stream calculus**—while drawing sharp positive–negative boundaries along **occurrence lineage, capacity obstruction, integral torsion blindness, law-free two-key settlement, and zero-evaluation certificate inversion** (exact primary-source verification scope recorded in `literature-v2.md`). Table 2 maps all five pillars before Sections 16.1–16.5 examine each seam in detail.

Table 2. Capability Boundaries Between Five Classical Traditions and the Present Five-Pillar Calculus.

Pillar & Core Challenge	Representative Primary Classics & Mechanisms	Classical Scope & Structural Ceiling	**Advances & Negative Controls Here (WA1–WA45)**
Pillar I · Carriers, Events & Scarcity	**Ellerman** [1] Pacioli group $[x//y] \in \mathcal{G}$; **McCarthy** [8] REA events; **Shavell** [9] judgment-proof model	$[x//x] = [0//0]$ erases equal-entry lineage; REA lacks proof-carrying fibre remainders; recoverable damages are bounded by assets	Bijective whole-event admission (WA1–WA3); 3 remainder folds on $\mathcal{L}(s)$; $C_{\text{avail}} < C_{\text{dem}} \iff$ CapacityObstruction**, never converting shortage to discharge** (WA4–WA5)
Pillar II · Type Identity, Safety & Potentials	**Martin-Löf** [10] Σ-types; **HoTT** [11] Univalence; **Necula–Lee** [2] PCC; **Girard** [12]; **Reynolds** [3]; **Hofmann–Jost** [13]; **Atkey** [4][14]	Transporting unindexed carrier types via `ua` conflates provenance; PCC checks single-step safety; frame rule needs disjoint $h_1 * h_2$; AARA & local credit lack whole-ledger obstruction; QTT Lemma 2.3 erases compile-time `0`-usage	T0 zero-axiom [7] source-bound Σ-pairs; coupled split/merge full cover & strict child descent without heap disjointness; **runtime zero effect $\epsilon = 0$ never erases, paying $c(r_m) = 2$ steps**
Pillar III · State Refinement & Torsion Blindness	**Winskel** [15] event structures; **Abadi–Lamport** [16] refinement; **Kalman** [17] $\mathbb{R}/\mathbb{C}$ linear observability	Nondeterministic extensional refinement needs external history & prophecy variables; over $\mathbb{R}/\mathbb{C}$, nonzero $\det(I - E_n) \neq 0$ enables state recovery	On source-prior deterministic ledgers, visits carry history & residuals spawn requests without history/prophecy variables; **invertibility of $D_n \neq 0$ in $\mathbb{C} \otimes_{\mathbb{Z}} C_n$ kills nonzero integral class z across all stages** (WA6, WA11–WA12)

Pillar & Core Challenge	Representative Primary Classics & Mechanisms	Classical Scope & Structural Ceiling	**Advances & Negative Controls Here (WA1–WA45)**
Pillar IV · Termination, Settlement & Revision	**Podelski–Rybalchenko** [18] Transition Invariants (Thm. 1); **AGM** [19] partial-meet contraction $K \dot{-} p \subseteq K$	Transition invariants govern bare state pairs without locking debt lineage or settlement; AGM drops defeasible beliefs via choice in a fixed language	**T0 law-free** $\leq; <; \leq$ **well-foundedness on** X_a **& wash refutation** (WA7); **Key 1 (terminal receipt)** $\times$ **Key 2** $(T(x) = \emptyset)$ (WA8, WA10); **T0 initial coface** $\mathcal{L}_{\text{old}} \sqcup \{\eta\}$ **with zero debt contraction** (WA9)
Pillar V · Semirings, Static Diff & $\mathbb{Z}$-Set Inversion	**Green et al.** [5] $\mathbb{N}[X]$; **Green–Ives–Tannen** [20] $\mathbb{Z}$-relations; **Cai et al.** [21] static Derive(t); **Budiu et al.** [22] DBSP $\mathbb{Z}$-sets	$\mathbb{N}[X]$ tracks static positive queries; Derive(t) gives extensional deltas; single-step $\mathbb{Z}$-relations/$\mathbb{Z}$-sets merge $+1/-1$ to zero and group eval collapses ker(freeEval)	**Residual spawns paid request, 3-branch double negation recovers paid AST for** $f^{(n+1)}$**, primitive replay drives** $\tau_1, d^{0,1}$ (WA38–WA42); $\mathbb{Z}^{(G)}$ **integer certificate (with empty-list zero atom) inverts** ker(freeEval) **& bars slice drop/dup** (WA43–WA45)

16.1 Double-Entry Difference Groups, REA Event Accounting, and Judgment-Proof Scarcity

Ellerman formulates double-entry bookkeeping via the Pacioli abelian group $\mathcal{G}$ of nonnegative pairs modulo $[x//y] = [u//v] \iff x + v = y + u$, clarifying how debit/credit columns represent signed scalar and vector balances without negative numbers [1]; McCarthy's REA model argues that traditional ledger accounts are manual aggregation artifacts and records primary economic events and give–get duality directly in shared databases [8]. Preserving net-balance algebra where sound and sharing REA's event-first insight, our calculus closes three structural gaps across accounting and law-and-economics: first, whereas $[x\,//\,x] = [0\,//\,0]$ collapses equal debits and credits into the identity of $\mathcal{G}$, Equation (46) proves that "zero in the Pacioli group" never implies two-key settlement; second, Theorem 3.4's three generative remainders and Definition 5.3's parent/child coverage equip potentially infinite coupled responsibility fibres with proof-carrying evolution absent from entity–relationship schemas; third, confronting Shavell's judgment-proof problem, in which available assets cap what can be recovered [9], Theorem 3.7 (`ResponsibilityCapacityKernel`) establishes $C_{\text{avail}} < C_{\text{dem}} \iff$ CapacityObstruction at the type level, preserving all affected dependencies and protected boundaries on the ledger and supplying the kernel lock consumed downstream by cross-boundary responsibility retention in our economic-constitution companion [24].

16.2 Type Identity, Proof-Carrying Code, and Substructural, Amortised, and Quantitative Resource Logics

Within type theory and verification, Martin-Löf's intensional Σ-types [10] package support positions with open witnesses; Homotopy Type Theory's Univalence Axiom [11] (Axiom 2.10.3, $(A =_{\mathcal{U}} B) \simeq (A \simeq B)$) promotes type equivalence within a universe to equality—modeling lower ledger readouts as unindexed carrier types and transporting port outcomes across equivalences via `ua` would conflate distinct sources sharing isomorphic lower states while differing in projection laws (violating Section 2.1's anti-shell-borrowing uniqueness). Working inside Lean 4's [7] zero-axiom T0 kernel (in concert with our ontological front-door companion [23]), Definition 2.1 and Proposition 2.5 index every lawful state and receipt over its registered occurrence Occ from the outset, extending Necula and Lee's Proof-Carrying Code [2] from single-snippet receiver safety policies to a fourfold contract binding source occurrence, bijective event admission, two-sided fibre evolution $(d_\varepsilon, o_\varepsilon)$, and canonical next.

On the resource side, Girard's linear logic [12] restricts weakening and contraction on individual context variables, Reynolds's classical separation logic [3] frames disjoint local heaps $h_1 * h_2$, Hofmann and Jost's AARA [13] and Atkey's amortised separation logic [4] allocate nonnegative scalar potentials via linear programming or local credit specifications (including dynamic `acquire`), and Atkey's Quantitative Type Theory (QTT) [14] grades variable usage over an ordered semiring with typing inversion for compile-time `0`-usage (Lemma 2.3: $\Gamma \vdash M \overset{0}{:} S \implies 0\Gamma = \Gamma$). Our calculus advances across four seams: (i) Theorem 3.4 folds an entire open fibre in one occurrence without disjoint-heap assumptions or ad-hoc ghost-state protocols, while Theorems 5.4–5.6 enforce full parent/child cover and strict child descent across coupled splits and merges; (ii) whereas insufficient potential in AARA or local `acquire` in amortised separation logic either blocks static typing or branches on local credit, Theorem 3.7 constructively emits a whole-ledger `CapacityObstruction` retaining all affected dependencies and protected boundaries and empties further maintenance debits at zero budget; (iii) Sections 7 and 9 derive syntactic credit endogenously from the residual AST without external LP annotations or budget refills, isolating fresh debts from prior rows via `inl`/`inr` sum tags; and (iv) Proposition 9.4 proves that runtime zero numerical effect ($\epsilon = 0$) must never be conflated with QTT's compile-time `0`-usage erasure, still paying two concrete same-debt reduction steps.

16.3 Event Structures, State Refinement, and Characteristic-Zero Observability

Winskel's event structures replace state snapshots with causal partial orders and conflict sets [15], while Abadi and Lamport—addressing extensional trace refinement between general specifications whose future nondeterministic choices may be resolved late (complete under machine closure, internal continuity, and finite invisible nondeterminism)—prove that state-space refinement mappings generally require externally adjoined **history variables** and **prophecy variables** [16]. Our calculus pinpoints why **source-prior deterministic emitter ledger continuation** needs neither external auxiliary device: temporal visits in Proposition 2.5 carry arrival histories intrinsically, while Theorems 8.3 and 9.1 form the next writable request directly from the literal residual AST r_{m_n} inside $\mathcal{M}_{\text{fut}}$ with zero representative choice or future branch prophecy. On

the dual observation side, Kalman's classical observability decomposition over $\mathbb{R}/\mathbb{C}$ [17] relies on characteristic-zero linear algebra where nonvanishing block determinants enable state inversion; Theorem 4.2 and Theorems 13.1–13.3 establish the exact arithmetic obstruction over $\mathbb{Z}$: for the prime-3 boundary class z generated by a unit transfer receipt, division-free cofactors enforce $D_n z_n = 0$ ($D_n = \prod(1-p)^2 \neq 0$), so **the very invertibility of D_n in $\mathbb{C}$ annihilates $1 \otimes_{\mathbb{Z}} z_n = D_n^{-1}(1 \otimes D_n z_n) = 0$ at every finite stage**, blinding the entire infinite complexified family F against any linear or nonlinear recovery map.

16.4 Disjunctive Transition Invariants and AGM Belief Contraction

Podelski and Rybalchenko's Transition Invariants (Theorem 1) [18] prove that a program terminates iff its reachable transitive closure satisfies $R^+ \cap (\mathrm{Acc} \times \mathrm{Acc}) \subseteq T_1 \cup \cdots \cup T_n$ for a finite union of well-founded relations T_i. Over multi-party ledgers, however, bare state-pair descent neither stops debts from masquerading as active via $b \to b$ carrier shuffles nor distinguishes a stalled zero-budget state from lawful discharge. Main Theorem 10.3 and Proposition 10.4 establish strict well-foundedness and irreflexivity of $\leq; <; \leq$ macro-steps on the lineage-locked space X_a in **zero axioms with zero global termination law**, while Main Theorem 11.4 and Example 11.6 separately prove that finite settlement requires both a faithful compiler terminal receipt and an empty successor same-debt fibre. Moreover, classical AGM theory change [19] is formulated for **defeasible belief revision inside a fixed propositional language** via an external selection function ($K \dot{-} p \subseteq K$)—treating expressive and realizational bankruptcy on an obligation-bearing ledger as fixed-language belief contraction would trigger a category error by writing off historical debts; Proposition 12.2 and Theorem 12.3 instead generate in zero axioms the categorical initial coface $\mathcal{L}_{\mathrm{old}} \sqcup \{\eta\}$, routing all migrated debts and the new direction through one shared first write with **zero debt contraction**.

16.5 Provenance Semirings, Static λ-Differentiation, and $\mathbb{Z}$-Set Stream Calculus

Database provenance and incremental computation form the closest modern counterparts to Sections 7–9 and 14–15. Green, Karvounarakis, and Tannen factor positive relational queries through universal $\mathbb{N}[X]$ polynomials [5]; to accommodate deletions and relational difference, Green, Ives, and Tannen introduce $\mathbb{Z}$-relations [20], and Budiu et al. build DBSP automatic incremental view maintenance over abelian-group streams (equipped with exact mutually inverse stream differentiation and integration $\mathcal{I}(\mathcal{D}(s)) = s$, Theorem 2.20) and $\mathbb{Z}$-sets $A \to \mathbb{Z}$ (whose bilinear delta formula in Theorem 3.4 shares the exact three-term algebra of our Equation (22)) [22]; for higher-order programs, Cai et al. establish static λ-differentiation Derive(t) with extensional change soundness (Theorem 3.11) [21].

Retaining that exact finite-difference algebra, our calculus crosses three algebraic frontiers orthogonal to stream time-integration: first, Theorems 9.1–9.3 turn the all-future residual into an endogenous writable request that pays its own same-debt node budget and delivers four typed outputs upon suffix completion; second, Theorem 15.1 (WA41–WA42) uses three-branch double negation `retainedCode` to recover the paid AST $(-\mathrm{id})((-\mathrm{id})(p_\star))$ across cofinal stops for $f^{(n+1)}$ while nine-constructor primitive replay drives chain morphism τ_1 and cochain differential $d^{0,1}$

without equating traces by endpoint values; third, notwithstanding DBSP's time-stream inverse $\mathcal{I} = \mathcal{D}^{-1}$, single-step $\mathbb{Z}$-sets and $\mathbb{Z}$-relations still merge opposite multiplicities on the same tuple $+1 \cdot a + (-1) \cdot a$ to zero (kept distinct at ordered stock slots in Sections 7–8), and group summation freeEval : $\mathbb{Z}^{(G)} \to G$ collapses opposite entries $[g] + [-g]$ into ker(freeEval)—Theorems 15.2–15.5 (WA43–WA45) construct explicit integer certificates `actualCertificate` (retaining empty-list zero atom $-[\mathrm{inr}(0,0)]$) in $\mathbb{Z}^{(G)}$ to invert that zero-evaluation kernel fibre without loss while locking contiguous per-step slices and sealed prefixes against drops, duplicates, or recomputation.

16.6 Conclusion: You May Owe It, But You Cannot Pretend You Paid It

The zero on the opening line is still there—now flanked by a mechanically checkable architecture of provenance and continuation.

Bijective event-inventory admission refuses to trim continuing exits; open responsibility fibres lose not a single row when omitted from a local patch; capacity shortages emit positive obstruction witnesses rather than a capacity-based write-off; equal-budget transfers and split/merge shuffles wash away no lineage; invertible complex determinants vanishing across every finite stage cannot sign off on a nonzero integral class; and final settlement demands both a faithful compiler terminal receipt and an empty same-debt fibre in the actual successor. And whenever obligations remain open and computation must go on, the ledger needs neither prophecy variables nor an external oracle to push it forward: out of its own all-future residual it grows the next request, pays the same debt node by node, recovers its paid AST via double negation for the next homomorphism, inverts zero-sum cancellations via explicit integer relation certificates, slices next-batch requests step by step against drops or duplicates, and locks its sealed prefix to continue strictly from the tail.

You may leave a debt unpaid, never pretend it was settled; and when the ledger continues, it pays every step itself.

Appendix A Shared Theorems with the Process-Algebra Companion and Independent Proofs

The published second edition of the process-algebra companion [6] supplies foundational process identity, single-step primitive evaluation, and the maximal future model; this paper preserves those theorem identities while independently proving whole-event ledger conservation, capacity-shortage obstruction, two-key settlement, endogenous residual same-debt payment, long-window trace relay, zero-evaluation certificate inversion, and step-grouped continuation (**WA1**–**WA45**).

Location Here	Named Shared Results from Companion [6]	Independent Main Theorems and Controls Proved Here (WA1–WA45)
Secs. 2–3	Authoritative roots, temporal visits, canonical next; Prop. 6.1	Bijective event admission & terminal exhaustion (Thm. 2.3, Ex. 2.4, WA1–WA3); 3 remainder folds, rowwise total disposition, and **capacity-shortage obstruction & source-bound debit bounds** (Thms. 3.2–3.7, WA4)
Secs. 4, 13	Unit arithmetic root and base representation interface	Consumer-relative observer safety & `FullTraceRedirect` (Thm. 4.2, Ex. 4.3, WA6); 4-way receipt residual classification (Prop. 4.4, WA12); prime-3 nonzero class & **unrecoverability from all finite complex observations** (Thms. 13.1–13.3, WA11)
Secs. 5, 7	Prop. 3.7; Thm. 6.10, Supps. S0–S1	Full parent/child split/merge cover, zero-budget split bar & 2-Bool wash counterexample (Thm. 5.4, Ex. 5.5, WA5); full-type certificate & remainder lifting (Thm. 5.6, WA15–WA17); owner-free whole-ledger debit & exact coefficient/substitution/$7n + 5/\Sigma_n + 2(n + 1)$ charges (Thm. 7.2, Props. 7.3–7.4, WA13–WA14, WA18–WA19, WA21, WA27–WA28)
Secs. 6, 10–12	Base layer of Thm. 6.7; Thms. 8.4, 8.6–8.7, Prop. 8.9	Target-6 end-to-end repair & settlement (Thm. 6.2, WA10); **law-free** $\leq$; $<$; $\leq$ **macro-step well-foundedness** & wash refutation (Main Thm. 10.3, Prop. 10.4, WA7); **two-key settlement**, fluid zero-budget counterexample & stage-3 instance (Main Thm. 11.4, Cor. 11.5, Exs. 11.6–11.7, WA8); shared first-write revision continuity (Prop. 12.2, Thm. 12.3, Ex. 12.4, WA9)

Location Here	Named Shared Results from Companion [6]	Independent Main Theorems and Controls Proved Here (WA1–WA45)
Secs. 8–9	Thms. 6.12, 7.7, 8.12, 8.14–8.15, 9.5–9.12; Supps. S1–S5	Cross-language `HEq` stock readback, monotone injection $\iota(j)$, same-receipt total writeback, difference stock & full-segment cofinal stops (Thms. 8.1–8.3, Prop. 8.2, WA20, WA22–WA26, WA29–WA32, WA34–WA37); **all-future residual request, positive-credit same-debt debit,** $(Q-1)$**-step 4-port completion & zero-effect controls** (Thms. 9.1–9.3, Prop. 9.4, WA38–WA39)
Sec. 14	Base evaluator, paired language, Thm. 9.12 future kernel	Resolvent-comb installation (WA33); conditional right-half-plane 4-trace window stock, 6-trace inverse-residual correction, source-price difference & joint quotient $\mathcal{Q}_a$ (Thms. 14.1–14.2, WA40)
Sec. 15	Base method & stored-history primitives (C51–C54)	**Double-negation paid-AST homomorphism continuation & primitive replay driving** $\tau_1, d^{0,1}$ (Thm. 15.1, WA41–WA42); **free constant-table relation correction & zero-evaluation integer certificate inversion** (Thm. 15.2, Ex. 15.3, WA43); **step-interval anti-drop/anti-dup gate & prefix-preserving tail continuation** (Thms. 15.4–15.5, WA44–WA45)

Appendix B Core Ledger Object Contracts, Capacity Gates, and Units

Ledger Object or Gate	Source-Generated Evidence Required to Inhabit It	Shortcut Provably Reduced to the Empty Type
Faithful Local Terminal	Bijective whole-event presentation + independent ordinary outgoing-edge exhaustion	Deleting a coexisting write event and declaring termination on a singleton (Ex. 2.4)
Rowwise Total Disposition	Same whole-ledger compilation + in-place, full finite, or transported remainder	Treating open rows omitted from a sparse local patch as vanished (Thms. 3.4–3.5)
Capacity-Shortage Obstruction	Readout $\mathcal{R}$ with $C_{\mathrm{avail}}(\mathcal{R}) < C_{\mathrm{dem}}(\mathcal{R})$ retaining all admitted rows, dependencies & boundaries	Writing off historical obligations or pruning branches due to scarce capacity (Thm. 3.7)
Source-Bound Maintenance Debit	Lifecycle edge equal to `.maintained` on a live slot + lineage lock + strict budget drop	Detaching a bare natural-number countdown from concrete slots, or debiting at zero budget (Thm. 3.7)

Ledger Object or Gate	Source-Generated Evidence Required to Inhabit It	Shortcut Provably Reduced to the Empty Type
Paid Same-Debt Macro-Step	Actual process successor + unique same-debt target + non-refilling pre/post enclosing $B(u') < B(u)$	Passing off $B(u') = B(u)$ transfers, bearer swaps, or renamings as payment (Prop. 10.4)
Two-Key Final Settlement	**Key 1** faithful local terminal $\mathrm{LocalTerm}(x)$ + **Key 2** empty successor fibre $\mathrm{IsEmpty}(T(x))$	Inferring settlement from zero budget alone or skipping successor inspection (Prop. 11.2, Ex. 11.6)
Zero-Eval Constant Inversion	Free $\mathbb{Z}^{(G)}$ tables + explicit integer relation certificate `actualCertificate` (retaining $-[\mathrm{inr}(0,0)]$)	Erasing free entries upon zero-sum cancellation so `newConstants` becomes unrecoverable (Thm. 15.2, Ex. 15.3)
Batch Slice & Horizon Extend	Contiguous per-step address $r = \mathrm{first} + s$ + sealed prefix identity `extendTo_prefix`	Omitting or duplicating any nonempty step slice (Thm. 15.4), or recomputing/altering a sealed prefix (Thm. 15.5)

Strict Separation of Units. Syntactic budgets count AST primitive nodes $c(a) \in \mathbb{N}$; lifecycle same-debt budgets $b(e) \in \mathbb{N}$ are read from open witnesses; capacity readouts $(C_{\mathrm{avail}}, C_{\mathrm{dem}})$ carry domain carrying units; and free constant tables take coefficients in $\mathbb{Z}$. Syntactic completion, strict same-debt debit, capacity obstruction, and two-key settlement maintain distinct types and receipts across one coherent chain without collapsing into a single scalar.

Code and Data Availability

The public proofs for WA1–WA45 are fixed at H0mework commit `5f94af7e988c9831c92e54fe89747699fa29cd85`. WA1–WA40 use the first-edition map and its 13 historical environments (material commit `f607eb646d4e9cddba3512755365842895af44ca`), with 5,025 complete declaration types and the original acceptance records. Example 11.7 uses the Y1 elimination proof repair.

The second-edition map records `ResponsibilityCapacityKernel.lean`, WA41–WA45, their direct consumers, and original receipts. The research sources are `df421ff600085cbc290a7ae946193961bed7c83b` for WA41–WA43 and `7e4da746f300b3796b8b18afb26cb0765c29afcb` for WA44–WA45. The local acceptance record binds six proof package builds, trust checks, and 177 complete declaration types. The editable archive records the bilingual anchors and fixed declarations in `release-selection-v2.json`.

Run `python3 tools/whole_ledger_v2.py entry` at the H0mework repository root. See the reproduction guide for the toolchain, strict builds, and source recovery. Earlier shared theorems cite the process core second edition 10.5281/zenodo.23297175. For C51–C54 in the third edition of the process core, see DOI 10.5281/zenodo.24147130. The first edition of this paper is 10.5281/zenodo.23437454. Original code is Apache-2.0. Paper text, proof supplements, and original figures are CC BY 4.0.

References

[1] David Ellerman. *On Double-Entry Bookkeeping: The Mathematical Treatment*. Accounting Education, 23(5), 483–501, 2014. DOI: 10.1080/09639284.2014.949803. Author's preprint.

[2] George C. Necula and Peter Lee. *Safe Kernel Extensions Without Run-Time Checking*. Proceedings of OSDI, 229–243, 1996. DOI: 10.1145/238721.238781. USENIX paper.

[3] John C. Reynolds. *Separation Logic: A Logic for Shared Mutable Data Structures*. Proceedings of LICS, 55–74, 2002. DOI: 10.1109/LICS.2002.1029817. Author's paper.

[4] Robert Atkey. *Amortised Resource Analysis with Separation Logic*. Logical Methods in Computer Science, 7(2:17), 2011. DOI: 10.2168/LMCS-7(2:17)2011. Author's paper.

[5] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. *Provenance Semirings*. Proceedings of PODS, 31–40, 2007. DOI: 10.1145/1265530.1265535. Author's paper.

[6] Jian Gao. *The State Is Not the History, the Source Is: Identity, Responsibility, and Minimal Revision in Processes*. Version 2, 2026. DOI: 10.5281/zenodo.23297175. Main text and complete proof supplement.

[7] Leonardo de Moura and Sebastian Ullrich. *The Lean 4 Theorem Prover and Programming Language*. Automated Deduction – CADE 28, Lecture Notes in Computer Science, 12699, 625–635, 2021. DOI: 10.1007/978-3-030-79876-5_37. Paper PDF.

[8] William E. McCarthy. *The REA Accounting Model: A Generalized Framework for Accounting Systems in a Shared Data Environment*. The Accounting Review, 57(3), 554–578, 1982. JSTOR stable.

[9] Steven Shavell. *The Judgment Proof Problem*. International Review of Law and Economics, 6(1), 45–58, 1986. DOI: 10.1016/0144-8188(86)90038-4.

[10] Per Martin-Löf. *Intuitionistic Type Theory* (Notes by Giovanni Sambin). Bibliopolis, Naples, 1984. Retypeset text.

[11] The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, Princeton, 2013. Book home.

[12] Jean-Yves Girard. *Linear Logic*. Theoretical Computer Science, 50(1), 1–101, 1987. DOI: 10.1016/0304-3975(87)90045-4.

[13] Martin Hofmann and Steffen Jost. *Static Prediction of Heap Space Usage for First-Order Functional Programs*. Proceedings of POPL, 185–197, 2003. DOI: 10.1145/604131.604148.

[14] Robert Atkey. *Syntax and Semantics of Quantitative Type Theory*. Proceedings of LICS, 56–65, 2018. DOI: 10.1145/3209108.3209189. Author's paper.

[15] Glynn Winskel. *Event Structures*. Petri Nets: Applications and Relationships to Other Models of Concurrency, Lecture Notes in Computer Science, 255, 325–392, 1987 (Univ. Cambridge Tech. Rep. UCAM-CL-TR-95, 1986). DOI: 10.1007/3-540-17906-2_31. Cambridge report.

[16] Martín Abadi and Leslie Lamport. *The Existence of Refinement Mappings*. Theoretical Computer Science, 82(2), 253–284, 1991. DOI: 10.1016/0304-3975(91)90224-P. Author's paper.

[17] Rudolf E. Kalman. *Mathematical Description of Linear Dynamical Systems*. Journal of the Society for Industrial and Applied Mathematics, Series A: Control, 1(2), 152–192, 1963. DOI: 10.1137/0301010.

[18] Andreas Podelski and Andrey Rybalchenko. *Transition Invariants*. Proceedings of LICS, 32–41, 2004. DOI: 10.1109/LICS.2004.1319598.

[19] Carlos E. Alchourrón, Peter Gärdenfors, and David Makinson. *On the Logic of Theory Change: Partial Meet Contraction and Revision Functions*. The Journal of Symbolic Logic, 50(2), 510–530, 1985. DOI: 10.2307/2274239.

[20] Todd J. Green, Zachary G. Ives, and Val Tannen. *Reconcilable Differences*. Proceedings of ICDT, 212–224, 2009. DOI: 10.1145/1514894.1514920.

[21] Yufei Cai, Paolo G. Giarrusso, Tillmann Rendel, and Klaus Ostermann. *A Theory of Changes for Higher-Order Languages: Incrementalizing λ-Calculi by Static Differentiation*. Proceedings of PLDI, 145–155, 2014. DOI: 10.1145/2594291.2594304. arXiv:1312.0658.

[22] Mihai Budiu, Tej Chajed, Frank McSherry, Leonid Ryzhyk, and Val Tannen. *DBSP: Automatic Incremental View Maintenance for Rich Query Languages*. Proceedings of the VLDB Endowment, 16(7), 1601–1614, 2023. DOI: 10.14778/3587136.3587137. arXiv:2203.16684. PVLDB paper.

[23] Jian Gao. *Ontological Rigidity and Universal Single-Mother Formation: Axiom-Free Exclusion of Unaccounted Differences, Causal Admission, and Unified Evolution*. Second Edition, 2026. DOI: 10.5281/zenodo.24135399 (First Edition DOI: 10.5281/zenodo.23437456).

[24] Jian Gao. *An Economic Constitution of Possibility: Producing Real Freedom, Settling Contributions, and Renewing Authority*. Second Edition, 2026. DOI: 10.5281/zenodo.23917902.